Ethernet Gateway

User Manual

Version 0.9.3

Ricky Bryce

A telnet/SSH file-transfer gateway, SSH proxy, Hayes-compatible modem emulator (with optional telnet-serial console bridge), text-mode web browser, AI chat client, and weather service for retro hardware — XMODEM, XMODEM-1K, YMODEM, ZMODEM, Kermit, and Punter over PETSCII, ANSI, and ASCII terminals.

Disclaimer

This software is provided on an "as is" basis, without warranties of any kind, express or implied. Use at your own risk. The author is not responsible for any data loss, security breaches, or damages resulting from the use of this software. The user is solely responsible for securing their own network, credentials, and data. Telnet is an inherently insecure protocol — do not use this software on untrusted networks.

Portions of this project were developed with the assistance of AI tools, principally Claude Code (Anthropic) with ChatGPT (OpenAI) alongside it.

Table of Contents

1. Introduction

Ethernet Gateway is a standalone server written in Rust that bridges retro hardware and modern networks. It accepts telnet and SSH connections, speaks the XMODEM / XMODEM-1K / YMODEM / ZMODEM / Kermit / Punter file-transfer protocols, emulates a Hayes AT modem on **two physically independent serial ports** (each port also flips to a telnet-serial console bridge on demand), proxies outbound connections to remote telnet and SSH servers, renders web pages as plain text, looks up the weather, and provides a terminal AI-chat front end for the Groq API.

It was designed to let 40-year-old computers use 40-minute-old networks. The telnet interface is the headline feature because it works out of the box with any machine that can speak a serial stream: Commodore 64, CP/M systems, the AltairDuino, RC2014 / SC126, and countless other homebrew boards. Once the server is running on a PC, connect from anywhere on your LAN:

```
telnet 192.168.1.160 2323
```

Or from a serial-attached retro machine, dial through the emulator:

```
ATDT 192.168.1.160:2323
```

1.1 Supported Terminal Types

On every inbound connection the gateway asks the user to press `Backspace` so it can auto-detect the terminal. The byte received uniquely identifies the client:

Byte	Terminal	Typical hardware
`0x14`	PETSCII	Commodore 64, VIC-20, C128 in 40-column mode
`0x08` or `0x7F`	ANSI	Modern terminal emulators, PuTTY, xterm, macOS Terminal
anything else	ASCII	Dumb terminals, line printers, bridge boards without colour

After terminal detection the user is asked whether to enable colour. There is no default — the user must explicitly press `Y` or `N`. Colour output uses PETSCII colour codes on the Commodore, ANSI escape sequences on modern terminals, and plain text on ASCII.

1.2 Feature Overview

- **File transfer** — XMODEM (128-byte), XMODEM-1K, YMODEM, ZMODEM (with autostart), Kermit (incl. standalone server mode), and Punter C1 (Commodore 64/128), with auto-detection on upload and a selection prompt on download.
- **Gateway Shell** — a CP/M-inspired `A>` file-manager prompt over telnet/SSH (DIR, TYPE, COPY, MOVE, ERA, REN, MKDIR, …), operating on the transfer directory as drive A: (no Z80 emulation; chapter 8.10).

- **CP/M emulator** — a `K` main-menu item (on by default) that runs a real CP/M 2.2 environment on an emulated Z80 — or 8080, see `cpm_cpu` — (the `iz80` core), executing user-supplied `.COM` software (PIP, STAT, ASM, WordStar, …) sandboxed to `CPM/A` – `CPM/H` under the transfer directory (chapter 8.11).
- **Hayes AT modem emulator** — two physically independent serial ports (Port A and Port B), each with its own enabled flag, mode, baud, AT/S-register state (S0–S26), and stored phone-number slots, plus the `+++` escape.
- **Telnet-serial console bridge** — either port can be flipped to *console mode* per-port, so a telnet/SSH user can drive a microcontroller or RS-232 device directly through the **G Serial Gateway** menu (chapter 9.13). Both ports can host bridges concurrently.
- **SSH server** — optional encrypted access with its own credentials and a persistent Ed25519 host key.
- **SSH gateway** — outbound proxy for SSH servers, with TOFU host-key trust and Ed25519 public-key auth.
- **Telnet gateway** — outbound proxy with RFC-1143 Q-method option negotiation or a raw-TCP escape hatch.
- **Web browser** — text-mode HTTP/HTTPS/Gopher with forms, bookmarks, and 40-column-friendly layout.
- **AI chat** — Groq API, `llama-3.3-70b-versatile`, paginated answers wrapped to the terminal width.
- **Weather** — current conditions and a 3-day forecast from Open-Meteo for any city or postal code worldwide (MET Norway fallback).
- **GUI console** — eframe/egui configuration editor with live server log.

> **Security notice**
>
> Telnet transmits all data, including passwords, in cleartext. The telnet interface is intended for *local or private network use only*. Use the SSH interface for anything leaving your LAN, and keep `egateway.conf` at mode `0600` because it contains plaintext credentials.

2. Installation

2.1 Linux AppImage (pre-built)

The simplest installation on a modern Linux distribution is the pre-built AppImage that ships with every release:

```
chmod +x Ethernet_Gateway-x86_64.AppImage
./Ethernet_Gateway-x86_64.AppImage
```

AppImages bundle their own libraries and run on any mainstream glibc-based distribution without installation. On first run the AppImage creates `egateway.conf` in the current working directory.

> **Verification**
> Each release publishes a SHA-256 checksum, an optional GPG signature, and a Sigstore keyless signature bound to the publisher's GitHub identity. Verify at least the SHA-256 before running any binary from the internet: `sha256sum -c ethernetgateway-v0.9.3-…tar.gz.sha256`.

2.2 Building from Source

The project is written in pure Rust using the 2024 edition. The minimum supported compiler is Rust **1.92**.

2.2.1 Debian 13 / Ubuntu

```
sudo apt update
sudo apt install -y build-essential pkg-config cmake curl libudev-dev
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
```

2.2.2 Fedora / RHEL / AlmaLinux

```
sudo dnf install -y gcc gcc-c++ make cmake pkg-config curl
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
```

2.2.3 Arch Linux

```
sudo pacman -S --needed base-devel cmake curl
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
```

2.2.4 macOS

```
xcode-select --install
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
brew install cmake
```

2.2.5 Windows

1. Run the rustup installer from `https://rustup.rs`.
2. When prompted, install the Visual Studio C++ Build Tools.
3. Install CMake from `https://cmake.org/download/` or via `winget install Kitware.CMake`.
4. Open a fresh PowerShell or cmd.exe so rustup is on your `PATH`.

2.2.6 Verify the toolchain

```
rustc --version    # should show 1.92 or later
cargo --version
cmake --version
```

2.2.7 Compile

```
cargo build --release
```

The resulting binary is at `target/release/ethernetgateway`. The first full build pulls a couple of hundred crates and may take 3–8 minutes depending on your machine; subsequent incremental builds are quick.

2.2.8 Key dependencies

The gateway pulls the following well-known crates as direct dependencies: `tokio` (async runtime), `russh` (SSH server and client), `serialport` (cross-platform serial I/O), `ureq` (blocking HTTP client used by the browser and AI chat), `html2text` (HTML to plain-text rendering), `url` (URL parsing for the browser), `eframe` / `egui_extras` (GUI), `rfd` (native file-open/save dialogs), `arboard` (clipboard access), `signal-hook` (POSIX signals), and `serde_json`. Full transitive list is in `Cargo.lock`.

2.3 Running as a systemd Service (Linux)

A hardened unit file is bundled at `contrib/systemd/ethernetgateway.service`. To install:

```
# Create a dedicated unprivileged service user
sudo useradd --system --home-dir /var/lib/ethernetgateway \
             --shell /usr/sbin/nologin ethernetgateway
sudo install -d -m 0750 -o ethernetgateway -g ethernetgateway \
             /var/lib/ethernetgateway

# Install the binary
sudo install -m 0755 target/release/ethernetgateway /usr/local/bin/

# Install and enable the unit
sudo install -m 0644 contrib/systemd/ethernetgateway.service \
             /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now ethernetgateway.service

# Watch the log
journalctl -u ethernetgateway -f
```

The unit ships with defensive hardening: `NoNewPrivileges`, `PrivateTmp`, `ProtectSystem=strict`, `ProtectHome`, namespace restrictions, `SystemCallFilter=@system-service`, an empty capability bounding set, a 512 MiB memory cap, and `TasksMax=256`. The service user has no shell, no home directory, and write access only to `/var/lib/ethernetgateway`.

If you want to listen on privileged ports (telnet on 23 or SSH on 22), uncomment the `CapabilityBoundingSet=CAP_NET_BIND_SERVICE` and matching `AmbientCapabilities` lines in the unit.

`systemctl reload ethernetgateway` sends SIGHUP, which the binary handles as a graceful stop. Restart the service to pick up config changes.

3. First Run & Configuration File

On the very first launch the gateway looks for `egateway.conf` in the current working directory. If the file does not exist it is created with sensible defaults and a short log message is printed. On subsequent launches the file is read, every missing key is filled in with its default, and the file is rewritten so that future editors see the full set of keys and comments.

> **UNIX PERMISSIONS**
> `egateway.conf` holds the plaintext login password (shared by telnet, SSH, and the web UI) and the Groq API key. On Unix the file is set to mode `0600` (owner-only read/write) on every write. The same applies to `dialup.conf`, `gateway_hosts`, `ethernet_ssh_host_key`, and `ethernet_gateway_ssh_key`.

3.1 The First-Run Setup Wizard

On a genuinely fresh install the desktop GUI does not open into the configuration editor — it opens into a setup wizard that walks through the decisions a new gateway needs, in plain language, and ends by telling you how to connect and which ports to open. It is the fastest path from a first launch to a working gateway, and it is entirely optional: *Skip setup — use defaults* on the first screen leaves every default in place.

> **GUI ONLY — BY DESIGN**
> The wizard exists only in the desktop console. The telnet/SSH menu and the web UI already expose every key it touches, and this is initial setup at the machine, not a third configuration surface to keep in sync. Consequently a headless install (`enable_console = false`) never sees it; the wizard simply appears the first time someone does open the GUI.

The screens, in order:

1. **Username and password** — typed twice; the wizard will not advance until they match, and a checkbox reveals what you are typing. It states plainly that the password is stored in cleartext in `egateway.conf`. These are the unified credentials used by telnet, SSH, and the web UI.
2. **Telnet server** — enable/disable and port (default `2323`), with the cleartext warning.
3. **SSH server** — enable/disable and port (default `2222`). SSH uses the credentials from screen 1 and always authenticates, whether or not you require a login for telnet.
4. **Access control** — the three checkboxes (`security_enabled`, `disable_ip_safety`, `disable_gateway_connections`), each with a paragraph explaining what it actually changes. The router checkbox names the address this machine actually routes through, read from its own routing table.
5. **Web configuration server** — enable/disable and port (default `8080`), plus the URL you'll use.
6. **File transfer directory** — typed or chosen with a folder browser. The CP/M emulator's drives live in a `CPM` subdirectory of it, and the Gateway Shell treats it as drive A:.

7. **CP/M emulator** — whether to enable it, and whether to give it the virtual modem it can dial out with (`cpm_emu_uart`). Notes that EGT8080.COM is placed on drive A: for you, with EGT80.COM — the Z80 build, for Z180 ASCI consoles — beside it. It also offers, ticked to begin with, to **download the sample CP/M disks** — the same set the *Download sample disks* button on every disk screen fetches (§6.4.3), and the same one implementation, so the two offers cannot disagree about what they fetch. The screen says how many disks, how many megabytes, and which repositories they come from before you decide; the disks are not ours, and this fetches them on your behalf, so untick it if you would rather fetch them yourself or not at all. Ticking it is not a setting and nothing about it is written to `egateway.conf`: the download starts once you press *Save and Restart Server*, lands in the transfer directory you chose on the previous screen, and leaves any file already in the images folder alone. It runs in the background, so the window stays usable, and the console line tells you how it went.
8. **Gateway role** — Standalone, Master, or Slave, with a description of each. Choosing Master arms `master_accept_relays`. It does *not* silently switch the SSH server on: if you turned SSH off on step 3, the screen explains that a slave links to its master by logging into the master's SSH server — so with SSH off no slave can ever connect — and offers a button to enable it. Decline and the role is still saved; the review screen and every startup then carry the warning. Choosing Slave adds one more screen for the master's host, SSH port, username, and password.
9. **Review and finish** — see below.

Every screen has *Back*, *Next*, and an exit that keeps your current settings. Nothing is written to `egateway.conf` or to the running server until the final *Save and Restart Server* button: the wizard edits its own draft throughout, and exiting or skipping writes exactly one key (below). Settings the wizard never asks about keep their existing values.

Next also validates. It refuses an empty username or password, mismatched passwords, a port that isn't a number from 1 to 65535, and a port already claimed by another listener — including the standalone Kermit server, which the wizard doesn't ask about but which this process will still bind.

The final screen summarises what will be saved, then prints the actual commands for the configuration you chose — `telnet 192.168.1.50 2323`, `ssh -p 2222 admin@192.168.1.50`, `http://192.168.1.50:8080/` — followed by the **inbound TCP ports to allow on your firewall**, one line per listener with what each is for. (All are TCP; the gateway binds no UDP ports, and the serial features use no network ports at all. A slave needs no inbound rule of its own — it dials out to its master.) It also warns, without blocking, about choices worth a second look: both telnet and SSH left off, IP safety off with no login required, or a port below 1024 that will need root. Finally it points out that serial ports are *not* configured here — enable and configure Port A / Port B afterwards in the GUI's Serial Port frames or over telnet.

Key	Type	Default	Description
`setup_wizard_completed`	bool	`false`	The wizard appears only while this is `false`. Set to `true` when the wizard is completed *or* skipped. Note the deliberate asymmetry: a config file that *lacks* the key entirely is read as `true`, so upgrading an already-configured gateway never drops it into the wizard — only a config file the gateway creates itself carries `false`.
`open_screen_after_restart`	bool	`false`	Not a setting — a one-shot marker the desktop UI leaves for itself. Turning the web server on from the **VDM / Dazzler** button restarts the gateway, and this is how the window that comes back knows to finish opening the screen you asked for. It is cleared the moment it is read, so a launch that never manages to open a browser does not keep trying. Setting it by hand simply opens the screen once on the next start. No other surface shows it.

To run the wizard again later, open the GUI's *Server — More…* window and click *Run setup wizard…* (§4.2.1); setting `setup_wizard_completed = false` by hand does the same thing on the next launch. A re-run pre-fills every answer from your current configuration except the password, which is never echoed back and must be retyped.

3.2 File Format

The file is a simple `key = value` text file. Blank lines and lines starting with `#` are comments and ignored. Values are trimmed of surrounding whitespace. Unknown keys are silently ignored, so you can annotate your own notes as needed. Boolean values accept `true` / `false` in any case.

```
# Ethernet Gateway Configuration
telnet_enabled = true
telnet_port = 2323
ssh_enabled  = false
security_enabled = false
```

3.3 Configuration Key Overview

The table below summarises every key recognised by the parser. Full details including sanitisation, range checks, and live-edit behaviour are in Appendix A. The reserved-zero S-registers and stored phone-number slots are covered in chapter 9.

Key	Type	Default	Description
`telnet_enabled`	bool	`true`	Run the telnet server. Set `false` for SSH-only mode.
`telnet_port`	u16	`2323`	Telnet listen port. Must be ≥ 1.
`telnet_gateway_negotiate`	bool	`false`	Outbound gateway sends proactive `WILL TTYPE` / `WILL NAWS` / `DO ECHO`. Ignored when `telnet_gateway_raw` is set — raw TCP has no IAC layer to negotiate over — so every config surface greys this control in raw mode and leaves its stored value alone.
`telnet_gateway_raw`	bool	`false`	Outbound gateway bypasses all IAC parsing. Supersedes negotiate.
`gateway_debug`	bool	`false`	Byte-level trace of SSH/Telnet gateway sessions, and of every key typed at a session prompt. Passwords are excluded: the trace is suppressed for the duration of any password prompt, so enabling this cannot put a credential in the log. Toggle from the GUI/web General frame or the telnet Other / Serial Configuration menus; takes effect next session. `EGATEWAY_GATEWAY_DEBUG` env var forces it on. See §16.10.
`gateway_term_width`	u16	`0`	Columns reported to the remote host by both gateways (SSH PTY request / telnet NAWS). `0` = automatic: your client's NAWS if it sent one, else the terminal-type default. Set it when a client cannot report its real width — see §16.14.
`gateway_term_height`	u16	`0`	Rows counterpart of `gateway_term_width`. `0` = automatic. Resolved independently, so you can pin the width and leave the rows automatic.
`enable_console`	bool	`true`	Open the GUI at startup. Set `false` for headless systemd operation.
`setup_wizard_com`	bool	`false`	Set once the GUI's first-run setup wizard has been completed or skipped; the wizard appears only while it is `false`. A config file missing the key reads as `true` (an upgrade is never sent through the wizard). See §3.1.

Key	Type	Default	Description
pleted			
open_screen_after_restart	bool	false	A one-shot marker, not a setting: the desktop UI sets it when enabling the web server from the VDM / Dazzler button so the screen opens after the restart. Cleared when read. See §8.11.
security_enabled	bool	false	Require username/password login on telnet, SSH, and the web UI.
username, password	string	admin, changeme	Unified login credentials — shared by telnet, SSH, and the configuration web UI. (The legacy per-protocol ssh_username / ssh_password keys were removed in v0.6.0; configs that still have them auto-migrate into this pair on first load when the unified pair is still at the factory defaults.)
ssh_enabled	bool	false	Run the SSH server on ssh_port.
ssh_port	u16	2222	SSH listen port.
web_enabled	bool	false	Run the configuration web server on web_port. Off by default; mirrors the GUI settings page in a browser. See §5.6.
web_port	u16	8080	Web-server listen port.
ssh_gateway_auth	string	password	Outbound SSH-gateway auth mode: password prompts for the remote password each connect; key uses the gateway's auto-generated Ed25519 client key (which must be installed in the remote's authorized_keys first).
gateway_role	string	standalone	Master/Slave role: standalone | master | slave. See §9.14.
master_accept_relays	bool	false	Master only: accept relay connections from slaves (never implied by enabling SSH). **Requires ssh_enabled** — the relay listens on the SSH server's port, so with SSH off no slave can connect. That combination is called out at startup and shown in every config surface (telnet: Accept relays: ENABLED (SSH off!); GUI and web: a warning beside the checkbox), but it is never "fixed" by switching SSH on for you.
slave_ma	string	*(empty)*	Slave only: the master's hostname/IP.

Key	Type	Default	Description
`ster_host`			
`slave_master_port`	u16	2222	Slave only: the master's SSH port.
`slave_master_username`	string	*(empty)*	Slave only: must match the master's unified username.
`slave_master_password`	string	*(empty)*	Slave only: must match the master's unified password.
`relay_transport`	string	`ssh`	Relay transport; only `ssh` is implemented.
`transfer_dir`	string	`transfer`	Base directory for file transfers, relative to the working dir.
`place_bundled_terminals`	bool	`true`	Write `EGT8080.COM` and `EGT80.COM` when they are missing. They are this gateway's own CP/M terminal in period assembly, compiled into the binary, and each is placed in *two* places: CP/M drive A:, where the emulator runs it, and loose in `transfer_dir`, where the file-transfer menus can send it to real hardware without starting the emulator at all. Nothing else puts them there on a fresh install. A file already present is **never** overwritten whatever this says — each terminal saves its settings inside its own `.COM`, so refreshing a copy would discard your configuration — so this only decides whether a *missing* one is written. Turn it off if you keep your own build, or your own `EGT80.COM` from before 0.9.2, and would rather a file you deleted stayed deleted instead of reappearing on the next restart.
`gui_zoom`	string	`auto`	Desktop GUI display scale. `auto` follows the monitor's scale factor; a number (e.g. `1.0`, `1.25`) pins the console window's pixels-per-point so an over-reported DPI doesn't render it oversized. Clamped to 0.5–3.0.
	usize	`50`	Concurrent telnet session cap.

Key	Type	Default	Description
`max_sessions`			
`idle_timeout_secs`	u64	`900`	Session idle timeout. 0 disables.
`disable_gateway_connections`	bool	`false`	Refuse connections that come from this network's router. The address is *not* guessed: the gateway asks the operating system for the default route's next hop and blocks exactly that (see §7.6), so a router on `.254` is caught and an ordinary machine on `.1` is not. On a host where the query fails it falls back to the old `x.x.x.1` rule. **Off by default**, so such connections are allowed while the IP allowlist stays in force; loopback is never affected. The narrow alternative to `disable_ip_safety`, which drops the allowlist altogether. Every UI shows the detected address in the label — "Block connections from the router (192.168.1.1)".
`disable_ip_safety`	bool	`false`	When `true`, the gateway accepts inbound telnet/SSH from any address. Default `false` restricts to RFC-1918 private ranges and loopback. See §5.4.
`groq_api_key`	string	empty	Key for AI chat. Empty disables the feature.
`browser_homepage`	string	`http://telnetbible.com`	Page loaded when the browser opens.
`weather_location`	string	empty	Last-used city or postal code, worldwide (migrates legacy `weather_zip`). Updated automatically.
`weather_units`	string	`auto`	Weather units: `auto`, `us`, or `metric`.
`cpm_emu_enabled`	bool	`true`	The CP/M emulator `K` main-menu item, which runs arbitrary Z80 `.COM` software. On by default (jailed, bounded, and shipping its own terminal); set `false` to hide the item and reject the key. configured in the "AI, Browser & Weather — More" panel (telnet: Configuration → `C`). See §8.11.
`cpm_scre`	bool	`true`	May the web UI's *VDM / Dazzler* page send keystrokes to a booted disk? The screen is readable whenever the web server is on; this decides

Key	Type	Default	Description
`en_input`			whether it is also a keyboard. Both keyboards share one queue, so the terminal and the browser can type at the same guest. `ESC ESC` is never honoured from the browser. See §6.4.3.
`cpm_emu_max_minstr`	u32	`2000`	CP/M emulator runaway ceiling, in millions of instructions per program run (2000 = 2 billion). Bounds a compute-bound `.COM` that never reads the console; must be ≥ 1, and anything above **1000000** is *capped* at that rather than refused — asking for "no limit" gets you as close as this goes, which at emulated speed is over three months of continuous running. Bounds one transient in the *emulator* only; a booted disk is the session and has no ceiling. See §8.11.
`cpm_emu_uart`	string	`rc2014_1b`	How the emulated CP/M reaches the virtual modem. Defaults to `rc2014_1b` (0x82/0x83), the port the bundled EGT8080 terminal expects, so the two work together untouched; every UI offers a one-click "Default port" to come back to it. `off` = none; a machine/port profile such as `rc2014_1b` (RC2014 SIO/2 0x82/0x83) or `altair_2sio1` (Altair 88-2SIO 0x10/0x11); `aux` (BDOS AUX: device, for SC126/RomWBW); or `hbios_1`/`hbios_2` (RomWBW HBIOS serial unit, via `RST 8`). See §8.11.
`cpm_boot_image`	string	*(empty)*	What the CP/M menu item runs. Empty = the CP/M emulator (our BDOS, drives A:–P:, EGT8080, the virtual modem). A bare filename in `CPM/images` instead cold-boots that disk on the emulated controller its boot loader names, and the disk's own operating system takes the whole machine — no jail, no `A>` from us, no EGT8080. The image is opened **writable** unless `cpm_boot_writable` is turned off. A name that is not in the folder falls back to the emulator and is shown as `(missing)`; one that is there but carries no boot program is shown as `(not bootable)` and does the same. See §8.11.
`cpm_boot_machine`	string	`auto`	Which machine a *booted* disk runs on — its console **and** its disk controllers. `auto` reads the ports the disk's own boot loader drives, so ordinarily you set nothing; when a disk does not say plainly, the Altair 88-2SIO machine stands and the boot screen says which happened. Set it explicitly to override. Getting it wrong looks like a broken disk: the image loads, reads its sectors and then goes silent, because the guest is printing to hardware that is not there. See §8.11.
`cpm_boot_backspace`	string	`backspace`	What a *booted* disk is handed when you press Backspace. Your terminal's key sends DEL (`7Fh`), which most of these operating systems read as a Teletype *rubout*: they delete the character and then print the character they deleted, so `TESTING` backspaced over reads `TESTINGGNIT`. `backspace` sends BS (`08h`) instead, which they erase on; `rubout` passes the key through, which is what CP/M 1.x wants — there the rubout is the editing key and BS prints a literal `^H`. This is the whole answer now: the telnet boot picker that used to ask again per disk was removed in 0.9.2. Ignored by the CP/M emulator, which reads its own console line and accepts either. See §8.11.
`cpm_boot`	bool	`true`	May a *booted* disk write to the images it is running? On by default: a vintage operating system saves files, formats disks and updates its own

Key	Type	Default	Description
`_writable`			directory, and a read-only boot makes every `SAVE` appear to work and vanish at the next boot. Worth knowing what it costs: a mounted image is read through the gateway's own filesystem, which can refuse a bad request, but a booted disk owns the whole image and rewrites the file when it leaves — so nothing is left that understands what the guest is asking for. Turn it off to keep every disk exactly as it is. It covers the boot disk *and* every image mounted beside it, and it is a standing setting, so it applies to everyone who reaches CP/M. Ignored by the emulator. See §8.11.
`cpm_cpu`	string	`z80`	Which processor **both** CP/M machines run — the emulator's transient programs and a *booted* disk's whole operating system. The Z80 is a strict superset of the 8080, so it runs every disk here, and Altairs were very commonly fitted with a Z80 upgrade board; `8080` is the processor the Altair actually shipped with. **The terminals:** `EGT8080.COM` is built to the 8080's instruction set, so it runs on *either* setting — reach for that one. `EGT80.COM`, the Z80 build, is placed beside it because it carries the **Z180 ASCI** ports (an SC126's console), whose instructions cannot appear in an 8080 binary at all; under `8080` it stops at its first Z80-only opcode, which is what the sign-on line warns about. Choose the 8080 when you are running period 8080 software, notably diagnostics that identify the CPU from `DCR A` setting parity rather than overflow and are therefore *right* to fail on a Z80. See §8.11.
`cpm_printer`	string	`text`	Where CP/M printer output goes. Reaches **both** CP/M machines, like `cpm_cpu`, but by two entirely different routes: in the emulator the printer is an operating-system *service* (BDOS function 5 and the BIOS `LIST` vector), so WordStar, MBASIC's `LPRINT` and `PIP LST:=FILE.TXT` all arrive there; a *booted* disk drives a printer *board* instead, chosen with `cpm_printer_port`. Either way one document is written into a `printer` folder inside the transfer directory — its own folder so a printer left on does not scatter files through yours, and not onto a CP/M drive, because it is for you rather than for the guest. `odt` = OpenDocument text, monospaced, one page per form feed; `text` = plain text with the form feeds kept; `off` = printer output appears on your terminal, which is where it has always gone. **Text is the default since 0.9.2** — nothing is written until a guest actually prints, so an operator who never uses `LST:` never sees a file, whereas `off` meant the first printout anyone made was scattered across their terminal with no way to get it back. Named `PRINT-YYYYMMDD-HHMMSS` from this machine's clock. A job is finished 5 seconds after the last character printed — CP/M has no end-of-print signal, so silence is the only one there is — and in the emulator also the moment the program returns to `A>`, which is exact. Bold and underline survive into an `.odt`. Period software does not ask for them, it **overstrikes** — WordStar prints the line, sends a bare CR and reprints just the emphasised run at the same columns — and that is turned into real styling. Measured against WordStar 3.0, which is also why `cpm_printer_autolf` matters: with that

Key	Type	Default	Description
			switch on, an overstrike pass lands on a line of its own instead of on top of the text. See §8.11.
`cpm_printer_port`	string	`altair_c`	Which printer board a *booted* disk finds. Ignored by the emulator, whose printer is a BDOS service with no port at all, and ignored entirely when `cpm_printer` is `off`. `altair_c` = the Altair line printer, data register `03h` — measured, not reasoned: Altair Hard Disk BASIC answering `LINEPRINTER? C` initialises with `OUT 03h←11h` / `OUT 02h←00h` and then sends one 7-bit ASCII character per byte to `03h`. `off` = a booted disk has no printer. The status register is deliberately not emulated: an unclaimed port reads `0xFF` here and every period convention reads a high bit as ready. The board also carries an **auto-line-feed** setting, the DIP switch real Centronics interfaces had, and for the same reason: a bare CR that does not advance the paper is how *overstrike* makes bold and underline, while a bare CR that does advance is how much other software ends a line, and only the hardware can settle which. Measured for `altair_c`: two `LPRINT`s send `ALPHA<CR>BETA<CR>` and no line feed at all, so the switch is on — otherwise a whole report prints on one line. A CR LF pair sent to it has the line feed absorbed rather than double-spacing. See §8.11.
`cpm_mounts`	string	*(empty)*	Disk images mounted on CP/M drives, as `A=name.dsk,C=other.dsk`. Values are bare filenames inside `CPM/images`, never paths. A mounted drive uses the CP/M filesystem inside the image instead of its folder; the folder's files are untouched and return on unmount. An image whose name carries no format prefix is identified by inspection and mounts **read-only**. See §8.11.
`verbose`	bool	`false`	Detailed per-block / per-subpacket protocol diagnostics (XMODEM, YMODEM, ZMODEM, Kermit, Punter).
`log_to_file`	bool	`true`	Mirror the log to `log_file` as well as stderr and the console buffer. On by default. Configured in the General "More…" panel of the GUI and web UI (telnet: Other Settings → `L`). Takes effect on the next restart.
`log_file`	string	`ethernetgateway.log`	Active log file, relative to the working dir unless absolute. Owner-only (0600 on Unix); reopened in append mode so a restart extends it. Empty disables file logging.
`log_max_size_kb`	u64	`1024`	Rotate once a write would push the active file past this size. `0` disables size rotation (unbounded growth).
`log_max_files`	u32	`5`	Rotated generations to keep; older ones are **deleted**. Worst-case disk use is `log_max_size_kb × (log_max_files + 1)` — 6 MB with the defaults. `0` keeps no history.
`xmodem_negotiati`	u64	`45`	Seconds to wait for the far-end sender/receiver to start. Shared by XMODEM, XMODEM-1K, and YMODEM (same protocol code path).

Key	Type	Default	Description
`on_timeout`			
`xmodem_block_timeout`	u64	`20`	Per-block timeout.
`xmodem_max_retries`	usize	`10`	NAK retry cap per block.
`xmodem_negotiation_retry_interval`	u64	`7`	Seconds between `C`/`NAK` pokes during the initial handshake. Christensen's XMODEM and Forsberg's reference implementations use ~10 s; 7 s is a compromise that starts quickly but avoids stockpiling extras in a slow-starting sender's input buffer.
`zmodem_negotiation_timeout`	u64	`45`	Seconds to wait for the ZRQINIT / ZRINIT handshake to complete.
`zmodem_frame_timeout`	u64	`30`	Per-frame read timeout once a transfer is live (headers / subpackets).
`zmodem_max_retries`	u32	`10`	Retry cap for ZRQINIT, ZRPOS, and ZDATA frames.
`zmodem_negotiation_r`	u64	`5`	Seconds between ZRINIT / ZRQINIT re-sends during the handshake. Analogous to `xmodem_negotiation_retry_interval` but for ZMODEM.

Key	Type	Default	Description
`etry_interval`			
`kermit_negotiation_timeout`	u64	`300`	Send-Init handshake timeout in seconds.
`kermit_packet_timeout`	u64	`10`	Per-packet read timeout in seconds.
`kermit_idle_timeout`	u64	`300`	Server-mode idle timeout between commands. `0` disables.
`kermit_max_retries`	u32	`5`	NAK / timeout retransmits per packet.
`kermit_max_packet_length`	u16	`4096`	Advertised long-packet length (10–9024).
`kermit_window_size`	u8	`4`	Sliding-window size (1–31). 1 = stop-and-wait.
`kermit_block_check_type`	u8	`3`	1 = 6-bit checksum, 2 = 12-bit checksum, 3 = CRC-16/KERMIT.

Key	Type	Default	Description
`kermit_long_packets`	bool	`true`	Advertise long-packet capability.
`kermit_sliding_windows`	bool	`true`	Advertise sliding-window capability.
`kermit_streaming`	bool	`true`	Skip per-packet ACKs on reliable links (TCP/SSH).
`kermit_attribute_packets`	bool	`true`	Send/accept A-packets carrying size, mtime, mode, resume.
`kermit_repeat_compression`	bool	`true`	Encode runs of identical bytes with the repeat-prefix.
`kermit_8bit_quote`	string	`auto`	8th-bit quoting policy: `auto` (when peer asks), `on`, or `off`.
`kermit_resume_partial`	bool	`false`	Resume partial uploads using A-packet disposition='R'. Off by default — a peer that ignores disposition='R' would corrupt the result.
`kermit_resume_max_ag`	u32	168	Maximum age of a partial file we'll resume (default one week).

Key	Type	Default	Description
e_hours			
kermit_locking_shifts	bool	false	Use SO/SI region markers instead of per-byte 8th-bit prefix when the peer advertises CAPAS_LOCKING_SHIFT.
kermit_server_enabled	bool	false	Run the standalone Kermit-server TCP listener at startup.
kermit_server_port	u16	2424	Kermit-server listen port.
allow_atdt_kermit	bool	false	Allow the serial-modem dial target ATDT KERMIT (or kermit-server) to drop into the Kermit server. Off by default — bypasses the telnet auth gate.
allow_peer_dial	bool	false	Let a modem port dial another serial port directly (ATD <Port>@<IP>, or pick a modem port in the Serial Gateway) instead of always landing on the gateway menu. Off by default — it lets any modem caller ring any addressable port. Gates *dialing* only: a slave still announces its ports and CP/M endpoint to its own master without it (§9.14.3). See §9.2.3.
allow_relay_kermit	bool	false	**Master only.** Serve this master's Kermit server to a slave port that is in Kermit-server mode, so the device on the slave's wire lists, uploads to and downloads from *this* machine's transfer directory (§9.14.3). Off by default: Kermit's server mode has no authentication of its own, so this hands a remote wire unauthenticated read/write access to the transfer directory.
kermit_wait_for_receiver	bool	true	On an interactive Kermit *download*, wait for the receiver's first NAK before sending our Send-Init, so the packet doesn't land on the terminal as on-screen garbage before the client is ready. Falls through and sends anyway after the negotiation timeout. Server and test paths never wait.
punter_block	u16	255	Punter C1 block size in bytes (8–255). 248-byte payload + 7-byte header at the default; lower toward ~40 on noisy lines.

Key	Type	Default	Description
`_size`			
`punter_negotiation_timeout`	u64	45	Seconds to wait for the file-type handshake to complete.
`punter_block_timeout`	u64	20	Per-block read timeout once a transfer is live.
`punter_max_retries`	u32	10	Retry cap per block before the session aborts.
`punter_max_bad_rounds`	u32	30	Consecutive corrupt-block resend rounds tolerated before giving up. Kept higher than `punter_max_retries` — a real C64 peer never caps its resends, so a low value would make the gateway quit first and strand the peer.
`punter_hangup_on_failure`	bool	`false`	On give-up, drop the connection (carrier) so a C64 — which C1 cannot be told to abort — sees loss-of-carrier and exits instead of hanging. Ends the whole session; off by default.
`punter_negotiation_retry_interval`	u64	5	Seconds between handshake re-sends while waiting for the peer.

The serial subsystem exposes **two physically independent ports**: **Port A** and **Port B**. Each has its own copy of every per-port key listed below, prefixed with `serial_a_` or `serial_b_` respectively (e.g. `serial_a_enabled`, `serial_b_baud`). The two ports are completely

independent — separate manager threads, separate AT&W persistence targets, separate console-bridge slots — so you can run a Hayes modem on one wire and a telnet-serial bridge on the other simultaneously.

> **Legacy migration**
>
> A pre-dual-port `egateway.conf` using bare `serial_*` keys auto-migrates into Port A on first read; the writer always emits the dual-port form on save. Existing single-port deployments upgrade transparently, with Port B disabled by default.

Per-port key (suffix)	Type	Default	Description
`enabled`	bool	`false`	Activate this port. Pairs with `mode` to choose its role.
`mode`	string	`modem`	`modem` = run the Hayes AT command emulator on this port (chapter 9). `console` = expose the port via the telnet menu's **G Serial Gateway** A/B picker as a telnet-serial bridge (chapter 9.13).
`port`	string	empty	Device path (e.g. `/dev/ttyUSB0`, `COM3`).
`baud`	u32	`9600`	Baud rate. Must be ≥ 300.
`databits`	u8	`8`	Data bits: 5, 6, 7, or 8.
`parity`	string	`none`	`none` / `odd` / `even`.
`stopbits`	u8	`1`	1 or 2.
`flowcontrol`	string	`none`	`none` / `hardware` / `software`.
`echo`	bool	`true`	Saved `ATE` state. Restored by `ATZ`.
`verbose`	bool	`true`	Saved `ATV` state.
`quiet`	bool	`false`	Saved `ATQ` state.
`s_regs`	string	see ch. 9	Comma-separated decimal S0–S26 values.
`x_code`	u8 0–4	`4`	Saved `ATX` level.
`dtr_mode`	u8 0–3	`0`	Saved `AT&D` mode.
`flow_mode`	u8 0–4	`0`	Saved `AT&K` mode.
`dcd_mode`	u8 0–1	`1`	Saved `AT&C` mode.
`stored_0` … `stored_3`	string	empty	Stored phone-number slots for `AT&Zn=s` / `ATDSn`.
`petscii_translate`	bool	`false`	Saved `AT+PETSCII` state (PETSCII translation on direct-TCP dials). Also editable from the telnet, web, and GUI config surfaces.
`drive_carrier`	bool	`false`	Drive **DTR** as a hardware carrier proxy. When true the modem emulator asserts and drops DTR with the connection (tied to `AT&C`), so a terminal wired DTR→DCD sees carrier detect. When false the gateway never touches the modem-control lines, so a port with no DCD wiring behaves exactly as before. Modem mode only. Editable in all three config UIs.

Input sanitisation

String values have embedded newlines and carriage returns stripped before being written back to disk. That means you cannot accidentally corrupt the config by pasting a multi-line value into the GUI.

4. GUI Configuration Console

Unless you have explicitly set `enable_console = false`, a GUI configuration window opens automatically when the gateway starts. It is backed by eframe/egui and provides live configuration editing plus a console that streams the server log in real time.

> **Headless mode**
> Systemd deployments, Docker images, or any other headless setup should run with `enable_console = false`. Otherwise the daemon will block waiting for a display server that does not exist.

Closing the GUI window does not stop the server — it continues running in headless mode. To stop the gateway, send it `Ctrl+C`, `SIGTERM`, or `SIGHUP` (see chapter 15).

> **First launch**
> On a fresh install this window opens into the setup wizard instead of the editor described below, and stays there until you finish or skip it — see §3.1. Everything the wizard asks can also be changed here afterwards.

4.1 Configuration Frames

The window is organised into three rows of two framed cards each, plus buttons and a console panel at the bottom:

- **Server** — toggle telnet/SSH/Web/Kermit listeners and set their ports. A *More…* button opens a popup with the session cap, idle timeout, GUI display scale, the Telnet/SSH gateway options, and the **Master/Slave** relay settings.
- **Security** — Require Login toggle, plus separate username and password fields for telnet and SSH.
- **File Transfer (XMODEM)** — transfer directory, XMODEM-family timeouts (negotiation, per-block, retries). The card's subtitle reads *(More for YMODEM / ZMODEM / Punter)*: a *More…* button opens a popup with the full per-protocol breakdown covering the XMODEM/XMODEM-1K/YMODEM shared keys, the independent ZMODEM tunables (handshake timeout, frame timeout, retry cap, and retry interval), and the Punter C1 tunables (block size, negotiation timeout, retry interval, block timeout, max retries).
- **AI Chat, Browser, Weather & CP/M** — weather location and browser homepage URL. A *More…* button opens a popup with the Groq API key (rendered as a password field), the weather units and the CP/M settings. The key is in the popup rather than on the frame on purpose: it is optional — AI Chat is the only thing that wants one — and a key field in the first row reads as something you must supply before the gateway will work.
- **Serial Port** — three-row layout for the dual-port subsystem. The header row carries an *Enabled* checkbox each for *Serial Port A* and *Serial Port B*, plus a single *Save* button on the right that persists both ports atomically and signals each manager to reconfigure. The two rows below show one port apiece: a device-path dropdown (with a refresh ↺ button on Port A's row that re-scans and refreshes both dropdowns), a baud field, and a per-port *More…*

button that opens that port's advanced popup (mode, framing, flow, full Hayes AT state). The two popups are independent so you can compare settings side-by-side.
- **General** — Verbose XMODEM Logging, Show GUI on Startup.

At the top of the window the title shows the gateway version and the local IP address that telnet clients will use to reach it. The IP is re-detected on every paint cycle, so if your DHCP lease changes the value updates automatically.

4.2 "More..." Popups

Advanced settings that are rarely changed live in popup windows opened from their frame's *More…* button.

4.2.1 Server — More

Mirrors the primary Server controls (so you can edit ports and toggles without leaving the popup) and adds two persistent outbound-gateway decisions that used to be asked interactively at dial time:

- **Telnet Gateway → Mode** — dropdown with *Telnet* or *Raw TCP*. Telnet is the default and parses IAC in both directions; Raw TCP disables the entire IAC layer for destinations that don't speak telnet at all. Sets `telnet_gateway_raw` in `egateway.conf`.
- **Negotiate TTYPE / NAWS with remote** — checkbox, enabled only when Mode is *Telnet*. Turn on when you dial real telnet servers that want cooperative option negotiation. Sets `telnet_gateway_negotiate`.
- **SSH Gateway → Auth** — dropdown with *Key* or *Password*. Password is the default because it works out of the box with most remote accounts. When Key is selected the popup also displays the gateway's public key in a read-only text box below, ready to paste into the remote's `~/.ssh/authorized_keys`. Sets `ssh_gateway_auth`.

Both decisions take effect on the next gateway connection — no server restart is needed. The session-side Gateway Configuration menu (*Configuration → G* from the main telnet menu) edits the same three settings from the telnet client: `T` toggles the telnet mode (Telnet / Raw TCP), `C` toggles cooperative TTYPE / NAWS negotiation (`telnet_gateway_negotiate`), and `S` toggles the SSH auth mode (Key / Password).

The popup also carries the **Master/Slave** relay settings (role, the master's accept-relays gate, and a slave's master host/port/credentials) and, at the bottom, a **Run setup wizard...** button. That button reopens the first-run wizard (§3.1) with your current settings pre-filled — the password excepted, since it is never echoed back — and closes this popup so the wizard has the window to itself. It changes nothing unless you walk it through to *Save and Restart Server*.

> **Popup styling**
> Popups render on a deep forest-green panel with lighter-green text-entry fields so they're visually distinct from the navy main window. This is a UI convention — it doesn't affect any setting.

4.2.2 Serial Port A — More / Serial Port B — More

Each port has its own popup, opened by the *More…* button on that port's row in the Serial Port frame. The two popups are independent — both can be open simultaneously so you can compare Port A's and Port B's advanced settings side-by-side. Each popup contains:

- **Mode** — dropdown with *Modem (AT Command) Mode*, *Telnet-Serial Mode*, or *Kermit Server Mode* (an always-on Kermit server on the wire — see §8.8). Saving from this popup reconfigures only the changed port; the other port keeps running.
- **Bits / Par / Stop / Flow** — data bits (5–8), parity (none/odd/even), stop bits (1/2), and flow control (none/hardware/software) for this port.
- **Echo (E1) / Verbose (V1) / Quiet (Q1)** — saved `ATE`, `ATV`, `ATQ` toggles for this port.
- **Result level (X)** — 0 to 4. See ch. 9 for what each level emits.
- **DTR (&D)** — 0 to 3.
- **Flow (&K)** — 0 to 4 (modem-layer; hardware-wire flow is set by the *Flow* dropdown above).
- **DCD (&C)** — 0 or 1.
- **S-Registers** — a comma-separated list of S0…S26 values, editable as free text. The format matches `serial_a_s_regs` / `serial_b_s_regs` in the config file.
- **Stored Phone Numbers** — four text fields for slots `&Z0` through `&Z3`. Leave empty for unset. `ATDSn` dials these.

4.2.3 File Transfer — More

Mirrors the transfer directory row from the main card and then splits the protocol settings into two clearly-labelled sections so operators aren't surprised when editing one page changes another:

- **XMODEM / XMODEM-1K / YMODEM** — a banner explains these three protocols share the same code path, and therefore the same keys. Shown: *Negotiate (s)*, *Block (s)*, *Retries*, and *Retry interval (s)* (the last is the C/NAK poke cadence, with an inline hint noting the spec suggests ~10 s).
- **ZMODEM** — independent tunables. Shown: *Negotiate (s)*, *Frame (s)*, *Retries*, and *Retry interval (s)* (ZRINIT/ZRQINIT re-send gap, default 5).
- **Punter (C1)** — independent tunables for the Commodore single-file protocol. Shown: *Block size* (8–255, default 255), *Negotiate (s)*, *Retry interval (s)*, *Block (s)*, and *Retries*.

All values apply on the next file transfer — no server restart is required. The corresponding telnet-side pages live under *Configuration → F (File Transfer) → X / Y / Z*.

4.3 Saving and Restarting

Each popup has its own **Save** button, which writes the current configuration to `egateway.conf` *without restarting the server*. This is useful for changes that take effect on the next session — updating stored phone numbers, toggling verbose logging, or adjusting XMODEM timeouts — because no listener needs to be re-bound.

The main window also has a large **Save and Restart Server** button. Changes that affect how the listeners are bound — turning SSH on or off, changing a listen port, enabling or disabling the serial modem, changing credentials — need a restart to take effect. The button saves the config, sets an internal restart flag, and triggers an orderly shutdown. The main loop detects the flag and re-spawns the server; the GUI reopens after the restart completes.

> **Out-of-band changes**
> If a telnet or SSH session toggles a configuration value (for example the Telnet Gateway's *T* mode toggle, or the in-session Configuration menu), the GUI notices and refreshes its fields — unless you are currently editing them, in which case your edits are not clobbered.

4.4 Live Console Output

The bottom pane of the window shows the server log: new sessions, auth failures, dialup lookups, XMODEM progress when verbose is on, SSH host-key trust decisions, and shutdown messages. It auto-scrolls to the tail and caps at 2000 lines.

5. Telnet Access

The telnet server is on by default and listens on TCP port **2323**. Change the port via the GUI, the in-session *Configuration* menu, or by editing `telnet_port` in `egateway.conf` and restarting.

5.1 Connecting

```
telnet 192.168.1.160 2323
```

Any standard telnet client works: `telnet`, PuTTY, SyncTERM, CGTERM on the Commodore 64, NetSwitcher on CP/M, a hardware WiFi modem plus a real C64, etc. The first thing the server does is negotiate a few telnet options via IAC (`WILL ECHO`, `WILL SGA`, `DO SGA`, `DO TTYPE`, `DO NAWS`), then prompt for terminal detection.

5.2 Terminal Detection

After the initial option handshake you will see:

```
Press BACKSPACE to detect terminal:
```

The byte you send determines the terminal class, as described in chapter 1. If the client already replied with an `IAC SB TTYPE IS …` subnegotiation during the initial handshake, the terminal is determined from that and the prompt is skipped entirely.

You are then asked to enable colour:

```
Use ANSI color? (Y/N):
```

No default: press `Y` or `N`. Answering `N` only turns off colour — it does *not* change the detected terminal type, so a PETSCII (Commodore) terminal keeps its 40-column, case-swapped layout and simply receives plain text.

5.3 Session Limits

- **Concurrent sessions** — capped by `max_sessions` (default 50).
- **Idle timeout** — `idle_timeout_secs` seconds (default 900 = 15 minutes). Set to `0` to disable. Idle timeouts apply to every prompt the server offers, including login, so users on slow modems should raise this.
- **Screen layout** — 40 columns on PETSCII, 80 on ANSI/ASCII, 22 rows of content (the remaining 2 rows are the header and prompt). Enforced by unit tests to avoid layout regressions.

5.4 Network Gating (Unauthenticated Mode)

When `security_enabled` is `false` (the default), the telnet server refuses connections from public IP addresses. Only the RFC 1918 private ranges — `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, plus loopback, link-local, IPv6 loopback (`::1`), IPv6 link-local (`fe80::/10`), and IPv6 unique local (`fd00::/8`) — may connect without authentication. Gateway addresses ending in `.1` are also rejected so a misconfigured NAT cannot accidentally bridge a telnet session through your router.

To accept telnet connections from any IP address, enable security and configure a strong username and password.

For lab and isolated-network setups where authentication is genuinely unwanted but the gateway still needs to listen on a non-private interface, the `disable_ip_safety` flag drops the allowlist entirely. The GUI Security frame and the in-session telnet menu both gate the toggle behind a confirmation popup that explains the risk before the change sticks; once enabled, *any* source address may connect with no credentials. Treat it as the explicit "I understand this is open" switch, not a default.

The configuration web server behaves differently on purpose. Because its page renders the login password and Groq API key into form fields, enabling "Require Login" does *not* open it to any IP the way it does for telnet: the web server keeps the private-IP allowlist whether or not login is required, and `disable_ip_safety` is the only switch that lifts it. So to reach the web UI from a non-private address you must set `disable_ip_safety` — enabling security alone is not enough.

5.5 The Main Menu

```
A  AI Chat
B  Simple Browser
C  Configuration
F  File Transfer
G  Serial Gateway
K  CP/M System
R  Troubleshooting
S  SSH Gateway
T  Telnet Gateway
W  Weather
X  Exit
```

K CP/M System appears only when `cpm_emu_enabled` is set; it boots the CP/M emulator (chapter 8.11). While the toggle is off the item is hidden and the `K` key is rejected.

G Serial Gateway appears whenever at least one of the two ports (Port A or Port B) is configured in *console* mode (see chapter 9.13). It opens an A/B picker showing both ports' status, then bridges the telnet/SSH session straight to the chosen port's wire so you can drive a microcontroller, RS-232 device, or other serial console remotely. From a session that came in over a serial port itself the option is hidden. Press `ESC` twice to leave the bridge. Both ports can host their own concurrent bridge — they're fully independent.

5.6 The Configuration Menu

Pressing `C` on the main menu opens a terminal-side configuration menu that mirrors most of what the GUI exposes. It is intended for headless systems and for the SSH-only path where the GUI is unreachable. Changes are written to `egateway.conf` on confirm and apply immediately to running services where possible.

```
Server addresses:
  192.168.1.10
ATD 192.168.1.10:2323

E  Security
G  Gateway Configuration
M  Serial Configuration
S  Server Configuration
F  File Transfer
C  CP/M Settings
O  Other Settings
R  Reset Defaults

Q  Back   H  Help
```

`E` Security
Toggle `security_enabled` and edit the unified `username` / `password` pair shared by telnet, SSH, and the configuration web UI.

`G` Gateway Configuration
Outbound gateway settings — telnet negotiation flags, SSH-gateway auth mode, browser homepage, weather location, AI key.

`M` Serial Configuration
Opens an A/B picker submenu listing both ports with their current status. Pick a port to enter that port's settings menu, where you can cycle its mode (per-port `T`: *Mode: Modem/Console/Kermit*), set its device, baud, framing, flow control, AT-state, dialup mapping, and ring emulator. Each port's settings are fully independent and persist under separate `serial_a_*` / `serial_b_*` keys (`serial_a_mode` / `serial_b_mode` = `modem`, `console`, or `kermit`). The per-port `T` item is hidden from sessions that arrived over a serial port (flipping that port would tear down the caller's own connection before it could confirm).

`S` Server Configuration
Listen ports and enable flags for telnet, SSH, the standalone Kermit server, and the configuration web UI; max sessions, idle timeout, the network-safety opt-out (`disable_ip_safety`), and `F` *Test ports* (§5.6.1).

`F` File Transfer
Per-protocol timeouts and retries for XMODEM, ZMODEM, Kermit, and Punter, plus the Kermit server toggle / port and `allow_atdt_kermit`.

`C` CP/M Settings

The CP/M emulator and everything under it: enable/disable, the runaway instruction ceiling, the virtual-modem port (`cpm_emu_uart`), mounting and unmounting disk images, booting a disk image so it runs its own operating system, the boot machine and CPU, and the `LST:` printer. Reached from *Other Settings* in releases before 0.9.2.

`0` **Other Settings**

The Groq API key, browser homepage, weather location and units, verbose transfer logging, the Show-GUI-on-Startup flag, the gateway debug trace, log-file settings and Restart Server.

`R` **Reset Defaults**

Restore every recognised key to its built-in default. Confirms with *Y/N* first; credentials are reset to `admin` / `changeme`, so you'll want to set new ones immediately afterward if security is enabled.

5.6.1 Testing the ports

A listener can bind its port perfectly and still be unreachable, because something on the machine is dropping the connections. **Test ports** — `F` on the telnet Server Configuration screen, and a button inside *Server — More* in the web and desktop interfaces — connects to each bound listener at this machine's own network address and reports what answered. It also runs once by itself at start-up, so you do not have to know to ask.

A port that does not answer is marked: its **Port** label turns red in the web and desktop interfaces (and the frame's *More…* button turns red with it, to point at the detail), and the telnet screen marks the row with a `*`. Both graphical interfaces also raise a summary window naming every listener and how it answered. Nothing polls: a red label is a snapshot from the last test, and stays red until the next one.

What the test actually proves

This matters more than the result, and it is different on each platform. The probe connects to *this machine's own address*, and whether such a connection meets the firewall at all is an operating-system decision:

	Linux	Windows	macOS
Finds a firewall on this machine	**yes**	no	no
Can raise a false alarm	no	no	no
Sees a router not forwarding	no	no	no

On **Linux** a connection to your own non-loopback address still passes through the `INPUT` chain, so an `iptables` or `nftables` rule that drops the port really is caught. On **Windows** the Filtering Platform exempts traffic a machine sends to its own address, so a port that Windows Defender is blocking answers anyway; on **macOS** the application firewall is per-*application* rather than per-port and does not filter this traffic either. On those two the test can only ever come back clear, which is why nothing here ever reports a port as "open" — the windows say *answered on this machine*, and say plainly that answering is not the same as reachable.

The middle row is the one that makes the test worth having: it never cries wolf. A red label always means something real, on every platform. And no row is a *yes* for the router: nothing running on this machine can see a port that your router is not forwarding, so the standing advice to **open these ports on your firewall** holds whether or not the test found anything.

Each probe is a real inbound connection to the gateway's own listener, so it occupies a session slot briefly and appears in the log as a connection from yourself. That is why the test runs on demand and at start-up rather than on a timer.

GUI vs terminal config

The GUI in chapter 4 is the friendlier interface for desktop systems. The terminal-side menu exists so that an SSH-only deployment, a serial-console session into the gateway, or any other no-display setup still has a way to edit settings without dropping out to a shell to hand-edit `egateway.conf`.

6. SSH Access

The SSH interface is the recommended way to connect to the gateway from outside your trusted LAN. It is *not* enabled by default — it runs on port **2222**, authenticates against the unified `username` / `password` pair shared with telnet and the web UI, and exposes the same menu system as telnet.

6.1 Enabling the SSH Server

Toggle the *SSH* checkbox in the GUI Server frame, or edit `egateway.conf`:

```
ssh_enabled = true
ssh_port    = 2222
username    = admin
password    = pick-something-strong
```

The same `username` / `password` pair is used by telnet, SSH, and the web configuration UI — changing them here changes authentication for all three. Versions prior to v0.6.0 had separate `ssh_username` / `ssh_password` keys; those keys are no longer recognized, but an upgrading config gets a one-time migration: non-default legacy SSH credentials are adopted into the unified pair on the first load when the unified pair is still at the factory defaults (a `Note: migrating legacy ssh_username=…` line lands in the log). After the next save the legacy keys disappear from the file.

Then click *Save and Restart Server*, or restart manually. On first start with SSH enabled, the gateway generates an Ed25519 host key and persists it to `ethernet_ssh_host_key` next to the binary (mode `0600` on Unix). Subsequent starts reuse the same key so clients can verify the server's identity.

6.2 Connecting

```
ssh admin@192.168.1.160 -p 2222
```

The first time your client connects it will display the server's SHA-256 fingerprint and ask whether to trust it. Accept, and the key is recorded in your client's `~/.ssh/known_hosts` as usual.

After authentication you land on the same Ethernet Gateway menu as a telnet user, rendered in ANSI terminal mode. All features — file transfer, SSH gateway, telnet gateway, browser, AI chat, modem emulator, weather — are available.

6.3 Public-Key Authentication

The built-in SSH server supports password authentication only. However, the *outbound* SSH gateway (chapter 10) offers Ed25519 public-key auth to remote hosts — a separate concept.

> **Plaintext credentials**
> The unified `username` / `password` pair in `egateway.conf` is stored in plaintext (used by telnet, SSH, and the web UI). The SSH connection itself is encrypted, but the file is not. Protect it with the `0600` permissions the gateway sets automatically, and do not reuse passwords from other services.

6.4 Web Configuration Interface

The *configuration web server* renders the same settings page the desktop GUI does, in a browser. It is off by default; toggle it on in the GUI's Server frame (the *Web Server* checkbox between Telnet and Kermit Server) or via the telnet menu's **Configuration > Server Configuration > W** key. The default port is **8080**.

```
web_enabled = true
web_port    = 8080
```

Once enabled and the server is restarted, browse to `http://<server-ip>:8080/`. The page mirrors every GUI frame — Server, Security, File Transfer, AI/Browser/Weather, Serial Ports, General — and each frame has its own Save button matching the GUI's semantics:

- **Server: Save and Restart** — persists every field and cycles the gateway through a full restart so new listener ports and enable-flags take effect (same flow as the GUI's *Save and Restart* button).
- **Serial Ports: Save** — persists every field and reopens the serial port managers only; telnet, SSH, and the web UI itself stay running.
- **Security / File Transfer / AI/Browser / General: Save** — persists only. The relevant code paths read these settings live, so no restart is needed.

The page also includes a live `Console Output` tail (polled every two seconds), a serial-port dropdown populated from the same device enumeration the GUI uses (with a refresh button that re-scans without a page reload), and inline confirmation dialogs that warn the operator before disabling the web server or changing its port — either action breaks the currently-open browser session, so the page nudges the operator to reconnect at the new address after saving.

6.4.1 Authentication and Network Gating

The web server respects the same `security_enabled` / `disable_ip_safety` flags telnet does:

- When `security_enabled` is **off** (the default), access is gated on source IP — only private / loopback / link-local addresses are accepted, and gateway-style `*.*.*.1` addresses are rejected. No password is required.
- When `security_enabled` is **on**, the web server prompts via HTTP Basic Auth against the unified `username` / `password` pair (the same one telnet and SSH use). The IP-safety allowlist is bypassed because the password is now the gate.
- Three failed Basic-Auth attempts from the same source IP trip the shared 5-minute lockout used by telnet and SSH. Subsequent requests get `HTTP 429 Too Many Requests` with a

`Retry-After: 300` header until the window expires. The lock fires *before* request parsing, so a flood of malformed POSTs from a banned IP can't keep the server busy either.

6.4.2 Defense-in-Depth Caps

- **30-second read timeout** per request — a slow-loris client can't park a worker indefinitely.
- **16 concurrent connections max**; excess receive `HTTP 503 Service Unavailable` with a short `Retry-After`.
- 16 KB cap on request headers, 64 KB cap on POST bodies — the save form is far smaller; this just bounds the worst case from a hostile peer.

6.4.3 Screens — the VDM-1 and the Dazzler

The web server has a second page, `/vdm`, reached with the **VDM / Dazzler** button in the configuration page's *AI Chat, Browser, Weather & CP/M* frame — or from the desktop UI, where a button of the same name in the CP/M *More* popup opens it in your browser. The desktop needs the web server running, since that is what serves the page; if it is off, the button offers to turn it on, which restarts the gateway and so asks you to confirm first. After that restart the page opens by itself. It shows the screen of a *booted* CP/M disk (§8.11) whose display is a **Processor Technology VDM-1** — a 1976 video card with no serial line and no data port at all. Software written for one puts a character on the screen by storing a byte into memory at `CC00`, so such a disk boots here, takes your keystrokes perfectly and leaves the telnet or SSH session it was started from completely blank. There is no character stream to show, because the guest never produces one.

Which disks this applies to

The button is offered for **every** booted session, not only the disks below — sampling the guest's memory costs it nothing, so withholding a screen on a guess would only hide one that works. A disk driving neither card just shows a blank grid. These are the ones known to paint something, and all but one are in the sample-disk download:

VDM-1 (64×16 characters) — `TDISK04` (CP/M 1.4 for the VDM-1) and `TDISK05` (CP/M 2.2), which boot and print nothing to a port at all; `DISK11`, VDM-1 programs that also boot CP/M; and `TDISK06`, a programs *library* with no boot program of its own — *mount* that one beside `TDISK05` rather than looking for it on the boot list. z80pack's `altairsim` library adds `cpm14-vdm`, `cpm22-vdm` and `vdm-stuff`.

Dazzler (colour) — `DISK10`, the library disk of about two dozen programs, which also boots CP/M and is the usual place to start; `DISK15`, the Felix animation system; and in z80pack, `dazzler` and `dazzlerII` (`imsaisim`) and `dazzler_graphics` and `dazzler_stuff` (`cromemcosim`).

The boot banner warns you when a disk's system tracks drive a VDM-1, but it cannot for `DISK11` — that disk's driver is in a CUTER ROM rather than on the disk, so there is nothing to find — and it cannot for a Dazzler at all: the Dazzler's two registers are ordinary low port numbers that occur by chance in most disks' data, so scanning for them would cry wolf on two images in three. A Dazzler announces itself the moment the guest switches one on, and the session list marks it then.

Shortest route to a picture: fetch the sample disks, set `DISK10` to boot, and press **VDM / Dazzler**.

Boot the disk as usual and keep typing in your terminal session — the keyboard is an ordinary port, and only the display was ever memory-mapped. Open `http://<server-ip>:8080/vdm` in a browser, pick the session from the list, and the 64×16 screen is drawn there, cursor and all. Two people can therefore share one session: whoever is typing and whoever is watching need not be the same person, which is the one thing a memory-mapped display makes easier rather than harder.

Every booted session offers a screen, not only the disks known to use the card, because sampling one costs the guest nothing: it is a read of the guest's own memory with no write, no interruption and no change in timing. The list marks the sessions that have really driven the card — a guest that has written the scroll register on port `C8h` is running a VDM-1 driver — and a screen showing only whatever happens to live at `CC00` says so beneath the picture rather than leaving you to puzzle at the noise. Nothing is copied at all while no browser is watching: the page asks for a frame about seven times a second and each request produces exactly one, so closing the tab ends the cost rather than reducing it.

When you boot a disk whose own system tracks drive the card, the boot banner in your terminal session says so and prints the address to open — including the case where the web server is switched off, which is worth knowing before you spend an evening wondering why a working disk looks dead. There is no such warning for a Dazzler and there cannot be: its two

registers are ordinary low port numbers that turn up by chance all over a disk's data, so a scan before boot would cry wolf on two images in three. A Dazzler announces itself when the guest switches one on, and the session list marks it then.

Which disks these are. In the widely-circulated Altair-Duino collection, `TDISK04.DSK` is a CP/M 1.4 built for the VDM-1 (it signs on `TARBELL 48K CPM V1.4 OF 2-15-78 / VDM VERSION.`), and `DISK10.DSK` is Cromemco's whole Dazzler library — Kaleidoscope, Dazzle-mation, Life, XLIFE, the graphics demo, the plotting programs and about two dozen in all. In z80pack's `altairsim` library, `cpm14-vdm.dsk` is the same VDM-1 CP/M. Twelve more images across the collections carry Dazzler programs. `DISK11.DSK` looks like a third VDM-1 disk and stays dark: its driver lives in a CUTER monitor ROM rather than on the disk, and that ROM is not one the gateway has.

Getting them automatically. Every disk screen — the telnet mount wizard, the web *Mount CP/M Drives* page and the desktop *Mount CP/M Drives* window — offers to **download the sample disks** before you mount anything, and so does the CP/M settings screen on all three, because you can pick a *boot* disk without ever opening the mount screen. The desktop setup wizard offers it too, on its CP/M screen (§3.1), so a fresh install can arrive with the disks already in place. It fetches about 42 MB into `CPM/images`, where the gateway finds them without any further setup, and it offers **only the disks this gateway is known to run** — thirty-four of them, from the two collections named below. Four are left out on purpose: `DISK0B`, `DISK0D`, `DISK0F` and `TDISK06` are companion disks of *programs* for a system disk that does boot, and carry no boot program themselves, so downloading them only to find they do nothing is not a favour. You can still fetch those by hand if you want them — they are worth *mounting* beside the disk they belong to, which is what they are for. That list is not a remembered one: each disk is cold-started from the bytes its pinned address really serves before it earns a place in the offer. Anything already in the folder is **never overwritten**, so a disk you have edited or put there yourself is safe. The download is pinned to a particular commit and every file is checked against a recorded SHA-256, so what arrives is what was tested.

Where to get them by hand. The disks are other people's work and are not shipped with the gateway. The Altair-Duino set is the `disks/` folder of David Hansel's Altair 8800 simulator, github.com/dhansel/Altair8800. Four hard disks come from Jim McNeely's github.com/jpmcneely/AltairDuino-Disks — the Infocom adventures, BASIC, COBOL and dBase II, which the first collection does not have. Take care if you browse that one by hand. It also holds four files called `DISK13` to `DISK16` which are *different disks* from Hansel's of those names, are undocumented in its own catalogue, and one of which does not boot; the gateway takes those four from Hansel. Its `DISK17` looks like a fifth disk Hansel has not got, and is not: byte for byte it is Hansel's `DISK12`, the same IMP modem executive under another number, so it is not offered either. The rest are the `<sim>/disks/library` folders of Udo Munk's z80pack, github.com/udo-munk/z80pack. Put the `.dsk` files in `CPM/images` under your transfer directory. **`repodisks.txt`**, which the gateway writes into that folder beside its readme, lists every disk in those collections and every file on it — read through the same mount path the gateway itself uses, so it is each disk's own directory rather than somebody's notes — with the address each collection came from at the head of its section.

The same page also shows a **Cromemco Dazzler**, the first colour graphics card for microcomputers (1976). It works the same way — the picture is the guest's own memory, read by DMA, with no data port — but it can be anywhere in memory rather than at a fixed address, and it is a picture rather than characters: 32×32 or 64×64 in sixteen colours, or 64×64 and 128×128 in black-and-white, depending on the mode the program picks. A session can have both cards at once, and one Altair disk in the common collection (`DISK10`) is Cromemco's whole software library: Kaleidoscope, Dazzle-mation, Life, and a couple of dozen more.

> **Joystick games**
>
> Cromemco's `SPACEWAR`, `GOTCHA`, `DOGFIGHT`, `TANKWAR`, `CHASE`, `AMBUSH` and `TRACK` read the D+7A analog board — a pair of joysticks with four switches and two analog axes each. They run, and they draw, but nothing a terminal or a browser produces is an X/Y voltage, so they cannot be played. That board is deliberately not emulated.

The screen is also a **keyboard**. Click it and type: the bytes reach the guest through the same translation the terminal's own bytes use, so the backspace key you chose behaves identically from either. Both keyboards work at once — there is one key queue, exactly as two keyboards on one port would share one — so the person at the terminal and the person in the browser can both type at the same guest, and their characters interleave if they do it at the same moment. `Ctrl+C` and `Ctrl+S` reach the guest; `ESC ESC` deliberately does not, because ending a session somebody else is sitting at is not a keystroke. Set `cpm_screen_input = false` to make the page read-only; the screen stays readable either way.

> **Who can watch**
>
> This page belongs to the configuration server, so it authenticates the *administrator* — while the person typing at the guest is on telnet or SSH. Anyone who can open the configuration page can watch any booted session's screen — and, with `cpm_screen_input` on as it is by default, type at it. On a gateway whose web UI already shows the password and the API key that is not a new privilege, but it is a different sentence from “the operator can see their own screen”.

> **Encryption**
>
> The configuration web server speaks plain HTTP, not HTTPS. Don't expose it directly to the public Internet — use it on a trusted LAN, or tunnel via SSH (`ssh -L 8080:127.0.0.1:8080 …`) when configuring the gateway from elsewhere.

7. Security

Security is optional. In the default configuration the gateway trusts anyone on your private LAN and refuses public-Internet connections. Turn security on when you want explicit authentication, or when you need to accept connections from public addresses over SSH.

7.1 Enabling Login

Check the *Require Login* box in the GUI Security frame, or set:

```
security_enabled = true
username = admin
password = your-password
```

A single `username` / `password` pair gates all three authenticated interfaces — telnet, SSH, and the configuration web UI. Three failed logins from the same source IP — across any of the three — trip a 5-minute lockout that blocks every protocol on that IP.

7.2 Credential Comparison

Passwords are compared byte-for-byte using a constant-time equality check (`constant_time_eq`) so an attacker cannot measure tiny timing differences to recover the password one character at a time. This matters less over telnet (because the password is transmitted in cleartext anyway) but it is a genuine defence for SSH logins.

7.3 Per-IP Lockout

The telnet, SSH, and web servers share a single in-memory lockout table keyed on the remote IP address. After **3** failed authentication attempts within 5 minutes the IP is blocked across *all three* protocols for 5 minutes. An attacker cannot bounce between protocols to reset the counter. The web server responds to a locked-out client with `429 Too Many Requests` and a `Retry-After: 300` header; telnet and SSH simply refuse the session.

Once the block expires the counter resets on the next successful login. Lockout events are logged to the console panel so a legitimate user who locked themselves out can see what happened.

7.4 File Permissions

On Unix, every file the gateway writes that may contain secrets is set to mode `0600` (owner read/write, no access for group or other):

- `egateway.conf` — the unified login password (used by telnet, SSH, and the web UI), Groq API key.
- `dialup.conf` — no secrets, but dial-up mappings.
- `gateway_hosts` — known host fingerprints for outbound SSH dials.

- `ethernet_ssh_host_key` — the SSH server's private host key.
- `ethernet_gateway_ssh_key` — the gateway's outbound SSH client key.

Config file writes are done to a per-PID temporary filename and then renamed into place. This closes a TOCTOU window against symlink-swap attacks in shared working directories.

7.5 Audit Logging

Important events are written to the live console panel (and to stderr in headless mode):

- Auth successes and failures, with the remote IP.
- Lockout events when an IP hits the 3-failure threshold.
- SSH Gateway host-key trust decisions — TOFU accept, key update, and rejection — with host, port, key algorithm, and SHA-256 fingerprint. This gives you a forensic trail if a changed host key turns out to be a man-in-the-middle attempt.

7.6 Blocking the Router — How the Address Is Found

The optional `disable_gateway_connections` rule refuses connections that appear to come from this network's router. Traffic forwarded in from outside the LAN often arrives wearing that source address, so refusing it closes a path into an unauthenticated gateway — at the cost of also refusing an administrator working from the router's address, and hairpinned traffic from inside your own network. That trade is why it is off by default.

The address is **queried from the operating system**, not guessed. Earlier versions assumed the router was `x.x.x.1`, which is a convention rather than a fact: it missed a router living on `.254` and refused an ordinary machine that happened to sit on `.1`. The gateway now reads the default route's next hop — precisely the address the router's traffic arrives from — and blocks exactly that, IPv4 and IPv6 alike. A multi-homed host with two default routes has both blocked.

Platform	Source
Linux	`/proc/net/route` and `/proc/net/ipv6_route`, read directly — no subprocess.
macOS / BSD	`route -n get default` (and `-inet6`); the `gateway:` line.
Windows	`route print`; the `0.0.0.0 0.0.0.0` and `::/0` rows. Matched on the addresses, not the column headings, so a localised Windows parses correctly.

The query runs on a background thread at startup (and again if the cached answer ages past five minutes, so a DHCP change is picked up without a restart). It is *never* run while a connection is being checked — the accept path only reads the cached answer, so no incoming connection can ever wait on a subprocess. The detected address is logged at startup (`Router address: 192.168.1.1`) and shown in the checkbox label in all three configuration UIs.

If the query fails

On a host where the router cannot be determined — an unsupported platform, a container with no default route, a locked-down system where the command cannot run — the rule falls back to the historical "refuse any address ending in `.1`" behaviour, and the UIs show `x.x.x.1` in place of an address. The setting therefore never becomes silently weaker than it was before the query existed. Loopback (`127.0.0.1`) is exempt either way, and no detected address can widen the allowlist: a public source address is still refused even if it somehow *is* the default gateway.

8. File Transfer

File transfer is the original reason this project exists. The gateway implements six protocols, covering everything from a 1977 vintage client to a modern terminal that drag-and-drops files onto the window:

XMODEM (classic, 128-byte blocks)
: The original 1977 protocol. Uses a `SOH` header, a block number and its complement, 128 bytes of payload, and either an 8-bit checksum or a CRC-16. Compatible with every retro terminal program ever made.

XMODEM-1K (1024-byte blocks)
: A 1980s extension that uses an `STX` header and 1024 bytes of payload. Reduces protocol overhead by a factor of eight. Senders may mix 1K and 128-byte blocks freely within a single transfer; the gateway branches on the header byte per block.

YMODEM (batch + metadata)
: XMODEM-1K plus a block-0 header. Block 0 has `SOH`, block number `0`, complement `0xFF`, and a 128-byte payload containing the filename, size, and (optionally) mtime. The receiver uses the size to truncate trailing `SUB` padding exactly.

ZMODEM (Forsberg 1988)
: Streaming protocol with ZDLE escaping, hex / binary16 / binary32 headers, CRC-16 and CRC-32, batch send and receive, ZSKIP, and an autostart trigger so the user can drop a file onto the terminal from any menu without first navigating to the file-transfer screen. See §8.5.

Kermit (Columbia, multi-flavor)
: The classic over-7-bit-line protocol with sliding windows, long packets, attribute packets, and resume. The gateway acts as both client (push/pull from the menu) and server (idle in `kermit-server` mode for incoming GET / SEND / FINISH commands), and accepts a standalone TCP listener as well. See §8.8.

Punter C1 (“New Punter”, Commodore)
: The single-file protocol that CCGMS, Novaterm, and StrikeTerm speak natively on Commodore 64/128 BBSes. A two-phase handshake (a file-type block, then the data blocks) with two per-block checksums (a 16-bit additive checksum and a 16-bit cyclic checksum) and 3-byte ASCII handshake codes (GOO / BAD / ACK / S/B / SYN). This is single-file C1 — not the older PET “PTP” or multi-file “MPP.” See §8.9.

8.1 The Transfer Directory

All transfers are scoped to the `transfer_dir` setting (default `transfer`), relative to the gateway's working directory. The gateway canonicalises the path on every access to prevent symlinks from escaping the jail, and filenames are validated to reject path traversal.

Filename rules:

- Letters, digits, dots, hyphens, and underscores only.

- Maximum 64 characters.
- Cannot start with a dot.
- Cannot contain `..`.
- Must include at least one letter or digit.

File size is capped at **8 MB**. Larger uploads are aborted server-side with three `CAN` bytes.

8.2 Navigation

From the main menu press `F`:

```
  U  Upload           (receive file from your terminal)
  D  Download         (send file to your terminal)
  X  Delete           (remove a file from the transfer dir)
  C  Change directory (subdirectory within transfer_dir)
  M  Make directory    (create a subdirectory)
  K  Kermit server mode (idle and wait for client commands)
  S  Gateway Shell     (A> file-manager prompt — see §8.10)
  I  Toggle IAC escaping (for binary files with 0xFF bytes)
```

8.3 Uploads — Protocol Selection & Auto-Detection

Press `U` and enter a filename. Before the transfer starts, the gateway asks which protocol to use:

```
──────────────────────────────────────
  SELECT UPLOAD PROTOCOL
──────────────────────────────────────

  X  XMODEM/YMODEM  128/1K, auto
  Z  ZMODEM         1K, autostart
  K  KERMIT         any flavor, auto
  P  PUNTER         C1 CCGMS/Novaterm

  Pick one, or ESC to cancel:
```

The choice is immediate — one key, no Enter required. ESC (or `←` on PETSCII, byte 0x5F) cancels. The gateway then prints the matching *Start … send* prompt and waits for your terminal to begin. For the XMODEM/YMODEM family it prints:

```
Start XMODEM/YMODEM send from your terminal now.
```

The four families differ in how the incoming transfer is handled:

- **XMODEM/YMODEM** (key `X` or `Y`) — accepts classic XMODEM (128-byte `SOH`), XMODEM-1K (1024-byte `STX`), or YMODEM (block-0 filename/size header) and *auto-detects which one the sender is using;* the logic is detailed below.
- **ZMODEM** (key `Z`) — receives a ZMODEM stream, including a multi-file batch (colliding names are declined via ZSKIP).

- **Kermit** (key `K`) — receives any Kermit flavor; the peer's capabilities are read from its Send-Init. (Kermit's own **server mode**, main menu → `F` → `K`, remains available as an alternative — see §8.8.)
- **Punter** (key `P`) — receives a single-file Punter C1 transfer (§8.9). The file is saved under the name you entered, with a `.prg`/`.seq`/`.usr` extension appended to match the sender-declared CBM type when your filename has none.

Within the **XMODEM/YMODEM** family the gateway auto-detects the exact variant. The logic is:

1. The receiver first sends `'C'` (0x43) for CRC-16 mode. It retries every three seconds for two thirds of the negotiation timeout. Most modern clients respond to this.
2. If no response comes, the receiver falls back to sending `NAK` (0x15) to request checksum mode. This rescues very old clients that never implemented CRC.
3. The first header byte from the sender determines the block size: `SOH` (0x01) means a 128-byte block, `STX` (0x02) means a 1024-byte XMODEM-1K block.
4. If the first header is `SOH` with block number `0` and complement `0xFF`, it is a **YMODEM block 0**. The gateway reads the filename header, parses the file size, validates the CRC, `ACK`s, and sends a second `'C'` to start the data phase.
5. Within a single transfer the sender may mix 128-byte and 1024-byte blocks freely. Each block's header byte selects the payload length.
6. Transfer ends on `EOT` (0x04), which the server `ACK`s. For YMODEM, the server then sends `'C'` and consumes the end-of-batch "null block 0" the sender emits to signal "no more files" (Forsberg §7.4).
7. Once the transfer is complete, the file is truncated: YMODEM uses the exact size field from block 0 so files that legitimately end in `0x1A` bytes (EXEs, some archives) aren't corrupted; plain XMODEM and XMODEM-1K strip trailing `SUB` (0x1A) padding since there's no length metadata.

Cancellations are handled both ways: sending `ESC` from your side aborts the transfer, and three `CAN` bytes from either side abort with "Transfer cancelled".

> **Duplicate filename**
> If the name you type is already in the transfer directory, the server asks `Overwrite? (Y/N)` *before* starting the transfer. Answering `N` cancels cleanly; answering `Y` proceeds and truncates the existing file on save. This prevents a whole transfer from running only to fail at the final write step.

8.4 Downloads — Protocol Selection Prompt

Press `D`, then pick a file from the numbered list (paginated, 10 per page). Before the transfer starts, the gateway asks which protocol to use:

```
──────────────────────────────────────
  SELECT PROTOCOL
──────────────────────────────────────

  X  XMODEM     128-byte blocks
  1  XMODEM-1K  1024-byte blocks
  Y  YMODEM     name+size hdr, 1K
  Z  ZMODEM     autostart, 1K
  K  KERMIT     any flavor, auto
  P  PUNTER     C1 CCGMS/Novaterm

  Pick one, or ESC to cancel:
```

Pick the protocol that your terminal program supports. The choice is immediate — one key, no Enter required. ESC (or ← on PETSCII, byte 0x5F) cancels.

The selection affects what the sender does:

- **XMODEM** — the sender transmits 128-byte `SOH` blocks exclusively.
- **XMODEM-1K** — the sender transmits 1024-byte `STX` blocks, with an opportunistic 128-byte `SOH` fallback on the final short block.
- **YMODEM** — the sender begins with a block-0 `SOH 0x00 0xFF` header containing the filename and size, waits for `'C'` acknowledgement, then transmits 1K data blocks just like XMODEM-1K.
- **ZMODEM** — the sender emits a `ZRQINIT` hex header, then negotiates flags via `ZRINIT`/`ZSINIT`, sends a `ZFILE` subpacket with filename + size + mtime + mode, streams `ZDATA` with CRC-16 or CRC-32 subpackets, and finishes with `ZEOF` + `ZFIN`. ZSKIP allows the receiver to decline individual files in a batch.
- **Kermit** — the gateway waits for the receiver's initiating `NAK`, then exchanges `S`/`I` negotiation packets (block-check type, packet length, window size, capabilities), sends one or more `F`/`D`/`Z` file groups with optional `A` attribute packets for size/mtime/mode/resume, and ends with `B`. Long packets and sliding windows are negotiated from what the peer offers. (A clean download may show a single “retry” on the client — that's the receiver's own initiating `NAK`; see §8.8.)
- **Punter** — the gateway first sends a file-type block (PRG, SEQ, or USR), then streams the data blocks, each carrying a 248-byte payload behind a 7-byte header and protected by both a 16-bit additive checksum and a 16-bit cyclic checksum. Handshaking is driven by the 3-byte ASCII codes GOO / BAD / ACK / S/B / SYN. The outbound file type is auto-detected from the filename extension (text-like `.seq`/`.txt`/`.doc` → SEQ, `.usr` → USR, else PRG).

8.5 ZMODEM Autostart Detection

ZMODEM (Chuck Forsberg's 1988 protocol) is fully supported, including batch send, batch receive, and ZSKIP. Many terminal programs (Qodem, ZOC, SyncTerm, ExtraPuTTY) start a ZMODEM send by emitting the autostart sequence `**\x18B`, `**\x18A`, or `**\x18C` directly into whatever menu the user happens to be sitting at — users expect to drag a file onto the

terminal and have it transfer without first navigating to a menu. The gateway's idle input loop watches for that prefix and:

1. Logs "File transfer: ZMODEM autostart detected; switching to receive".
2. Drains any residual ZRQINIT bytes the sender already pushed (it'll retransmit once we ZRINIT).
3. Hands off to the regular ZMODEM receive flow — the sender's filename is used (after path-traversal validation), with mtime and mode applied if carried in the ZFILE subpacket.
4. Returns to whichever menu the user was on when the autostart fired.

You can also drive a ZMODEM transfer manually: from the File Transfer menu pick `U` (upload) or `D` (download), select `Z` at the protocol prompt, and start the send/receive on your terminal. Manual mode is the only option for clients that don't emit the autostart prefix (some MS-DOS-era implementations, scripted `sz` on Unix).

> **ZCOMMAND IS RECOGNIZED BUT NEVER EXECUTED**
> The ZMODEM spec defines one *optional* frame the gateway deliberately does **not** implement: `ZCOMMAND` (frame `0x12`). It is not part of ZMODEM's required core — a fully spec-compliant implementation need not support it (Forsberg's own reference receiver, `rz`, refuses it unless started with an explicit `--allow-commands` flag), and most transfers never use it. It lets a connected *sender* push a shell command for the *receiver* to run, returning the command's exit status in a `ZCOMPL` header — on Chuck Forsberg's original Unix `rzsz` reference code the command string is handed straight to `/bin/sh -c` (its classic use was a sender pushing `sz somefile` to auto-start a download). That is, in plain terms, **arbitrary remote code execution on the host**, which is an unacceptable default for a long-lived, multi-session server: unlike Forsberg's throwaway single-session `rz`, one ZMODEM session here shares the process with every other connected user. Rather than ignore the frame, the gateway handles it defensively — it recognizes `ZCOMMAND`, drains the command subpacket (so the wire stays in sync), replies a non-zero `ZCOMPL` (refused), and ends the session. Every ZCOMMAND request is rejected, and there is no configuration switch to enable it. If you need a shell on the gateway host, use the authenticated SSH interface (Chapter 6) — that is the proper, encrypted, login-gated path for command access. Every other Forsberg 1988 frame is fully implemented.

8.6 Telnet NVT Transforms

Telnet's Network Virtual Terminal (RFC 854) defines two wire-level transforms for a text stream. For binary file transfers the gateway applies only the first and treats the channel as 8-bit transparent (RFC 856 binary-transmission semantics):

- **IAC escaping (applied).** Byte `0xFF` is the IAC (Interpret As Command) marker. Any `0xFF` in data — including a CRC byte that happens to be `0xFF` — must be doubled on the wire as `0xFF 0xFF`. The receiver collapses the pair back to a single `0xFF`.
- **CR-NUL stuffing (NOT applied).** RFC 854 says a bare `0x0D` (CR) in NVT *text* should be sent as `CR NUL` (`0x0D 0x00`). The gateway does **not** do this for file transfers: the payloads are binary, and inserting a `0x00` after every `0x0D` corrupts the data for any peer that doesn't strip it back out — real C64/CP-M clients and telnet↔serial bridges (e.g. tcpser) do not. CR and every other non-IAC byte pass through literally.

The honest reality: there's no way for the server to know for certain whether a given client actually expects IAC escaping during an XMODEM/YMODEM transfer. Strict clients (PuTTY / ExtraPuTTY, Tera Term, C-Kermit, SecureCRT) do; many retro clients (CCGMS, StrikeTerm and NovaTerm on a Commodore, IMP8 on CP/M, some AltairDuino firmware) don't — they treat a TCP connection to port 23 as raw bytes. Rather than guess from the terminal type, the gateway keys the default on a signal it can actually observe: *whether the client engaged in telnet option negotiation during the opening handshake*. A client that answered our `WILL`/`DO` options (the strict clients above) gets IAC escaping **on**; a client that stayed silent — raw-TCP retro clients, and anything on a serial line, which skips negotiation entirely — gets it **off**. That observed behaviour correlates closely with the detected terminal type, so in practice the defaults line up like this:

Terminal (typical)	Typical client	Negotiates?	Auto default
ANSI	PuTTY / ExtraPuTTY, Tera Term, C-Kermit, SecureCRT, xterm	yes	IAC **on**
PETSCII	Commodore 64/128 (CCGMS, StrikeTerm, NovaTerm)	no (raw TCP)	IAC **off**
ASCII	Dumb terminals, AltairDuino, custom hardware	no (raw TCP)	IAC **off**

SSH and serial sessions skip both transforms unconditionally — SSH has no in-band command byte, and serial streams aren't subject to NVT rules.

If the auto default is wrong for your client — the most visible symptom is consistent block-CRC failures in one direction — press `I` on the File Transfer menu to flip both transforms and retry. The choice is session-local; it isn't persisted to `egateway.conf`, so the next connection from the same client gets the same auto default again (the client negotiates — or stays silent — the same way each time).

> **What you'll see if NVT is wrong**
>
> An upload that runs fine for one or two dozen blocks and then CRC-fails repeatedly is usually the first block containing a raw `0xFF` or `0x0D` byte that isn't being escaped by one side. A download that stalls after 3–4 blocks with the client repeatedly sending `'C'` instead of `ACK` is the same thing on the send side. Toggle `I` and retry.

8.7 Protocol Timeout Tunables

Transfer timings are grouped by protocol family. The *XMODEM-family* keys apply to classic XMODEM, XMODEM-1K, and YMODEM (which share a single send/receive code path in `xmodem.rs`). ZMODEM, Kermit, and Punter each have their own independent sets because their frame- or block-based protocols have different cadence requirements.

Each key can be changed from the telnet *Configuration → F File Transfer → X / Y / Z / K / P* menus, from the GUI *File Transfer → More…* popup, or by editing `egateway.conf` directly. Changes take effect on the next file transfer — no restart required.

XMODEM family (XMODEM / XMODEM-1K / YMODEM)

Key	Default	Meaning
`xmodem_negotiation_timeout`	45 s	Maximum time the receiver will wait for the first `SOH`/`STX` header from the sender, and vice versa.
`xmodem_negotiation_retry_interval`	7 s	Gap between successive `C`/`NAK` pokes during the initial handshake. Christensen's original XMODEM and Forsberg's reference implementations specify ~10 s; 7 s is a compromise that starts quickly but avoids stockpiling extras in a slow-starting sender's buffer (which can be misinterpreted as "retry block 0" requests, causing spurious duplicate-block retransmissions).
`xmodem_block_timeout`	20 s	Per-block timeout. If a block header does not arrive within this long after an `ACK`/`NAK`, the receiver retries.
`xmodem_max_retries`	10	Maximum `NAK` retries per block before the receiver cancels the transfer with three `CAN`s.

ZMODEM

Key	Default	Meaning
`zmodem_negotiation_timeout`	45 s	Maximum time to complete the ZRQINIT / ZRINIT handshake before giving up on the session.
`zmodem_negotiation_retry_interval`	5 s	Gap between ZRINIT / ZRQINIT re-sends during the handshake. Analogous to the XMODEM retry interval but for ZMODEM's frame-based protocol.
`zmodem_frame_timeout`	30 s	Per-frame read timeout once a transfer is live. Applies to headers and subpackets alike.
`zmodem_max_retries`	10	Retry cap for ZRQINIT, ZRPOS, and ZDATA frames. Once exceeded, the session cancels with a ZABORT.

Punter (C1)

Key	Default	Meaning
`punter_block_size`	255	Maximum block size in bytes (8–255). A block carries a 248-byte payload behind a 7-byte header at the default. Lower it toward ~40 on noisy lines to shrink the retransmit unit.
`punter_negotiation_timeout`	45 s	Maximum time to complete the file-type handshake before giving up on the session.
`punter_negotiation_retry_interval`	5 s	Gap between handshake re-sends while waiting for the peer to respond.
`punter_block_timeout`	20 s	Per-block read timeout once a transfer is live.
`punter_max_retries`	10	Retry cap per block (BAD/SYN re-sends) before the session aborts.
`punter_max_bad_rounds`	30	Consecutive corrupt-block resend rounds tolerated before giving up — kept higher than `punter_max_retries` because a real C64 peer never caps its resends, so a low value would make the gateway quit first and strand the peer.
`punter_hangup_on_failure`	false	On give-up, drop the connection (carrier) so a C64 — which C1 cannot be told to abort — sees loss-of-carrier and exits instead of hanging. Ends the whole session; off by default.

> **Verbose diagnostics**
> Set `verbose = true` to log every header byte, CRC result, block number, and timeout event. This is invaluable when a transfer fails on a specific block and you need to know which side hiccuped. The log identifies each event with its protocol prefix (`XMODEM recv:`, `ZMODEM send:`, etc.) so multi-protocol sessions remain traceable.

8.8 Kermit — Server & Client Mode

Beyond XMODEM/YMODEM/ZMODEM, the gateway implements a spec-complete Kermit file-transfer protocol following Frank da Cruz's *Kermit, A File Transfer Protocol* (Digital Press, 1987) plus the long-packet, sliding-window, and streaming extensions used by C-Kermit. Both directions are supported: the gateway can act as a Kermit *server* (idle waiting for commands from a connecting client) and as a Kermit *client* (connect to a remote server and pull/push files programmatically).

Why Kermit alongside XMODEM/ZMODEM?

Kermit was designed in 1981 for environments where bytes can be lost, mangled, or stripped to 7 bits — old serial links, packet-switched networks, and bridged terminal hardware. It survives conditions that XMODEM/YMODEM/ZMODEM cannot: 7-bit-only links (mainframe gateways, some telnet bridges), lossy connections that need selective retransmit, and multi-

platform interop (every retro platform that mattered has a Kermit). ZMODEM is faster on clean modern links; Kermit is the right choice for the gnarly ones.

Entering server mode

There are four entry paths into Kermit Server Mode:

- **Telnet / SSH menu** — from the File Transfer menu press `K`. Honors authentication, the network gating allowlist, and the per-session sandbox the same as any other gateway feature.
- **Standalone TCP listener** — set `kermit_server_enabled = true` and the gateway binds `kermit_server_port` (default `2424`). Every accepted connection drops straight into protocol mode — no menu, no auth gate. Use it from C-Kermit with `kermit -j host:2424`.
- **Serial dial** — with `allow_atdt_kermit = true`, dialing `ATDT KERMIT` from the modem emulator hands the serial caller to the Kermit dispatcher directly. See section 9.2.
- **Serial port Kermit Server Mode** — set a serial port's mode to `kermit` (`serial_a_mode` / `serial_b_mode = kermit`, or pick *Kermit Server Mode* in the GUI / web / telnet port settings). As soon as the port is enabled it runs a persistent Kermit server directly on the wire — no AT layer, no dialing, always on — re-arming after every FINISH/BYE and reopening the device if it disappears. Functionally the same as the serial-dial path above, but permanent rather than reached with `ATDT KERMIT`. See section 9.

The bottom three paths bypass `security_enabled` by design — they exist so vintage receivers that can't paint a menu still work. All three are off by default; the serial-dial and dedicated-listener paths have explicit GUI / telnet warnings before they enable, and serial port Kermit Server Mode should likewise only be enabled on trusted serial lines.

However the session enters server mode, the terminal displays a banner listing the supported peer commands (`send`, `get`, `remote dir`, `remote cwd`, `remote type`, `remote delete`, `remote rename`, `remote mkdir`, `remote rmdir`, `remote kermit`, `remote help`, `finish`, `bye`). The gateway then idles waiting for protocol packets; when the peer sends `finish`, `bye`, or disconnects, control returns to whatever surface launched the session: the File Transfer menu (telnet/SSH path), a closed TCP socket (standalone listener), or `NO CARRIER` (ATDT KERMIT). The File Transfer path also shows a per-file ✓/× summary of received files.

What the peer can do

Peer command	Wire packet	Effect
`send <file>`	S + F + A + D… + Z + B	Upload a file from the peer to the gateway's `transfer_dir` (or current sub-directory). A name that already exists is renamed, not overwritten or dropped — see the note below.
`get <file>`	R, then we send	Download a file from the gateway to the peer.
`remote dir`	G D	List files in the current effective directory.
`remote cd <subdir>`	G C	Change the gateway's working sub-directory for subsequent commands. Field-encoded path argument per spec §6.7. Bare `remote cd` (no argument) resets to the `transfer_dir` root; `remote cd ..` (cdup) pops one component (no-op at root); a target that doesn't exist on disk is refused with E-packet rather than silently ACKed. The optional second §6.7 field (a per-directory password) is **accepted and ignored** — see the note below.
`remote space`	G $	Report free disk bytes at the current effective directory.
`remote kermit`	G K	Identify the server (`Ethernet Gateway Kermit 0.9.3`).
`remote help`	G H / G ?	List supported subcommands.
`remote delete <file>` / `remote era`	G E	Delete a file in the current effective directory. Path-traversal rejected; missing files return E-packet (not silent ACK). One filename per call — no wildcard expansion.
`remote rename <old> <new>`	G R	Rename a file in place. Both names field-encoded per spec §6.7. Server refuses if the destination already exists, to avoid silent clobber.
`remote type <file>`	G T	Display a file's contents on the peer's terminal via the X+Z inverse-transfer pattern. Intended for text files; binaries will spew control bytes.
`remote mkdir <dir>`	G m	Create an empty subdirectory under the current effective directory. Wire letter is *lowercase* `m` (per C-Kermit's `setgen('m', …)`) — distinct from `M` reserved for `remote message`.
`remote rmdir <dir>`	G d	Remove an *empty* directory. Lowercase `d` wire letter — distinct from `D` (DIRectory listing). Non-empty directories are refused; clear contents first with `remote delete`.
`finish` / `bye` / `logout` / `remote exit`	G F / G L / G I / G X	End the protocol session politely. The gateway accepts all four since C-Kermit emits different letters for them (e.g. `logout` sends G I, `remote exit` sends G X).
any other `remote ...`	G *letter*	Refused with an explicit E-packet (per spec §6) so the peer's UI surfaces a clean “command not supported” error instead of a phantom-success ACK.

"Password:" prompt on `remote cd` (CP/M Kermit / kercpm3)

Some CP/M Kermit clients — notably **kercpm3** on RomWBW — display a **Password:** prompt when you change the remote directory (`remote cd`). *This is normal client-side behavior and is not the gateway asking you to log in.* The Kermit CWD command (Protocol Manual §6.7) is defined with two field-encoded arguments, `C <directory> <password>` — the second field is an optional *per-directory* password inherited from multi-user hosts (VMS / TOPS-20) where a directory could itself be password-protected. Clients like kercpm3 prompt for that field **before sending anything on the wire**, whether or not the server wants it.

The gateway's Kermit server is **unauthenticated by design** (see §9), so it reads only the directory field and **ignores the password entirely**: the directory change succeeds no matter what you type. Simply **press Enter** at the prompt to send an empty password. This is independent of the *Require Login* (`security_enabled`) setting, which gates telnet, SSH, and the web UI only — it never applies to the Kermit server, and is deliberately *not* wired to this per-directory password field.

Because Kermit runs in cleartext over serial/telnet, do **not** type a real secret at this prompt — it would travel unencrypted and serves no purpose, since the gateway discards it. Press Enter instead.

Upload name collisions — renamed, never dropped

When an uploaded file's name already exists in the target directory, the gateway **renames** the incoming file rather than dropping it or overwriting the original. The new name follows the DOS / CP-M 8.3 convention CP/M Kermit clients (e.g. kercpm3) use on a download collision: the base name (before the last dot) is kept within 8 characters, appending the next free number when it fits and replacing the trailing base characters when it doesn't. The extension is left untouched.

```
abcdefgh.txt exists  ->  abcdefg0.txt, abcdefg1.txt, ... abcdefg9.txt,
                         then abcdef10.txt, abcdef11.txt, ...
hi.txt exists        ->  hi0.txt, hi1.txt, ... (base is under 8 chars,
                         so the number is simply appended)
README exists        ->  README0  (a name with no extension is numbered
                         the same way)
```

The original file is never overwritten, and the gateway keeps trying numbers until it finds a free name. The one exception is a **verified resume** (`kermit_resume_partial`): that intentionally replaces its own partial file in place by exact name. This behavior is identical across all three Kermit-server entry paths — the telnet/SSH File Transfer menu, the standalone TCP listener, and serial Kermit Server Mode / `ATDT KERMIT` (§9).

SET *BINARY* FILE MODE ON THE CLIENT BEFORE TRANSFERRING PROGRAMS OR DATA

Vintage Kermit clients default to **text** (ASCII) file mode. Sending a `.COM`, `.DAT`, disk image, or any other non-text file in text mode **silently loses bytes** — every packet's block check is valid, the transfer reports success, and the file that lands is subtly wrong. Always switch the client to binary first:

```
  kercpm3 / CP/M Kermit    SET FILE MODE BINARY      (some builds: SET FILE TYPE
BINARY)
  C-Kermit / Kermit 95     set file type binary      (or the -i command-line
flag)
  MS-DOS Kermit            set file type binary
```

The gateway itself always transfers **binary, byte-for-byte, in both directions** — it never translates line endings and never trims padding. This is purely a client-side setting, so a download can be perfect while an upload of the very same file is damaged.

The damage is not a truncation you would notice at a glance. A CP/M client reads the file in 128-byte records, and in text mode it treats a `^Z` (0x1A, the CP/M text end-of-file pad) as padding rather than data — so a `^Z` that happens to fall on a record boundary is *deleted* and every following byte shifts down one position. A real example: an 8,704-byte `WITNESS.COM` and a 104,960-byte `WITNESS.DAT` uploaded from an SC126 in text mode arrived 1 and 8 bytes short respectively — nine dropped bytes out of 113,664, enough to stop the program running but far too few to spot by comparing file sizes casually. Re-uploading with `SET FILE MODE BINARY` produced files byte-identical to the originals.

The gateway catches this *when the client tells it how big the file should be*: if an upload ends up shorter than the length declared in the attribute packet, the file is refused with an E-packet and never written (§8.8, `kermit_attribute_packets`), and the log names binary mode as the fix. Many small clients send no length attribute at all, in which case there is nothing for the receiver to check the arriving size against — so treat binary mode as a habit rather than relying on the guard. To verify a transfer independently, compare checksums on both ends rather than comparing packet counts: the packet count legitimately differs between upload and download for the same file, because control and high-bit bytes are quoted into two or three wire characters each and the amount of that expansion depends on the negotiated packet length and on which direction is sending.

Capabilities advertised

The gateway negotiates the full set of modern Kermit extensions: long packets (up to 9024 bytes per packet), sliding windows (up to 31 outstanding packets with selective-repeat retransmit), streaming (no per-packet ACKs on reliable links), CRC-16 block check, attribute packets carrying file size + mtime + mode + record format + creator ID + ~10 other typed metadata fields, repeat-count compression, RESEND (resume partial uploads), and locking shifts (SO/SI region quoting for 8-bit transit on 7-bit links).

Two of these are off by default and gated behind config keys: `kermit_resume_partial` for RESEND (off because a peer that doesn't honor `disposition='R'` would corrupt the result),

and `kermit_locking_shifts` for SO/SI quoting (off because no modern peer uses it — QBIN per-byte 8th-bit prefixing covers the same use case).

C-Kermit interop

Real C-Kermit 10.0 interop is verified by the test suite for both directions. Example invocations:

```
$ kermit -B -j 192.168.1.160:2323 -i -q -s ./localfile.bin
# Upload localfile.bin to the gateway. -B batch, -j telnet-protocol TCP,
# -i image (binary), -q quiet, -s send action.

$ kermit -B -j 192.168.1.160:2323 -i -q -g report.txt -a /tmp/report.txt
# Pull report.txt from the gateway and save locally as /tmp/report.txt.
# -g get action, -a alternate local name. The gateway must be in
# Kermit Server Mode (File Transfer menu → K) when ckermit dials in.
```

Why a download shows "1 retry" on some clients

When a vintage client (for example `kercpm3` on an SC126/CP-M machine) pulls a file from the Kermit server, its status display may report *Number of retries: 1* even though the transfer is completely clean. This is normal and expected — it is **not** an error or a lost packet. Kermit's file transfer is *receiver-driven at the start* (Frank da Cruz, *Kermit Protocol Manual* §4): the receiving side sends a single `NAK` to solicit the sender's Send-Init (`S`) packet. The gateway waits for that solicitation and answers it with exactly one Send-Init, so nothing is retransmitted — but the client still counts the `NAK` it sent to get things going, and shows it as one retry. **Uploads** read *0 retries* because there the client is the *sender* and initiates with the Send-Init directly, so it never has to poke. Earlier gateway builds showed 2–3 retries here because a case-sensitive filename lookup and an unsolicited Send-Init (which crossed the client's poke and was resent as a duplicate) each cost an extra retry; both are fixed, leaving only the client's own initiating `NAK` .

Full Kermit reference

A standalone reference page `kermit.html` documents every G subcommand, the full capability table, all 18 `kermit_*` configuration keys with defaults (including `kermit_idle_timeout` which accepts `0` to disable the server-mode disconnect), additional examples, and spec citations. See it on the project website alongside this manual.

8.9 Punter (C1) — Commodore

Punter is the file-transfer protocol that Commodore 64/128 terminal programs — CCGMS, Novaterm, and StrikeTerm — speak natively on the BBSes of that era. The gateway implements single-file **C1** ("New Punter"): not the older PET `PTP` and not the multi-file `MPP` variant. It works over both the telnet path (with IAC escaping) and the serial-modem path, and shares the same 8 MB maximum file size as the other protocols.

How a C1 transfer runs

C1 is a stop-and-wait, block-at-a-time protocol coordinated entirely by 3-byte ASCII handshake codes:

Code	Meaning
`GOO`	Block received intact; send the next one.
`BAD`	Checksum mismatch; retransmit the last block.
`ACK`	Acknowledge a control transition.
`S/B`	"Send/Begin" — ready to receive the next block.
`SYN`	Re-synchronise after a stall or a lost block.

1. **Two-phase handshake.** The transfer opens with a file-type block, then proceeds to the data blocks. Each block is protected by *two* independent 16-bit checksums — an additive checksum and a cyclic checksum — so a block is only accepted (`GOO`) when both agree; otherwise the receiver answers `BAD` and the sender retransmits.
2. **Block layout.** Blocks are up to 255 bytes — a 248-byte payload plus a 7-byte header. The block size is configurable from 8–255 (default 255) via `punter_block_size`; lower it toward ~40 on a noisy line so each retransmit costs less.
3. **File-type detection on download.** When sending a file to the C64, the outbound file type is auto-detected from the filename: text-like extensions (`.seq`, `.txt`, `.doc`) are sent as **SEQ**, `.usr` as **USR**, and everything else as **PRG**.

Selecting Punter

Punter appears in both the upload and download protocol pickers under the key `P` ("PUNTER"). From the File Transfer menu, press `U` or `D`, choose your file, and pick `P` at the protocol prompt; then start the matching transfer in your Commodore terminal. The per-protocol tunables (block size, negotiation timeout, retry interval, block timeout, max retries) live on the *Configuration → F File Transfer → P Punter* screen, in the GUI's File Transfer *More…* popup, and on the web config page — see §8.7 for the `punter_*` key reference.

> **Cancelling and restarting**
>
> Punter has no in-band cancel signal — when you abort a transfer on the Commodore (`RUN/STOP` in CCGMS), the terminal stops locally and sends *nothing* over the wire, so the gateway cannot tell you bailed. It keeps the old transfer alive until it times out (up to `punter_block_timeout`, default 20 s), then prints `Transfer failed` and returns to the menu. **Wait for that `Transfer failed` message before restarting.** If you re-arm your Commodore receiver while the gateway is still finishing the aborted transfer, its fresh handshake collides with the dying one and the two ends desync. The reliable sequence is: cancel on the Commodore → wait for the gateway to report `Transfer failed` and redraw the menu → start the new transfer on the gateway → then start the matching transfer on the Commodore. (To shorten the wait, lower `punter_block_timeout`.) The gateway clears any stray bytes the aborted transfer left in the line before each new Punter transfer, so a clean restart resyncs reliably.

8.10 Gateway Shell (drive A:)

The Gateway Shell gives a logged-in telnet or SSH user a familiar `A>` command prompt for managing the files in the transfer directory. It is a **CP/M-inspired file manager written entirely in Rust** — the command style and `A>` prompt are modelled on CP/M, but it is not a CP/M-compatible shell: there is *no* Z80 CPU emulation and it does *not* run `.COM` programs. It simply presents the configured `transfer_dir` as drive **A:** and lets you list, view, copy, rename, move, erase, and organise files. It works identically over telnet and SSH.

Open it from the **File Transfer** menu with `S` ("Gateway Shell"). Type `HELP` for the command list and `EXIT` (or press `ESC`) to return to the File Transfer menu — leaving the shell does *not* disconnect your session. The prompt shows where you are: `A>` at the drive root, and `A:SUB>` when you have changed into a subdirectory.

Command set

Command	Aliases	Action
`DIR [pattern]`	`LS`	List the current directory, or a wildcard / path match. Naming a subdirectory lists its contents (`DIR SUB` is the same as `DIR SUB/`). Directories show as `<DIR>`, files with their size.
`FIND pattern`	`WHERE`	Recursively search *all* of drive A: (not just the current directory) for files whose name matches the wildcard, printing each hit's full A: path. Matches files only; the walk is bounded (scan + result caps) and never follows symlinks, so it stays inside the transfer-directory jail.
`TYPE file`	—	Display a *text* file a screenful at a time (see pagination below). Binary files are refused.
`DUMP file`	—	Hex + ASCII dump, paginated (8 bytes/row on a 40-column PETSCII screen, 16 on wider terminals).
`ERA file`	`DEL`, `RM`	Erase a file, or every file matching a wildcard (mass erase asks `(Y/N)` first). Refuses directories.
`REN new=old`	`RENAME`	Rename a file *within one directory* (also accepts `REN old new`). Refuses to overwrite an existing name. Cross-directory relocation is `MOVE`.
`COPY dst src`	`PIP dst=src`, `CP`	Copy a file; either operand may be path-qualified. Never overwrites an existing destination. A wildcard source copies each match into a directory destination.
`MOVE dst src`	`MV`	Relocate a file across directories. Never overwrites an existing destination. (An invented verb — stock CP/M has no move.)
`MKDIR name`	`MD`	Create a subdirectory.
`RMDIR name`	`RD`	Remove an *empty* subdirectory (refuses a non-empty directory, the current directory, and the drive root).
`CD [path]`	`CHDIR`	Change directory. `CD` or `CD /` returns to the root, `CD ..` goes up one level, `CD name` descends.
`PWD`	—	Print the current drive-A path.
`STAT [file]`	—	With no argument, summarise the directory (file count, bytes used, disk-space indicator); with a file, show its size and 128-byte record count.
`CLS`	`CLEAR`	Clear the screen.
`VER`	`VERSION`	Show the shell identity and gateway version (e.g. `Ethernet Gateway Shell 0.9.3`).
`HELP`	`?`	Show the command reference.
`EXIT`	`BYE`, `QUIT`, `ESC`	Return to the File Transfer menu.

CP/M's `USER` command (which selects one of the 0–15 file "user areas") is **recognised but not supported** — the shell presents the transfer directory as a single flat area, so typing

`USER` replies with a short notice rather than the `?` unknown-command error. It is the only recognised verb not in the table above.

Commands and filenames are case-insensitive: because `DIR` shows names uppercased, `CD Z80ASM`, `CD z80asm`, and a directory stored on disk as `z80asm` all refer to the same directory (an exact-case match wins, otherwise the first case-insensitive match is used). Names you *create* (`MKDIR`, a `COPY`/`MOVE` destination) are kept in the case you type. An unrecognised command is echoed back with a `?`, exactly as the CP/M console command processor does (e.g. `FOO?`).

Path syntax

Stock CP/M filenames carry no directory, so the shell extends them with a Unix-style path syntax using `/` as the separator (matching how the gateway stores the transfer working directory). This is what lets `COPY`, `MOVE`, and `CD` work *between* directories — something the base CP/M command set cannot express:

Operand	Resolves to
`FILE.TXT`	a file in the current directory
`SUB/FILE.TXT`	a file in / into subdirectory `SUB`
`/FILE.TXT`	a file at the drive-A root
`../FILE.TXT`	a file in the parent directory
`SUB/`	a directory target — “into `SUB`, keeping the source name”

`COPY` and `MOVE` take the **destination first** (the CP/M `PIP dest=source` order), so `COPY SUB/ FILE.TXT` copies `FILE.TXT` into `SUB`, and `MOVE /DONE/ OLD.DAT` moves `OLD.DAT` to the `DONE` directory at the root. Because this order is the reverse of what most users expect today, the shell shows two reminder examples when you enter it, and a failing `COPY` / `MOVE` (for instance `File not found.` after the operands were reversed) echoes the correct form — `e.g. COPY dst src (dest first)`.

Wildcards

`DIR`, `ERA`, and the `COPY` source accept the CP/M / DOS wildcards `*` (any run of characters) and `?` (exactly one character), matched case-insensitively against the whole filename — e.g. `DIR *.TXT`, `ERA *.BAK`, `COPY BACKUP/ *.DAT`. A wildcard `ERA` confirms before deleting.

Pagination

`TYPE`, `DUMP`, and `DIR` page their output one screenful at a time. At each `--More--` prompt, press `SPACE` for the next page, `RETURN` to advance a single line, or `Q` (or `ESC`) to stop.

Jailed to the transfer directory

Every command is confined to the transfer directory and its subdirectories; nothing above the drive-A root can ever be reached. Path operands are validated and canonicalised, and any attempt to climb out (`../../..`, an absolute host path, or a symlink pointing outside) is refused with `Access denied.` Filenames follow the same rules as the rest of the file-transfer subsystem — up to 64 characters of letters, digits, dots, hyphens, and underscores. Uploaded files are shown with uppercased names for CP/M authenticity, but their real (mixed-case) names are preserved on disk, so the shell and the File Transfer menu always see the same files.

Text-file guard

`TYPE` refuses to display a binary file (one containing NUL bytes or a high proportion of control characters) so it can't spray terminal-hostile bytes at a vintage screen — download such files with a transfer protocol instead. The `TYPE` and `DUMP` viewers cap the files they will read at **1 MiB** (paging a larger file over a retro link isn't useful — transfer it instead); `DUMP` shows 8 bytes per row on a 40-column PETSCII screen and 16 on wider terminals.

8.11 CP/M Emulator

The CP/M Emulator is a **real CP/M 2.2 environment running on an emulated Zilog Z80** (the BSD-licensed `iz80` CPU core driven by the gateway's own CP/M 2.2 BDOS). Unlike the Gateway Shell of chapter 8.10 — a Rust *file manager* that only *looks* like CP/M — this feature actually **runs user-supplied `.COM` software**: PIP, STAT, ASM, WordStar, MBASIC, and the rest. The two features are independent and have separate sandboxes; don't confuse them.

On by default — runs arbitrary Z80 code

Because a `.COM` is arbitrary machine code, the emulator is bounded on three axes rather than switched off: every file call is jailed under `CPM/`, a runaway program is stopped by `cpm_emu_max_minstr`, and a double-`ESC` always returns you to `A>`. It services BDOS and BIOS calls only — there is no path from guest code to a host command. An operator who wants the door shut sets `cpm_emu_uart = off` (no sockets from guest code) or `cpm_emu_enabled = false` (no guest code at all); while off, the main-menu item is hidden and the key is rejected. Both live in the GUI and web UI under the **"AI, Browser, Weather & CP/M — More"** panel, or over telnet/SSH in **Configuration → Other Settings →** `E`. Treat it as a trusted-LAN feature.

Launching

With the feature enabled, the main menu gains a `K` "CP/M System" item. Selecting it boots the machine and drops you at the CP/M `A>` prompt. Type `HELP` for the built-in command list; `EXIT` (or `BYE` / `QUIT`) returns to the gateway. The prompt tracks the current drive (`A>` … `H>`).

Drives & staging a program

The emulator has sixteen drives, **A:–P:** (the maximum CP/M 2.2 allows), each mapped to a folder under a dedicated `CPM/` container inside `transfer_dir` (`CPM/A` … `CPM/P`, created as soon as the emulator is enabled, together with `CPM/images` for disk images). Each is ready to use the moment it exists—an empty folder is a formatted, empty drive, so there is nothing to

format or `CLRDIR`. This is a *separate* sandbox from the Gateway Shell's drive A: (which is the whole transfer directory). To run a program:

1. Upload the `.COM` to the transfer area with any protocol (chapter 8.3).
2. Use the Gateway Shell (chapter 8.10) to `COPY CPM/A/ PROG.COM` it into a CP/M drive.
3. Enter the emulator (`K`) and type its name.

Mounting a disk image

A drive can use a **CP/M disk image** instead of its folder. Put `.dsk` files in `CPM/images` (the `readme.txt` written there explains the naming), then mount one from **Configuration → CP/M Emulator →** `I` over telnet/SSH, or from **Mount CP/M Drives** in the web and desktop interfaces. Mounting *hides* the drive folder; it does not touch it, and the folder's files return the moment you unmount.

Name an image `<format>_<anything>.dsk` — `ibm3740_cpm22.dsk`, `altairhd_cobol.dsk`. Six formats are supported, all measured from real images rather than transcribed from another tool's tables: `ibm3740` (the 256,256-byte 8″ single-density disk used by Tarbell, Cromemco and the IMSAI/z80pack library), `altairhd` (the 4,988,928-byte Altair 88-HDSK hard disk from the Altair-Duino set), `altair8` (the 337,568-byte Altair 88-DCDD 8″ floppy, which reads, writes and formats), and the two Cromemco double-density 8″ disks — `cromemcodd` (625,920 bytes, single sided) and `cromemcodsdd` (1,256,704 bytes, double sided), plus `z80packhd` (the 4,177,920-byte z80pack `cpmsim` hard disk, which has no reserved tracks at all — its directory starts at byte zero). Those last two have their track 0 recorded in *single* density so a single-density boot ROM can read them at all, and each interleaves its sectors differently from the other: skew belongs to the BIOS, not to the medium.

> **YOU DO NOT HAVE TO RENAME ANYTHING**
>
> An image without a format prefix is identified by *inspection*, and if its CP/M filesystem checks out it mounts read-write just the same. No two formats here are the same size, so the size names the format outright; what the inspection decides is whether the file really holds that filesystem — the whole directory is read and checked, every allocation block accounted for, no block claimed twice, every record count matching the blocks it claims. Random bytes under the wrong geometry fail that immediately.
>
> An image that does *not* check out still mounts, but read-only, and says what was wrong with it. That is the honest answer for a file which is the right size and is not this filesystem: plenty of disks are 256,256 bytes without being CP/M at all. A mount is also read-only when the file is read-only on the host. Renaming with a prefix is therefore an *override* — it says "I know what this is" and skips the inspection — not a requirement.

Mount changes take effect immediately in every session and are saved to `cpm_mounts`, so they survive a restart. A drive somebody is currently using cannot have its disk changed; the screens show which drives are in use. Note that mounting on **drive A:** hides the bundled terminal, which lives in the A: folder — and since files move in and out of a mounted image with XMODEM from inside the terminal, you usually want B: or later.

The mount screen changes with what is set to boot

This surprises people, so it is worth saying plainly: **the mount screens are not the same screen from one moment to the next.** What they offer, and what they call a drive, both follow `cpm_boot_image`.

With the **CP/M emulator** set to run — the default — every image in the folder is offered and the slots are drives `A:` – `P:`, because our own CP/M is underneath them and reads each image's filesystem directly.

With a **disk set to boot**, the slots take the name that disk's own board uses: `unit 0.0` … `unit 3.3` beside an Altair 88-HDSK, whose slots are *platters* rather than drives; `drive 0` … `drive 15` on the z80pack controller; `drive 0` – `drive 3` on an 88-DCDD floppy controller. The telnet screen even retitles itself *CHOOSE A SLOT*. The underlying setting is still a drive letter — `cpm_mounts` says `B=name.dsk` either way — but what the letter *means* belongs to the hardware, so calling slot 1 "B:" while an Altair hard disk is booting would be a promise the guest never made.

And the list gets shorter: only images on the *booted disk's own board* are offered, because the board an image lands on is chosen by its **size**. Mount a floppy while a hard disk is booting and it goes to the floppy controller, which that guest never reads — mounted, correct and invisible. Anything withheld is counted and explained on all three interfaces rather than quietly missing. To give a booted hard disk a second disk, mount another *hard disk* image.

Not every `.dsk` holds a CP/M filesystem. Altair DOS, Altair Disk BASIC, Time Sharing BASIC and UCSD p-System disks are common in the same collections and use their own formats; *mounting* them either fails or shows no files, which is correct rather than a fault. Those disks are for **booting** instead.

Booting a disk image

Mounting gives you one drive with the gateway's BDOS underneath. **Booting** hands the whole machine to the disk: 64 KB of memory, a disk controller, a console and the front-panel sense switches. The disk's own operating system then runs and does its own filesystem work — which is exactly why booting reaches the disks mounting cannot, and why it needs no knowledge of their layout.

Five disk controllers are emulated, and which of them a machine carries is set by `cpm_boot_machine` — which defaults to `auto`, so ordinarily you set nothing at all and the disk is asked: a boot loader has to drive its own controller's registers, so the image states what it is for. The boards are the MITS **88-DCDD** floppy controller, the MITS **88-HDSK** hard disk (the "Datakeeper"), the **Tarbell 1011**, the **Cromemco 4FDC/16FDC**, and the disk device from Udo Munk's **z80pack** simulator. Size alone cannot choose between them — a 256,256-byte file is a valid disk to three of the five — which is why the loader is read instead.

Thirty of the thirty-four images in David Hansel's Altair sample set boot and sign on, with nothing configured (a different thirty-four from the count above: the download offer draws on *two* collections): Altair CP/M 2.2, **CP/M 3.0**, CP/M 2.2AT, **Altair DOS**, **Altair Disk Extended**

BASIC, **Time Sharing BASIC** V1.1 and V2, 63K CP/M 2.2b and **Altair Hard Disk BASIC** on the 4.9 MB hard disks, four Tarbell CP/M systems, **Comal 80**, and all three Cromemco disks — **CDOS** and two more CP/M 2.2 systems. Of the four that do not, all four are correctly refused: every one is a *programs* disk — data rather than a system disk, the companion of a disk that does boot — and carries no boot program of its own. (One of them, `TDISK06`, was described here as "blank" until it was looked at: it is a full disk of VDM-1 programs, and it is the mount screen that wants it, not the boot list.) A fifth used to wait on a video board we did not emulate; it now boots and paints its screen in the web UI (§6.4.3). Separately, every one of the eleven 4.9 MB `HDSK*` hard-disk images boots, as do nine disks in z80pack's own library — including **MP/M** and **UCSD p-System IV**.

Choose what the CP/M menu item runs with `cpm_boot_image`: empty (the default) runs the CP/M emulator, and a bare filename in `CPM/images` boots that disk. On telnet it is `R` on the CP/M Boot Settings screen, which opens a **Choose what CP/M runs** list — the emulator first, then every disk that boots, with the current setting marked — and a **CP/M runs** dropdown in the web and desktop UIs. To boot something There is one way to boot, on purpose: a second, per-visit boot picker lived on the telnet disks screen until 0.9.2 and was removed — two boots that asked different questions and remembered different things was the single most confusing thing about the whole feature.

Both of those lists show only disks that will actually boot. A collection of Altair disks contains *companion* disks — a disk of programs for a system disk, carrying no boot program of its own — and they are the same size as the disk they belong to, so nothing about the file says which is which. The gateway settles it by doing the cold start: an image is offered only if the very sequence a real boot would run gets as far as an entry point. A companion disk is therefore missing from both lists, and belongs on the *mount* screen instead, which is where it is useful.

A disk this machine has no *board* for is a different case and is still listed. That one you can fix by changing `cpm_boot_machine`, and a disk that quietly vanished from the list when you changed a setting would be a worse puzzle than a boot that stops and names the boards the machine has.

If the disk it names is no longer in `CPM/images`, **the emulator runs instead** — deleting an image costs you the boot, not the whole CP/M feature. Every screen that shows the setting marks it `(missing)` in that case, or `(invalid name)` if the value could never have named a disk (a path where a bare filename belongs, say), and the log records the fallback once per visit. The disk screens follow what will really start, naming drives `A:`–`P:` again rather than the slots of a board that is not going to be booted.

The mount screens follow what is set to boot. Which images they offer, and what they call a drive, both depend on it — because the board an image lands on is chosen by its *size*. Under the CP/M emulator every image is offered and the slots are drives `A:`–`P:`. With a disk set to boot, only images on that disk's own board are offered, and the slots take that board's names: beside a booted Altair 88-HDSK they run `unit 0.0` to `unit 3.3`, because an 88-HDSK slot is a *platter* and not a drive. Anything withheld is counted and explained rather than silently missing.

This matters because the alternative was a baffling failure: mount a floppy while a hard disk is booting and it goes to the 88-DCDD, which that guest never reads. The disk is mounted, the file is fine, and the guest cannot see it — while its own `B:` is an empty platter and answers `Bdos Err on B: Bad Sector`. To give a booted hard disk a second disk, mount another *hard disk* image.

Booting is not mounting. Inside a booted disk there is no jail, no `A>` from us, no `EXIT`, no EGT8080 and no drives A:–P: of ours — the guest owns the machine and talks to hardware. Press **ESC twice** to return to the gateway. The image is opened **writable** and held by **one session** at a time: a booted guest writes raw sectors and nothing above it understands the format well enough to catch a mistake, but an operating system that cannot save loses the work silently, which is worse — so writes are allowed unless you say otherwise with **Disk writes** (`cpm_boot_writable`) on the CP/M Boot Settings screen. Note that it covers the booted disk *and* every image mounted beside it, for everyone who reaches CP/M, until you change it again. There is no instruction ceiling either — an operating system sitting at its own prompt is meant to run indefinitely, so an abandoned session is closed by the ordinary idle timeout instead.

The virtual modem comes along, as long as `cpm_emu_uart` names a pair of *ports*. A real Altair put its modem on the second port of its 88-2SIO, so `altair_2sio2` (0x12/0x13) is the natural choice: comms software under a booted Altair CP/M then finds a UART where it expects one and dials out through the gateway. `aux` and `hbios_1`/ `hbios_2` cannot — they are the gateway's own BDOS device and RomWBW firmware, and a booted disk brings its own of both — and neither can `altair_2sio1`, because 0x10/0x11 *is* the console on this machine. In each case the boot banner says so.

The Backspace key is a setting, because no one answer fits every disk. Type `TESTING` into a booted Altair disk and backspace over it, and the screen reads `TESTINGGNIT`. Nothing is broken: your terminal's Backspace key sends DEL (`7Fh`), and most of these operating systems read that as a Teletype *rubout* — they delete the character and then print the character they deleted, so the operator of a printing terminal can see what came off the line. `cpm_boot_backspace` decides which byte a booted guest is handed. Set it to match the disk you are booting: a CP/M 1.x disk wants `rubout`, and most others want `backspace`.

The default is `backspace`, which was measured rather than reasoned. Of the 38 images in the two disk folders that reach a prompt, 29 erase on BS (`08h`) — MITS CP/M 2.2, Altair Disk Extended BASIC and Altair Hard Disk BASIC among them — and 7 accept either, Digital Research's own CP/M 2.2, MP/M and UCSD p-System IV being the generous ones. **CP/M 1.3, 1.4 and the 1975 build are the exception that gives the setting its reason**: there the rubout *is* the editing key and BS prints a literal `^H`, so translating breaks something that already worked. A Commodore's own DEL key (PETSCII `14h`) is folded to whichever byte the setting names, under either setting, because no guest recognises `14h` at all. The CP/M *emulator* is unaffected throughout: it reads its own console line and has always accepted both bytes.

Typing a name that isn't a built-in loads `<name>.COM` from the current drive (a `B:` prefix selects a drive), sets up page zero exactly as CP/M's console command processor does — the

command tail at `0080h` and the two default FCBs at `005Ch` / `006Ch` — and runs it. A name with no matching `.COM` echoes back with a `?`, as CP/M does.

Built-in commands

These commands are handled by the built-in command processor and work with **no `.COM` file present** on the drive — anything else you type is loaded as a program (above). The **CP/M** column marks the ones that are authentic CP/M 2.2 resident commands; the others are gateway conveniences.

Command	CP/M	Action
`A:` … `P:`	✓	Change the current drive (the CP/M `d:` resident command). All sixteen exist, each a folder under `transfer_dir/CPM`.
`DIR` [*d:*][*afn*]	✓	List files. With no operand, everything on the current drive; with a filespec, only matching names (`DIR *.COM`); with a drive, that drive without selecting it (`DIR B:`, `DIR B:*.TXT`). Wildcards are the same ?/* the BDOS directory search and `ERA` use. A malformed filespec is reported rather than treated as “everything”.
`ERA name` (`DEL`)	✓	Erase file(s), wildcards allowed (`*.*` confirms first).
`REN new=old` (`RENAME`)	✓	Rename a file (also accepts `REN new old`). Refuses to overwrite an existing name.
`TYPE file`	✓	Stream a text file to the console, stopping at the `^Z` end-of-file marker. Binary files are refused.
`SAVE n file`	✓	Write `n` 256-byte pages of the TPA (from `0100h`) to a file — captures the image a prior program (e.g. `DDT`) left in memory.
`USER n`	✓	Select user area 0–15. Only area 0 exists (each drive is one flat area); a higher number reports that.
`VER` (`VERSION`)	—	Show the emulator identity (CP/M 2.2, iz80 core).
`HELP` (`?`)	—	Show the built-in command list.
`HELLO`	—	Demo: print a banner via BDOS, then warm-boot.
`ECHO`	—	Demo: echo typed keys (press `.` to end).
`EXIT` (`BYE`, `QUIT`)	—	Leave the emulator and return to the gateway.
`name`	—	Any other word loads and runs `name.COM` from the drive.
batch (`$$ $.SUB`)	✓	While `A:$$$.SUB` exists, each command comes from it instead of the keyboard, echoed as it runs. Written by CP/M’s own `SUBMIT.COM` — see below.

Batch files (`SUBMIT`)

CP/M 2.2 batch jobs work. `SUBMIT.COM` is a transient you supply (it ships with CP/M 2.2 distributions and with RomWBW): running `SUBMIT name` reads `name.SUB` and compiles it into a file called `$$$.SUB`, one command per 128-byte record. The command processor then takes each command from that file instead of the keyboard, echoing it as it runs, and erases the file when the batch finishes.

Three behaviours are inherited from real CP/M 2.2 rather than invented, and are worth knowing because two of them surprise people:

- **An error ends the batch.** A command the processor does not recognise erases `$$$.SUB` and returns you to the keyboard; the remaining lines do not run. This is CP/M's behaviour, not a limitation of the emulator.
- **Batches only work from drive A:.** The command processor reads `$$$.SUB` from A: whatever drive is current, while `SUBMIT.COM` writes it to the *current* drive. Start a submit from B: and the file lands on B:, where nothing reads it — so nothing happens. That is exactly how real CP/M 2.2 behaves.
- **Leaving the emulator abandons a running batch** rather than resuming it in a later session.

The richer `SUBMIT` of CP/M 3 — conditional `IF`/`ELSE`, `$1`…`$9` parameters, `PROFILE.SUB` at boot — is not part of CP/M 2.2 and is not emulated. Feeding keystrokes to a running *program* (as opposed to command lines to the processor) needed CP/M 2.2's separate `XSUB.COM`, which is not supported.

All six authentic CP/M 2.2 resident commands (`DIR`, `ERA`, `REN`, `TYPE`, `SAVE`, `USER`) — plus the `d:` drive change — are built in, so nothing needs uploading for everyday file work. Because the machine stays resident between commands (the transient program area survives a warm boot back to `A>`, as on real CP/M), `SAVE` can dump the memory image a previous program left behind. User areas beyond 0 aren't modelled — each drive is a single flat area.

Terminal — install for an ADM-3A

Full-screen CP/M software (WordStar, Turbo Pascal, dBASE, editors) is installed for a specific terminal and emits that terminal's cursor and screen-control codes. The emulator presents the guest with a **Lear Siegler ADM-3A** — the universal lowest-common-denominator CP/M terminal — so **configure your software for an ADM-3A** (many also list it as "Lear Siegler," "ADM-3A," or a compatible like Kaypro/Televideo in a pinch).

The gateway translates the ADM-3A control stream into whatever *your* terminal speaks — ANSI cursor sequences for a modern terminal, native cursor codes for a Commodore 64 (PETSCII), best-effort for a dumb ASCII TTY — and translates your arrow keys the other way, into the ADM-3A cursor codes the program reads. Line-oriented tools (PIP, STAT, MBASIC) need no terminal at all and work everywhere. (A C64's 40-column screen can't show an 80-column editor in full, but line-oriented use is fine.)

Breaking out & the runaway ceiling

Press `ESC` **twice** at any time to stop a running program and return to `A>` — this works even for a program stuck in a loop that never reads the keyboard (the gateway watches for it out-of-band between CPU bursts). A single `ESC` is still delivered to the program, so CP/M editors keep working; the C64 PETSCII escape byte works too. As a last-resort backstop, a compute-bound program is also bounded by a **runaway instruction ceiling** — `cpm_emu_max_minstr`, in millions of instructions per run (default 2000 = 2 billion) — after which it is aborted automatically. Raise or lower it in the same CP/M settings panel as the enable toggle.

> **Which `.COM` files to upload**
>
> The gateway ships *no* CP/M software — you supply your own. The resident commands (`DIR`, `ERA`, …) are provided by the built-in prompt above. The freely redistributable Digital Research transient utilities are the usual first uploads: `PIP.COM` (copy), `STAT.COM` (status), `ED.COM` (editor), `ASM.COM` (assembler), `LOAD.COM` (HEX→COM), `DDT.COM` (debugger), `DUMP.COM`, and `SUBMIT.COM` (batch jobs — these run, see above; `XSUB.COM` does *not*, as it patches itself in below the command processor). Full applications you own — WordStar, MBASIC, Turbo Pascal, dBASE II, SuperCalc — run too. Copy them off your own RC2014 / SC126 / RomWBW disk images or a CP/M archive. Disk/machine tools (`FORMAT`, `SYSGEN`, `MOVCPM`) are meaningless in a directory-backed system and are never needed.

> **Sandbox & limits**
>
> Every BDOS file call is confined to the `CPM/` container: 8.3 filenames are enforced (a host file that isn't a valid 8.3 name is invisible to CP/M), paths are canonicalised, and a symlink planted in a drive folder cannot point out of the jail. A single emulated file is capped at **8 MB** so a stray high random-record write can't fill the host disk, and each session's machine is a fixed 64 KB (bounded by `max_sessions`). The emulator services only the CP/M BDOS — it has no way to run host commands.

Virtual modem (outbound)

A CP/M communications program (a terminal, kercpm3, MEX) reaches a modem by reading and writing a **UART at a fixed I/O port address**. Because different machines place that UART at different addresses, the `cpm_emu_uart` setting lets you pick **which machine/port the emulated modem answers at**, chosen from a list with a description beside each entry (in the same CP/M settings panel as the enable toggle):

Profile	Machine / port	Status / data
`off`	no virtual modem	—
`rc2014_1a` … `rc2014_2b`	RC2014 / RomWBW Z80 SIO/2 (two boards, channels A/B). **`rc2014_1b` is the default** — the usual `AUX:` port, and the one the bundled EGT8080 terminal expects	`0x80/0x81` … `0x86/0x87`
`altair_2sio1` / `altair_2sio2`	Altair 88-2SIO (6850 ACIA)	`0x10/0x11`, `0x12/0x13`
`altair_sio`	Altair 88-SIO	`0x00/0x01`
`aux`	BDOS `AUX:` device (funcs 3/4) — SC126 / RomWBW, hardware-independent	—
`hbios_1` / `hbios_2`	RomWBW **HBIOS** serial unit 1 / 2, reached by `RST 8` — for software built for RomWBW rather than for a bare UART	— (no port)

On a RomWBW system the second SIO/2 channel (`rc2014_1b`, `0x82/0x83`) is the usual `AUX:` port. Match the profile to whatever your comms software was built for. The `aux` choice is the hardware-independent path for SC126/RomWBW software (the SC126's Z180 ASCI is reached with Z180 `IN0`/`OUT0` instructions our Z80 core doesn't implement, so a direct ASCI *port* profile can't work — use `aux` instead). Software that asks *RomWBW* to move the byte rather than touching a port at all needs neither: pick `hbios_1` or `hbios_2` and see "RomWBW HBIOS" below.

> **The virtual modem is polled, not interrupt-driven**
>
> The emulated UART is **polled-only**: the guest reads the status register to see when a byte is ready or the transmitter is free. The emulator never raises a serial interrupt, in any Z80 interrupt mode. This is true of *every* profile above — it is not a property of one chip. Comms software that polls the UART (the norm for CP/M terminal programs, and the standard RC2014/RomWBW and Altair/AltairDuino console paths) works on any profile; software that installs a serial *interrupt* handler and waits for the UART to interrupt will not receive one and will stall. Choose a profile to match the I/O port address and status-bit convention your software expects — not for interrupt support, which none of them provide. (The choice of chip family here — Z80 SIO, 6850 ACIA, 8080 SIO — only affects the port addresses and how the status bits are laid out.)

RomWBW HBIOS (the `hbios_1` / `hbios_2` profiles)

Not all CP/M comms software drives a UART. Software built for **RomWBW** asks the firmware to move the byte instead: it issues an `RST 8` with a function number in `B` and a serial unit number in `C`, and never reads an I/O port. On a port profile such a program hangs before it prints anything — its very first call goes nowhere — which no choice of port address can fix. The QTERM `h` builds (`QTERMH1.COM`, `QTERMH2.COM`) are exactly this kind of program.

Selecting `hbios_1` or `hbios_2` makes the emulator answer that API for one unit — the virtual modem — so those builds run here as they do on real RomWBW hardware. The `RST 8` vector is installed *only* for these profiles, so on any other setting the guest sees the untouched page

zero of a plain CP/M 2.2 machine and a program probing for RomWBW isn't told a system it can't have is present. Pick the profile whose unit number matches your build (unit 1 for `qtermh1`, unit 2 for `qtermh2`); a call for any other unit is refused, so a mismatch fails the same recognisable way a wrong port address does.

> **What this is — and what it is not**
> Implemented: the **serial (character device) group** — character in, character out, input and output status, device initialise, query and describe — plus the version and serial-unit-count calls a program uses to check what it is talking to. **Not** implemented: bank switching and memory management, disk, real-time clock, video, sound, and the DSKY. Those are RomWBW *hardware* services with no counterpart in this emulator, and every function outside the supported set returns a failure result, so a program that needs one finds out at the call instead of misbehaving later. This is an emulation of one API group so that RomWBW-targeted comms software can reach the virtual modem — the emulator is still CP/M 2.2, not a RomWBW system. It was written from the published HBIOS interface description; no RomWBW code is included.

Tested comms software & the profile to pair with it

The CP/M programs below have been run inside this emulator against the virtual modem. Pick the `cpm_emu_uart` profile in the same row — a program that pokes a hard-coded I/O port only answers at that port's address.

Program	Profile	Result
`qterm82.com` QTERM V4.3f, “Version for RC2014 SIO/2 - VT100”	`rc2014_1b` (`0x82/0x83`)	**Works.** `AT` → `OK`, `ATDT host:port` → `CONNECT`, transfers run over the connection.
`qterm84.com` same build, ported to the second SIO/2 board	`rc2014_2a` (`0x84/0x85`)	**Works**, identically. This is the same binary as `qterm82.com` with different port operands.
`imp8.com` IMP v2.45 (Irvin Hoff), Altair 8800 build — “HAYES compatible external modem on 2SIO, port 012H”	`altair_2sio2` (`0x12/0x13`)	**Works.** `AT` → `OK`, `ATDT host:port` → `CONNECT`. Reach the modem with `T` (terminal mode) at its `COMMAND:` prompt. See the type-ahead note below.
`QTERMH1.COM` QTERM V4.3f, “Version for RomWBW HBIOS - VT100” (HBIOS serial unit 1)	`hbios_1`	**Works.** `AT` → `OK`, `ATDT host:port` → `CONNECT`, and a Kermit transfer runs over the connection. Needs an HBIOS profile: on any port profile or `aux` it hangs before its banner.
`QTERMH2.COM` same build, HBIOS serial unit 2	`hbios_2`	**Works**, identically. Unit gating is real: `QTERMH1` under `hbios_2` gets no response at all.
`KERCPM22.COM` Kermit-80 v4.11, “Generic CP/M-80” overlay	—	**Cannot work, on any profile.** The generic overlay has no serial driver — every one of its `SET PORT` choices (CRT/PTR/TTY/UC1/UR1/UR2) still talks to the *console*, so the modem never sees a byte. Use a machine-specific Kermit-80 overlay, or QTERM.

> **Symptom of the wrong profile: complete silence**
>
> A port-poking program pointed at the wrong address gets *no* response at all — not even a local echo of what you type, because the echo comes from the virtual modem. `qterm82.com` under `rc2014_2a` behaves exactly this way. So if `AT` produces nothing, suspect the profile before suspecting the dial string. The profile only matters for programs doing real port `IN`/`OUT`: software that reaches the modem through BDOS funcs 3/4 or BIOS `PUNCH`/`READER` works with *any* profile enabled, so `aux` and `rc2014_1b` are equivalent for it.

EGT8080 and EGT80 — the gateway’s own CP/M terminal

Every terminal in the table above was built for one machine’s serial port, which is why choosing the profile is guesswork until it works. **`EGT8080.COM`** asks instead: it presents a menu, you pick the port, and it remembers. It is written in period assembly for real CP/M hardware as much as for this emulator, runs on CP/M 2.2 and CP/M 3, and assembles with period tooling (SLR Z80ASM, M80, ZMAC) so it can be rebuilt on the target machine.

It ships in two builds, and which to run is a one-line answer. `EGT8080.COM` is written to the 8080’s instruction set, which the Z80’s is a strict superset of, so it runs on every machine here and under either `cpm_cpu` setting — **reach for that one**. `EGT80.COM` is the Z80 build,

and it exists for a family of ports the other cannot have: a **Z180** board such as an SC126 drives its console from the ASCI channels *inside* the processor, reached with `IN0`/`OUT0` and found with `MLT BC` — all `ED`-prefixed instructions, and on a true 8080 an `ED` byte is an undocumented `CALL`, so such a probe would not fail, it would jump into the weeds. Those bytes cannot be in a binary that must also run on an 8080. Both are placed for you; run EGT80 only on a Z80 or Z180, and never under `cpm_cpu = 8080`, where it stops at its first Z80-only opcode. The emulator's sign-on says so on that setting.

They are already there. Both builds are carried inside the gateway binary and placed on CP/M drive A: the first time the emulator creates its drive folders — and in the *transfer directory* too, so the File Transfer menus can see them without your having to start the emulator to get at a file whose whole purpose is to give you a terminal at the far end. So `DIR` shows them and typing `EGT8080` runs it with nothing to install. A copy on A: is *never* overwritten afterwards, because each saves its settings inside its own file — refreshing it would throw away the port you chose. Delete it and the next launch puts the shipped copy back, which is the way to return to a known state. (An upgrade from an earlier release keeps *your* `EGT80.COM` with your settings in it, for the same reason.) Each release also ships both as loose files in the archive, which are the copies to send to a real CP/M machine over XMODEM (from QTERM use `xk`; never Kermit, which is text-only there and would truncate it).

It drives **five port families** — Z80 SIO/2, 6850 ACIA, Altair 88-SIO, RomWBW HBIOS (`RST 8`) and the CP/M `AUX:` device — plus, in the EGT80 build only, **Z180 ASCI**; each is picked from a menu, with a free-form address entry for boards at unusual ports, so it pairs with every profile in the table above. (The 6850 ACIA is in *both* builds: if your board has an ACIA at a port address, EGT8080 reaches it. Only the ports inside a Z180 need the other build. On EGT8080 the ASCI entry is still listed and answers "This processor is not a Z180, so it has no ASCI ports", so you find out from the menu rather than from silence.) Everything configurable lives under **Settings**: the serial port, the line settings (below), ANSI or ASCII output, local echo, LF-after-CR, the menu key (any control key you press), and how to clear the screen. All of it is saved into `EGT8080.COM` itself, so a copy carries the settings to another disk. It also moves files: **XMODEM** in both directions, from its menu or from the terminal-mode menu key (which doesn't hang up the call), tested against this gateway's own File Transfer menu. See `EGT8080/README.md`.

Line settings — and whether a baud rate means anything here. Settings → `B` sets speed and framing, but what that can actually do depends on the family, and the screen says which case you are in rather than offering a knob that goes nowhere. **RomWBW HBIOS** takes speed and framing together in one firmware call; **Z180 ASCI** really does hold the rate in its own registers; a **6850 ACIA** can set framing and divide the board's clock by 1, 16 or 64. A **Z80 SIO/2 cannot set a rate at all** — the chip has no baud generator, the bit rate arrives from the board's clock or CTC, and its registers are write-only, so EGT8080 declines rather than reprogram from a guess. The CP/M `AUX:` device belongs to the operating system.

Against *this gateway* none of it matters in either direction: the virtual modem is a TCP connection, which has no bit rate, so the emulated UART accepts line-configuration writes and ignores them — nothing set here can break the link. Accordingly EGT8080 **programs nothing at all** unless you press `A` to apply, and `R` returns to that state: the port keeps

whatever the ROM, the firmware or CP/M set up, which is the arrangement that already works on the machine. An applied setting is re-applied whenever the port is selected, including at startup, so it behaves as a setting rather than a one-off command. A combination the hardware does not have — 7 data bits without parity on a 6850, say — is refused by name instead of being rounded to the nearest, because a framing mismatch shows up as garbage characters, which is the symptom that sent you to this screen.

The screen is cleared and the banner redrawn before the main menu and on entering terminal mode, which also names the menu key and says that the key followed by `E` returns to the main menu. CP/M has no standard clear-screen — the terminal is not the computer's and the BIOS offers nothing — so Settings → `C` picks the dialect: the ANSI `ESC [ 2 J` (**the default** — what a terminal emulator over USB serial and the gateway's own ANSI clients understand), the ADM-3A's `^Z` for a period terminal or a PETSCII C64 through the gateway, whose translation re-renders it for the client, or off for a printing terminal.

Unlike the period terminals, a wrong choice is *diagnosed* rather than silently hung: a port that never accepts a byte is named and explained, and HBIOS is refused outright on a machine whose `RST 8` vector is absent (i.e. one not running RomWBW) instead of jumping into unused memory. Two families carry caveats the program states in its own help: Z180 ASCI cannot be exercised in this emulator (our Z80 core doesn't implement the Z180's `IN0`/`OUT0`), and CP/M 2.2 offers no way to poll the `AUX:` device, so that family reads a byte ahead and cannot receive a literal `^Z`.

> **Type at human speed — don't paste into a polling terminal program**
> The emulator gives the guest a deep console type-ahead buffer, where a real CP/M machine's console UART holds a single byte. A comms program whose terminal loop re-checks console status *before* forwarding the key it already read will overwrite that key when a second one is already waiting — so a pasted or scripted burst loses characters, while ordinary typing (where the keys arrive milliseconds apart) is fine. `imp8.com` behaves exactly this way: typed `AT` returns `OK`, pasted `AT` reaches the modem as `T` and returns `ERROR`. QTERM does not have this loop and takes bursts intact. This affects only keystrokes you paste — file transfers, which read from disk, are unaffected.

> **From QTERM, transfer with XMODEM — not Kermit**
> QTERM's Kermit is **text-only** and there is no setting to make it binary: it stops at the first `^Z` (0x1A) in the file and still reports "Transfer complete", so a binary upload is silently truncated. Use `xk` (XMODEM-1K), `x`, or `xy` for `.COM` files and other binaries. Note also that the QTERM builds above use **Ctrl-Y**, not the stock Ctrl-\, as the escape to command mode.

Dialling out

With a modem port selected, the CP/M program talks to it with ordinary Hayes `AT` commands. It can **dial a serial port** the same way Ports A/B dial each other: `ATD A` / `ATD B` reach this gateway's own ports, and `ATD A@<ip>` / `B@<ip>` reach a port on another gateway across the master/slave relay (same routing and `allow_peer_dial` gate as the physical modem). Or dial a network host directly with `ATDT host:port` — a target with no `:port` defaults to port 23, and a phone number is looked up in `dialup.conf`, both as on the physical

modem. On answer the modem reports `CONNECT` and becomes a transparent data pipe; `+++` returns to command mode and `ATH` hangs up.

It can also dial **this gateway's own menu**: `ATDT ethernetgateway` (or `ethernet-gateway`, or the built-in number `1001000`) — the same targets the physical modem answers. That is the quickest way to prove a CP/M terminal works, since it needs nothing outside the gateway: from EGT8080, pick the port, press `T` for terminal mode, and dial. The menu session runs with raw serial semantics, so no telnet negotiation bytes reach the CP/M program.

Settings persist, as they do on the physical ports. `AT&W` writes the current profile — echo, verbose, quiet, result-code level, DCD mode and all 28 S-registers — to the `cpm_emu_*` keys in `egateway.conf`, and the modem powers up with it every time you enter the emulator, so a comms program's init string survives leaving CP/M and restarting the gateway. `ATZ` restores that saved profile (the behaviour of a real modem, and of this gateway's serial ports); `AT&F` is the command that ignores it and returns to factory defaults. The profile is also editable in the web and GUI config editors, next to the other CP/M settings, exactly as each serial port's own `AT&W` block is — so a profile can be inspected or repaired without booting CP/M.

The command layer understands a chained Hayes init string (`ATE0Q0V1X4S0=1` applies each clause): `E` echo, `Q` quiet, `V` verbose vs numeric result codes, `X` result-code level, `S`*n*`=`/`?` S-register set/query (`S0` auto-answer, `S7` peer-dial carrier wait), `&C`/`&D` DCD/DTR, `ATZ` / `AT&F` reset, and `ATI` identify. Carrier is surfaced to the guest as the UART's DCD bit, and flow control is honoured both ways (the UART reports transmit-not-ready and the peer is back-pressured rather than dropping bytes when either side outruns the other).

Inbound calls (dial the CP/M machine)

When a modem port is enabled, the CP/M emulator is itself **dialable as a third serial port named `CPM`**. From another modem on this gateway (or the Serial Gateway), `ATD CPM@<ip>` rings it just like `A@<ip>` / `B@<ip>` ring Ports A/B. The running CP/M comms program sees `RING` and answers with `ATA` (or automatically after `ATS0=`*n* rings); the two machines are then joined transparently. Peer-dial must be enabled (`allow_peer_dial`).

Reachability follows the same master/slave rules as Ports A/B. CP/M running on the **master** is reachable from every attached machine: a device on a slave dials `CPM@<master-ip>` and the call is relayed to the master, which rings its own endpoint. CP/M running on a **slave** is reachable through the master crossbar the same way A/B ports are: while a modem CP/M shell is active, the slave registers `CPM` with the master, so `CPM@<slave-ip>` dialed on the master (or another slave) rings that slave's endpoint. That registration happens as soon as a modem-enabled CP/M session starts — like the serial ports, it is not gated on `allow_peer_dial` — and it re-forms within about a second if the master's address or credentials change. In short, `CPM@<ip>` works wherever `A@<ip>` / `B@<ip>` do.

Answering requires a running comms program

The emulator picks up an inbound call only while a program is running its modem (that's when it polls for the ring) — run your terminal/BBS software first, exactly as on real hardware. Set `ATS0=`*n* to auto-answer, or answer manually with `ATA` when you see `RING`. `CPM@<ip>` is a single dialable address, but every modem-enabled CP/M session joins an answer *pool* (a hunt group): the next inbound call rings whichever session is idle, so two concurrent CP/M users can both receive calls. On a slave, one session announces `CPM` to the master for remote reachability; if it exits while others remain, remote dialling pauses until a new session re-announces (local answering is unaffected).

9. Serial / Hayes AT Modem Emulation

The modem emulator turns the gateway into a virtual Hayes Smartmodem on a physical serial port. Retro hardware — a Commodore 64 with an RS-232 cartridge, a CP/M machine, an AltairDuino 2SIO2, an RC2014 — connects as if the gateway were a real modem. `ATDT`, `ATH`, `+++`, all of it works.

The gateway exposes **two physically independent serial ports**, **Port A** and **Port B**, each with its own enabled flag, mode, device, baud, AT/S-register state, and stored phone-number slots. The two ports run in separate manager threads and persist under separate `serial_a_*` / `serial_b_*` key blocks, so you can run a Hayes modem on one wire and a telnet-serial bridge on the other (or two modems, or two bridges) simultaneously without any cross-talk.

The emulator speaks a full Hayes command set on each port: `AT`, `ATZ`, `AT&F`, `AT&W`, `AT&V`, `ATE`, `ATV`, `ATQ`, `ATI` variants, `ATH`, `ATA`, `ATO`, `ATX`, `AT&C`, `AT&D`, `AT&K`, `AT&Z`, `AT+PETSCII`, `ATD` (with `T`, `P`, `L`, `S` variants), `ATSn`, S-registers S0–S26, the `A/` repeat-last-command shortcut, and the `+++` escape sequence with S2/S12 guard-time semantics. `AT&W` on Port A persists into the `serial_a_*` keys and never touches Port B's state, and vice versa.

9.1 Enabling a Port

Each port enables and configures independently. From the GUI Serial Port frame:

1. Check the *Enabled* box for the port you want to bring up — Port A and Port B each have their own checkbox in the frame's header row.
2. Pick a device from that port's dropdown. If your port does not appear, plug in the adapter and click the refresh (↻) button on Port A's row to re-scan both lists.
3. Set the baud rate inline on the port's row.
4. Click that port's *More…* button to choose *Mode* (Modem, Telnet-Serial, or Kermit Server), data bits, parity, stop bits, flow control, and the full Hayes AT state.
5. Click the *Save* button on the frame's header row. Both ports' settings are persisted atomically and each manager reconfigures with the new values.

You can also configure everything live from an in-session telnet / SSH user: *Configuration → M Serial Configuration → A* or *B*. That per-port menu has [E] to enable, [T] to toggle this port between modem and console mode, [S] to detect and select a device, and submenus for baud / framing / flow control. Changes applied through the in-session menu take effect immediately on the chosen port without affecting the other port.

> **Serial lockout protection**
> When you change parameters from a serial session (i.e. through the modem itself), the server asks for confirmation and *reverts* if you don't respond within 60 seconds. If your new baud rate is wrong, your serial terminal will stop echoing and you will simply wait out the timer — your old settings come back unchanged.

Typical C64-friendly parameters: 2400–9600 baud, 8N1, no flow control. CP/M systems usually want 9600 8N1. The AltairDuino 2SIO2 is commonly configured for 9600 8N1 as well; remember to press *stop* and *aux1 up* on the front panel to configure its serial ports.

Commodore 64 with a bit-banged userport serial adapter

If your C64 reaches the gateway through a **bit-banged userport RS-232 adapter** — such as the **EZ232**, which is a passive level-shifter with no UART of its own — use **1200 baud**, not 2400. On these adapters the C64 times every serial bit in software, and it has almost no timing margin to spare. When a long command wraps past column 40 the screen scrolls, the KERNAL briefly masks interrupts to shuffle screen memory, and any keystroke being transmitted during that scroll has its bit timing disrupted. The result is an intermittent, single-character corruption (a typed letter arriving as a stray PETSCII graphics byte) that runs a garbled command. This has been observed with **CCGMS** in PETSCII mode. Dropping both the gateway port (`serial_a_baud = 1200`) and the C64 terminal program to 1200 baud roughly doubles the per-bit timing budget and clears it up. A hardware-UART interface (SwiftLink / Turbo232) or a WiFi modem with its own UART is not affected and can run faster. See §16.9.

Device names can move — port settings don't

A port's *settings* (mode, baud, framing, AT/S-register state) are keyed to **Port A** / **Port B** and persist across restarts. The *device path* (`serial_a_port` / `serial_b_port`, e.g. `/dev/ttyUSB0`) is just a saved string — and on Linux the kernel hands out `ttyUSB0`, `ttyUSB1`, … in the order adapters enumerate. After a reboot, or if you unplug the adapters and plug them back in a different order, those names can **swap**, so Port A may suddenly point at the adapter you meant for Port B (with Port A's baud/mode applied to the wrong device). To pin a device regardless of enumeration order, set the port to a stable by-id path instead of `ttyUSBn` — e.g. `serial_a_port = /dev/serial/by-id/usb-FTDI_FT232R_USB_UART_A1234-if00-port0` (list them with `ls -l /dev/serial/by-id/`). After any re-cabling, confirm each port's device in the GUI dropdown or the telnet *Configuration → M* screen.

9.2 Dialing

The emulator routes dial strings over TCP. There is no real phone line — the "call" is a `tokio::net::TcpStream::connect()`.

Command	Action
`ATDT host:port`	Connect to a remote telnet server. `ATDT` and `ATDP` are equivalent (there is no audible difference over TCP).
`ATDT ethernetgateway`	Built-in shortcut — dials this gateway's own main menu, so the serial device can use all gateway features (file transfer, browser, AI chat, etc.) without a network detour.
`ATDT KERMIT`	Drop directly into Kermit server mode — no menu, no banner, indistinguishable on the wire from a real `kermit -j host` in `server` mode. Aliases: `ATDT kermit`, `ATDT kermit-server`, `ATDT kermit server`. Requires `allow_atdt_kermit = true`; off by default because it bypasses the telnet menu's auth gate. Idle timeout still honors `kermit_idle_timeout`; on session end the modem emits `NO CARRIER`.
`ATDP host:port`	Same as `ATDT` — pulse selector, ignored.
`ATDSn` (n = 0–3)	Dial stored number in slot *n*. Bare `ATDS` dials slot 0.
`ATDL`	Redial the last number.
`AT&Zn=s`	Store phone number or `host:port` in slot *n* (0–3). Preserves hostname case.

Dial-string modifiers (only honoured when the dial string looks like a phone number, i.e. digits and formatting characters):

Modifier	Effect
`,`	Pause for S8 seconds (default 2) before continuing. Each comma adds S8 seconds.
`W`	Wait for dial tone. Adds S6 seconds to the pre-delay.
`;`	After "connect", return to command mode instead of entering online data mode. Applies to hostnames too.
`*`, `#`	DTMF digits. Preserved in the dial target for phone-number lookup.
`P`, `T`, `@`, `!`	Pulse, tone, quiet-answer, hookflash selectors. Accepted, ignored.

Modifiers are *not* stripped from hostnames. Dialling `ATDT pine.example.com` connects to "pine.example.com" verbatim — the heuristic notices that the dial string contains letters and dots and skips modifier processing for everything except the trailing `;`.

9.2.1 Examples

```
ATDT ethernetgateway
CONNECT 9600
(local gateway main menu appears)

ATDT 192.168.1.160:2323
CONNECT 9600
(remote gateway appears)

ATDT bbs.example.com:23
CONNECT 9600

AT&Z0=bbs.example.com:23
OK
ATDS0
CONNECT 9600

AT&Z1=Pine.Example.com:22
OK
(note: hostname case is preserved)

ATDT 5551234
(looked up in dialup.conf, or NO CARRIER if unmapped)
```

9.2.2 Dial-up Mapping

The gateway also maintains `dialup.conf`, which maps phone numbers to `host:port` targets. The mapping table is **shared by every modem in the gateway** — one file, consulted by Port A, by Port B, and by the CP/M emulator's virtual modem — so a number you add once is dialable from any of them, including from a CP/M program running inside the emulator. Edit it through either port's Modem Emulator menu (*Configuration → M → A or B → D*); there is no separate CP/M copy to keep in step. A built-in entry, `1001000`, always maps to the local gateway menu (equivalent to `ATDT ethernetgateway`) and cannot be deleted — that one works from the emulator too. Numbers are matched by digits only — dashes, spaces, and parentheses are ignored — so `555-1234` and `5551234` hit the same entry.

> **Shared: the phone book. Not shared: the stored-number slots**
>
> `dialup.conf` is one table for the whole gateway, but the four `AT&Z`*n*`=`*s* stored-number slots are *per port* (`serial_a_stored_0` … `serial_b_stored_3`), so Port A's slot 0 and Port B's are different numbers. The CP/M emulator's modem has no stored slots at all — `AT&Z` and `ATDS`*n* are not implemented there; use the shared phone book, which reaches the same targets by number.

9.2.3 Calling Another Port (Peer-Dial)

Peer-dial lets a modem port **call another serial port directly** and talk to the device attached to it — the gateway equivalent of dialing a friend's modem to swap files. Where `ATDT ethernetgateway` lands on the gateway *menu*, peer-dial connects straight through to a specific

port. It is **off by default**; enable `allow_peer_dial` from *Configuration → M Serial Configuration →* `P`, the web *Serial* config, or the GUI.

Dial another port by its **address**, `<Port>@<IP>`:

```
ATD B@192.168.1.50        (call Port B on the gateway at 192.168.1.50)
```

The address is exactly what the **Serial Gateway** menu shows (*Dial: <Port>@<ip>*), so you dial what you see. You can also just pick the target port in that menu — it has the same effect as dialing its address. What happens next depends on the **target port's mode**:

- **Modem-mode target — it rings.** The device on that port answers per its *own* AT rules: automatically after `S0` rings (S0 = 0 disables auto-answer), or when its operator types `ATA`. This is a true dial-up call.
- **Console-mode (telnet-serial) target — it connects directly.** A console port has no modem to answer, so the bridge opens immediately (leased-line style).

Once connected it is a **transparent byte pipe** between the two devices — run a file-transfer protocol (XMODEM / YMODEM / ZMODEM / Kermit / Punter) end to end between them, exactly as you would over a real modem-to-modem call. Each port keeps its own baud rate; they need not match. (Turn PETSCII translation off — `AT+PETSCII=0` — before a binary transfer.)

Caller sees	When
`CONNECT`	Target answered (modem) or connected (console).
`BUSY`	Target modem port is already in a call.
`NO ANSWER`	Target rang but nobody answered within the caller's `S7` (needs `ATX3+`; else `NO CARRIER`).
`NO CARRIER`	Unknown/disabled port, peer-dial off, dialing your own port, or the call failed.

> **Local echo**
> The connection is a transparent link — neither gateway echoes the data, and (unlike dialing a host or BBS) there is no remote host echoing your keystrokes back. **Turn on local echo (half-duplex) on each terminal** to see what you type, exactly as with two terminals wired back to back. `ATE` does *not* help here: it only echoes `AT` commands in command mode, not the online data stream.

> **Ring count vs. answer timeout**
> Auto-answer waits `S0` rings at the ~6 s ring cadence, so the gateway default `S0 = 5` takes ~30 s — longer than the caller's default `S7 = 15` wait, which would give `NO ANSWER` first (authentic real-modem behaviour). On a port you intend to be dialed, set a low `S0` (e.g. `ATS0=1`) or have the operator answer with `ATA`.

Cross-gateway (over the master/slave relay): a device on a *slave* can dial a port on its *master* — `ATD <Port>@<master-ip>`. The slave relays the call; the master resolves the address

to one of its own ports and rings/connects it, bridging `device ↔ slave ↔ master ↔ port`. Requires the slave's `allow_peer_dial` on (to relay) and the master's `master_accept_relays` + `allow_peer_dial` on (to accept). Addressing is by IP, so master and slave need distinct addresses (normal for separate machines). Cross-gateway is symmetric: `<Port>@<slave-ip>` reaches **any** port a slave has — a **console** port connects directly, a **modem** port rings the attached device — from the master or, with the master as a crossbar, from another slave (`device ↔ slave-A ↔ master ↔ slave-B ↔ device`). A slave's ports announce themselves to the master automatically whenever `gateway_role = slave` and a master host is set — this is *not* gated on `allow_peer_dial`, because a slave its own master cannot reach defeats the purpose of the mode. The gate above still applies to the *dialing*: relaying a call out through the master needs the slave's `allow_peer_dial`, and the master accepting a third party's crossbar dial needs the master's.

9.3 Identification (ATI)

Command	Response
`ATI` / `ATI0`	Product ID and version: `Hayes-compatible Ethernet Gateway Modem Emulator v0.9.3`
`ATI1`	ROM checksum: `000` (virtual)
`ATI2`	ROM self-test: `OK`
`ATI3`	Firmware identifier: `Ethernet Gateway v0.9.3 (Hayes-compatible)`
`ATI4`	Product description: `Hayes-compatible virtual modem over TCP`
`ATI5`	OEM / country code: `B00`
`ATI6`	Link diagnostics: `No link diagnostics available`
`ATI7`	Product info: `Product: ethernetgateway (software emulator)`

9.4 Reset & Profile Storage

Command	Action
`ATZ`	Reset to the stored profile (read from `egateway.conf`). Drops any active connection.
`AT&F`	Factory reset to gateway-friendly defaults. Does *not* overwrite the stored profile, but does drop any active connection (same as `ATZ` in that respect).
`AT&W`	Save current settings to `egateway.conf`. Persists echo, verbose, quiet, all 27 S-registers, `X`, `&C`, `&D`, `&K`, the `+PETSCII` toggle, and the four stored-number slots.
`AT&V`	Display current configuration — one line of mode flags, a second line listing every S-register, and a third line listing stored numbers.

9.5 Echo, Verbose, Quiet

Command	Effect
`ATE0` / `ATE1`	Echo off / on (default on).
`ATV0` / `ATV1`	Numeric / verbose result codes (default verbose).
`ATQ0` / `ATQ1`	Result codes on / silent mode (default on).

In verbose mode results are text (`OK`, `CONNECT`, `NO CARRIER`, `ERROR`, `RING`, `BUSY`, `NO DIALTONE`, `NO ANSWER`). In numeric mode they are digits. Quiet mode suppresses results altogether.

9.5.1 ATX Levels

Level	Extended codes	CONNECT format
`X0`	Basic only; BUSY / NO DIALTONE / NO ANSWER collapse to NO CARRIER	`CONNECT` (code 1)
`X1`	Basic + baud in CONNECT	`CONNECT <baud>` (code per baud)
`X2`	Adds NO DIALTONE detection	`CONNECT <baud>`
`X3`	Adds BUSY detection	`CONNECT <baud>`
`X4` (default)	Full extended set	`CONNECT <baud>`

Numeric CONNECT codes follow Hayes conventions: 1 = 300 baud, 5 = 1200, 10 = 2400, 12 = 9600, 16 = 19200, 28 = 38400, 87 = 115200. Non-standard baud rates report code 1.

9.6 DTR, Flow Control, and DCD

Command	Meaning
`AT&C0`	DCD always asserted.
`AT&C1` (Hayes default)	DCD reflects carrier state.
`AT&D0` (*gateway default*)	Ignore DTR entirely.
`AT&D1`	Drop DTR → return to command mode.
`AT&D2` (Hayes default)	Drop DTR → hang up.
`AT&D3`	Drop DTR → hang up and reset.
`AT&K0` (*gateway default*)	No modem-layer flow control.
`AT&K1`	Auto-detect (Hayes convention). Accepted and stored; like every `&K` mode, does not drive the wire (see warning below).
`AT&K3`	Bidirectional RTS/CTS hardware flow control.
`AT&K4`	Bidirectional XON/XOFF software flow control.
`AT+PETSCII=0` (*gateway default*)	PETSCII translation off — direct-TCP dials (`ATDT host:port`) are raw ASCII passthrough.
`AT+PETSCII=1`	PETSCII translation on — vendor extension for C64/PET callers. The emulator case-swaps letters in both directions and maps ASCII BS ↔ PETSCII DEL so an ASCII BBS like `telehack.com` reads correctly. ANSI `ESC [ …` cursor/color sequences are stripped on the inbound side so they don't render as garbage. Inbound punctuation that renders wrong on a C64 in lower/upper mode is normalized to plain ASCII: back-tick (the old “left single quote”, which shows as a thick underscore) and UTF-8 “smart” single quotes → `'`; smart double quotes → `"`; tilde and en/em dashes → `-`; the ellipsis → `...`; other high bytes the C64 can't display are dropped or shown as `?`. While editing the AT command line under `+PETSCII=1`, the C64 `INST/DEL` key (PETSCII DEL, 0x14) works as backspace and echoes correctly. Applies only to direct-TCP dials; `ATDT ethernetgateway` already detects PETSCII terminals via the telnet menu and renders accordingly. **Text only — turn it off (`AT+PETSCII=0`) before running a file transfer over a direct-TCP dial: it rewrites bytes in both directions and will corrupt XMODEM/YMODEM/ZMODEM/Kermit/Punter binary data.** Persists immediately (and is re-saved by `AT&W`); also editable from the telnet, web, and GUI config surfaces.

Gateway-friendly default deviations

The emulator deviates from three Hayes defaults because retro clients rarely drive the relevant signals correctly:

- `AT&D0` instead of `&D2` — many C64, CP/M, and similar clients don't wire DTR. `&D2` would cause spurious disconnects.
- `AT&K0` instead of `&K3` — retro clients rarely implement hardware flow control. Physical-wire flow is still controlled by the `serial_x_flowcontrol` setting on each port.
- `S7=15` instead of `S7=50` — keeps failed TCP dials responsive. Internally capped at 60 seconds.

All three can be overridden interactively (e.g. `AT&D2`, `AT&K3`, `ATS7=50`) and persisted with `AT&W`.

Signal pins are mostly not driven

`AT&C`, `AT&D`, and `AT&K` are parsed, stored, persisted, and shown by `AT&V` — but the emulator drives no RS-232 hardware pins by default. RI, DSR, and the read direction of DTR are not controlled, and RTS/CTS handshaking on the wire is governed by each port's `serial_x_flowcontrol` setting rather than `&K`. The one exception is carrier: enable the per-port `serial_x_drive_carrier` opt-in (default off) and the emulator drives **DTR** as a carrier proxy following `AT&C` — assert on `CONNECT`, drop on `NO CARRIER` under `&C1` (or forced on under `&C0`) — which a terminal wired DTR→DCD sees as carrier detect. Most retro terminal software works fine without these signals; if your program requires DCD before communicating, enable that opt-in or look for a "force DTR" / "ignore carrier" option.

Note that `serial_x_drive_carrier` is a config-file setting, not modem state: because it reflects physical cabling it is **not** reset by `ATZ`/`AT&F`, **not** saved by `AT&W`, and **not** shown in `AT&V` (unlike `AT&C`). Change it in the GUI/web config or the telnet per-port modem menu (`C`) — the `AT&C` mode it follows is the normal, ATZ-resettable modem setting.

AT+PETSCII is a vendor extension

`AT+PETSCII=0/1` is not part of the original Hayes command set — it's a gateway-specific addition for PETSCII translation on direct-TCP dials. It lives in the ITU-T V.250 `+` extended-command namespace, the sanctioned home for vendor commands. It deliberately does *not* use `AT&P`: on real Hayes-compatible modems `AT&Pn` sets the pulse-dial make/break ratio, so reusing it would clash with that standard meaning. The command is set-only (`=0`/`=1`); `AT+PETSCII=1` persists the setting immediately, `ATZ` reloads the saved value, `AT&F` resets to off, and `AT&V` reports the state as `+PETSCII:n`.

9.7 S-Registers

Query with `ATSn?`, set with `ATSn=v`. Values are 0–255. `ATS?` prints a help table. `AT&W` saves all 27 values to `egateway.conf`; `ATZ` restores them from the stored profile; `AT&F` resets to factory defaults.

Register	Default	Description
S0	5	Auto-answer ring count (0 disables).
S1	0	Current ring counter.
S2	43	Escape character (+). Values > 127 disable escape detection.
S3	13	Carriage return character.
S4	10	Line feed character.
S5	8	Backspace character.
S6	2	Wait-for-dial-tone seconds (used by `W` modifier).
S7	**15**	Wait-for-carrier seconds. Hayes default is 50; gateway reduces it for responsive TCP dials. Capped internally at 60.
S8	2	Comma pause time (seconds).
S9	6	Carrier detect response time (1/10 s).
S10	14	Carrier loss disconnect time (1/10 s).
S11	95	DTMF tone duration (ms).
S12	50	Escape-sequence guard time (1/50 s). 50 = 1.0 s. Set to 0 to disable escape detection.
S13 – S24	0	Reserved placeholders. Stored and persisted so legacy init strings don't halt with `ERROR`, but they have no effect.
S25	5	DTR detect time (1/100 s). Reserved — no DTR pin.
S26	1	RTS-to-CTS delay (1/100 s). Reserved — no RTS/CTS pins.

Do not collide S3/S4/S5

Command-mode line editing dispatches on the raw byte and checks the CR branch before the BS branch. Setting `S3 = 8` would make backspace terminate the line. Keep S3/S4/S5 distinct (the Hayes defaults 13/10/8 do the right thing).

9.8 The +++ Escape Sequence

From online (data-transfer) mode, type `+++` surrounded by at least **S12 × 1/50** seconds of silence on either side to return to command mode without hanging up. Once in command mode, `ATO` resumes the connection and `ATH` hangs up. This follows the Hayes guard-time spec exactly.

The implementation details:

- Escape character is `S2` (default 43 = `+`).
- Guard time is `S12 / 50` seconds on both sides of the triple.
- Setting `S12 = 0` or `S2 > 127` disables escape detection entirely.
- Fewer or more than three consecutive escape characters are passed through as data.
- The escape is interrupted if the sender emits any non-escape byte mid-sequence.

Escape won't trigger? (bit-banged serial)
On a C64 with a software/bit-banged interface (EZ232 and similar) the wire can pick up a stray byte while "idle," which silently defeats the escape: a phantom byte just before your first `+` resets the leading guard timer, and one within a second of the third `+` flushes `+++` to the host and cancels the escape. To see exactly what happens, set `EGATEWAY_GATEWAY_DEBUG=1` (or toggle Gateway Debug Trace) and try again — the log shows `[esc]` lines for each escape character accepted, the silence measured before the first one, and the offending byte if a sequence is broken. Lowering `S12` (e.g. `ATS12=25` for a half-second guard) tightens the window that a stray byte can spoil.

9.9 A/ — Repeat Last Command

`A/` (capital A, forward slash) with no `AT` prefix and no carriage return repeats the last command you entered. Real Hayes modems recognise this as a special two-character command. Useful for re-dialling or re-sending the last `ATZ` while debugging.

9.10 Chained Command Lines

Multiple AT commands can be combined on a single `AT` line — the same way real Hayes modems have always accepted them in init strings. The emulator parses the line left-to-right, applies each subcommand, and emits exactly one `OK` at the end (or `ERROR` if any subcommand failed). Spaces between subcommands are tolerated.

```
ATE0Q1V1            ; echo off, quiet, verbose, single OK
ATE0 Q1 V1          ; same — spaces between tokens are fine
ATIE0               ; print version, then echo off
ATE0DT bbs.example.com:23   ; settings then dial (dial terminates)
AT&FE0           ; factory reset, then echo off — order matters
ATS0=2H             ; set S0=2 then hang up
```

The chain stops at the first *terminator* — anything that ends command mode or has a value that runs to end-of-line:

- `D…` — any dial command (`ATD`, `ATDT`, `ATDP`, `ATDL`, `ATDS`*n*) consumes the rest of the line and goes online.
- `A` — answer; the modem leaves command mode after this.
- `O` / `O0` — return to online mode.
- `&Z`*n*`=…` — store-number value runs to end of line (it can contain any character — phone numbers, hostnames, ports).

The chain also stops at the first **error**. Commands before the failure run to completion (their state changes apply); commands after do not run. A line like `ATS0=999E0` (S0=999 is out of range) returns `ERROR` and leaves the echo setting untouched.

S-register set values stop at the first non-digit, so chaining works naturally: `ATS0=2E0` sets S0 to 2 and then echo off, even though there is no separator between `2` and `E`. Unknown

subcommands (`ATL`, `ATM`, `ATB`, etc.) do not break the chain — they are accepted silently as if they ran to no effect.

9.11 Ring Emulator

Telnet and SSH users can simulate an incoming phone call to a serial device from the per-port Modem Emulator menu (`I` — Ring emulator). The Ring item is per-port: pick *Configuration → M (Serial Configuration) → A or B* first to choose which port should ring, then press `I`. The modem sends `RING` to that port at standard US cadence (every 6 seconds). After `S0` rings (default 5) the modem auto-answers and the serial device receives the Ethernet Gateway main menu, just as if it had dialled in with `ATDT ethernetgateway`. The serial device can also answer manually with `ATA` during ringing. The two ports' ring slots are independent — Port A and Port B can ring simultaneously.

9.12 Unknown AT Commands

Commands the emulator has no hardware to implement — `ATB`, `ATC`, `ATL` (speaker loudness), `ATM` (speaker mute), `AT&B`, `AT&G`, `AT&J`, `AT&S`, `AT&T`, `AT&Y`, etc. — are accepted and return `OK`. This keeps legacy init strings working instead of halting mid-setup with `ERROR`.

9.13 Console Mode (Telnet-Serial Bridge)

Either of the two physical serial ports can be repurposed as a transparent telnet-serial bridge — independently of the other. When a port has `serial_x_mode = console`, its Hayes emulator is disabled and that port idles until a telnet/SSH user picks **G Serial Gateway** from the main menu and chooses this port from the A/B picker. Bytes then flow in both directions between the user's session and the wire, with no AT processing. Both ports can host their own concurrent bridge — a session bridged to Port A doesn't block a separate session from bridging to Port B.

This is the right mode when a serial port is connected to something that already speaks its own protocol — a microcontroller's debug console, a router's serial console, an embedded test fixture — and you want to drive it remotely.

Switching modes (per port)

- **GUI:** open the chosen port's *More…* popup from the Serial Port frame and set its *Mode* dropdown — *Modem (AT Command) Mode*, *Telnet-Serial Mode*, or *Kermit Server Mode*. Saving reconfigures only the changed port; the other port keeps running.
- **Telnet/SSH menu:** from *Configuration → M (Serial Configuration) → A or B*, press `T` (*Mode: Modem/Console/Kermit*) inside the per-port menu to cycle the three modes. The banner rotates between `PORT A - MODEM EMULATOR`, `PORT A - SERIAL CONSOLE`, and `PORT A - KERMIT SERVER` (or B), and the Dialup Mapping / Ring Emulator items hide in the two non-modem modes (they're modem-only features). The `T` entry is hidden from sessions connected over the modem itself, since flipping the mode would tear down the caller's own connection before they could acknowledge — flip the mode from a telnet, SSH, or system-console session instead.
- **Config file:** set `serial_a_mode = console` (or `= kermit`, or `serial_b_mode = …`) in `egateway.conf`. The change is hot-applied within one manager-poll interval — no restart required.

9.13.1 Using the Bridge

1. From the main menu, press `G` (Serial Gateway). An A/B picker appears showing both ports' status; ineligible ports (disabled, modem mode, no device) are dimmed.
2. Press `A` or `B` to pick a port. The gateway prints that port's active framing (port, baud, data/parity/stop, flow) and asks `Connect now? (Y/N):`. Press `Y` to enter the bridge.
3. Type to the wire; the wire types back. The connection is fully 8-bit transparent and the gateway does not interpret AT commands or insert IAC sequences.
4. Press `ESC` twice in a row to leave the bridge (PETSCII `←` twice on Commodore terminals). A single `ESC` is held for one read cycle; if you type any other key after it the held `ESC` goes through to the wire, so editors that need `ESC` (vi, ed, emacs) keep working.

One session per port

The bridge is single-user **per port**. Port A and Port B each accept one bridge at a time. If a second telnet/SSH session tries to open a bridge to a port that's already in use, the second session sees *Another session is already using Port X* and is returned to the main menu. The port is released as soon as the first session leaves its bridge or disconnects. Bridges on the two ports do not interfere with each other.

Self-bridging is blocked

The Serial Gateway item is always shown; the picker is the single authority on what's eligible. A session that arrived over a serial port may bridge to a *different* port but not back to its own arrival port (that would loop your terminal to itself), so the picker marks only that one port ineligible. When nothing is available the picker says so rather than the menu dead-ending.

9.13a Kermit Server Mode (Always-On)

The third per-port mode turns a serial port into a permanent Kermit server on the wire. When a port has `serial_x_mode = kermit`, its Hayes emulator is disabled and — as soon as the port is enabled — it listens for Kermit packets directly on the wire: no AT commands, no dialing, no menu. It is functionally the same server the `ATDT KERMIT` serial dial reaches (§9.2) and the standalone TCP listener serves (§8.8), but permanent instead of dialed.

- Point a Kermit client on the attached hardware at the wire and drive its server commands (`SEND`, `GET`, `DIR`, `FINISH`, `BYE`). Uploaded files land in the configured `transfer_dir` with the same filename / path-safety validation as every other Kermit path. An upload whose name already exists is **renamed**, never dropped or overwritten: the base name is numbered DOS/CP-M-Kermit style, kept within 8 characters (e.g. `abcdefgh.txt` → `abcdefg0.txt` … `abcdefg9.txt` → `abcdef10.txt`; a shorter name like `hi.txt` → `hi0.txt`). A verified resume still replaces its own partial in place.
- The server re-arms after every completed exchange (FINISH / BYE / idle) so the wire stays a live server for as long as the port is open, and it reopens the device automatically if it disappears (e.g. a USB-serial adapter or socat bridge that comes and goes) — the same reopen policy as modem mode.
- Authentication and the telnet menu are bypassed by design, the same posture as `allow_atdt_kermit` and the dedicated Kermit TCP listener. Enable it only on trusted serial lines.
- A Kermit-Server-mode port only ever *serves*: it is not a Serial Gateway bridge target and is not dialable via peer-dial (§9.2.3), so it does not appear as a selectable target in those menus.

Set it exactly like Console mode — the GUI / web *Mode* dropdown, the telnet per-port `T` cycle, or `serial_x_mode = kermit` in `egateway.conf` (hot-applied within one manager-poll interval, no restart).

9.14 Master/Slave Serial Extender

A **slave** gateway can extend its serial ports to a **master** gateway over the master's SSH port, so a serial-attached device reaches the master's menu, file transfer, and dial-out as if it were attached to the master directly. **Files always land on the master.** The feature is entirely inert by default (`gateway_role = standalone`); a standalone gateway behaves exactly as it always has.

9.14.1 Configuring the Master

Enable the SSH server (`ssh_enabled = true`), set `gateway_role = master`, and turn on `master_accept_relays`. Relay connections share the existing SSH port (2222) alongside normal SSH logins — there is no per-slave configuration. Accepting relays is never implied by enabling SSH; if you set master + accept-relays but leave SSH disabled, the server logs a startup warning because relays ride the SSH server.

9.14.2 Configuring the Slave

Set `gateway_role = slave`, point `slave_master_host` / `slave_master_port` at the master (the port defaults to the SSH port, 2222), and enter `slave_master_username` / `slave_master_password` — which **must match the master's unified username/password**. The slave pins the master's SSH host key on first contact (trust-on-first-use, stored in `gateway_hosts`) and refuses a changed key. All of these settings are editable from the telnet **Configuration › S Server › M Master/Slave** sub-screen, the web config *Master/Slave* card, and the GUI Server *More…* popup.

9.14.3 How Ports Relay

- **Modem-mode port** — the slave runs the Hayes emulator locally and **registers the port with its master at startup**, so the link works in both directions. Inbound: the master's **G Serial Gateway** menu lists the port and can *ring* it — the slave rings its local UART, which answers per its own S0 / `ATA` rules, and the master is then bridged to the device. Outbound: when the device dials, the slave bridges the call to the master, which serves its menu or dials a resolved `host:port` onward. The slave's *local* dial map resolves the number and the master performs the dial ("resolve local, dial central"). Onward-dial requires the master's `allow_peer_dial` to be on — otherwise the master refuses the outbound connection. `+++` / `ATO` work normally. Registering with the master is *not* gated on `allow_peer_dial`: that setting governs this gateway *dialing* arbitrary peers, and a slave that its own master could never reach would defeat the purpose of slave mode. (A third party reaching the port through the master's crossbar is still gated, on the master.)
- **Kermit-server-mode port** — the slave pipes the wire to the master and the *master* runs its Kermit server on it, so a user at the device is talking to the master's server: `remote dir` lists the **master's** transfer directory, an upload lands there, and a download is read from there. The slave never serves Kermit itself in this mode and keeps no copy — which is what makes "files always land on the master" true by construction rather than by synchronising two directories. The channel is opened as soon as the link allows (a Kermit server is only useful if it is already there when the device starts a transfer), and while the master is unreachable the port serves **nothing**: falling back to a local server would let a transfer appear to succeed with the file on the wrong machine. The master must have `allow_relay_kermit` on — off by default, because Kermit's server mode has no authentication of its own.
- **Console-mode port** — the slave registers the port with the master; a master user reaches it from the master's **G Serial Gateway** menu, which lists local ports plus registered remote (slave) ports. A slave's own Serial Gateway menu shows its relayed console port as "→ master" (not selectable locally).

> **Re-registering after a change**
> A registration is a standing claim — the master holds an idle channel and rings it later — so one made under settings that have since changed is worse than none. Both ports therefore watch the settings a registration depends on (the role, the master's host / port / username / password, and that port's own enabled flag, mode, and device path) and re-register within about a second of any of them changing, whichever UI made the change. A serial or server restart re-registers too. Editing something unrelated — a weather location, a timeout — deliberately does *not* disturb a live registration.

Each gateway's serial ports are configured on the gateway they are physically attached to. A slave's ports use the *slave's* own `serial_a_*` / `serial_b_*` settings (device path, baud, mode, flow control, `drive_carrier`, S-registers, `AT&W`); the master's local ports use the *master's*. The relay carries the session — the byte stream, and for a console port its label — **not** the port configuration. Set each device's baud/mode/etc. on the gateway it is wired to; there is no per-slave serial configuration on the master.

In slave mode the gateway still serves its own telnet/SSH and shows a "SLAVE mode: ports relay to master" notice with the master's address on its main menu. The slave reconnects automatically if the link drops. Only the SSH transport is implemented (`relay_transport = ssh`).

9.14.4 Relay Limitations and Operational Notes

- **A modem-mode relay reports only `NO CARRIER` on any failure.** Master unreachable, wrong credentials, a protocol-version skew, or a number not in the *slave's* dial map all look identical at the device — the specific cause is in the **slave's** server log, so check there when a relay dial won't connect.
- **Relay and console-registration channels count against the master's `max_sessions`.** Each active relay call *and each idle console-port registration* uses one master session slot, so size `max_sessions` to cover your slaves' registered ports plus interactive telnet/SSH/web users; a master at capacity refuses further relays.
- **Master and slave must run a compatible relay protocol version.** A skewed pair refuses to relay (the log says "upgrade the older gateway"); keep both gateways on the same release.
- **A changed master SSH host key strands a slave until you clear it.** The slave pins the master's key on first contact; if it later *changes* (e.g. the master was reinstalled) the slave refuses to reconnect as a possible MITM and cannot prompt (it is headless). Delete the master's entry from the slave's `gateway_hosts` file to let it re-pin.
- **Role and relay settings apply at server startup.** Changing `gateway_role` or `master_accept_relays` (or the slave's master address) takes effect on the next server restart, not live — the telnet Master/Slave screen shows a restart reminder; web/GUI users must restart the server themselves. An unrecognized `gateway_role` or `relay_transport` value (or `slave_master_port = 0`) is silently reset to its default.
- **A slave retries with a failure-aware backoff and won't lock out its own IP.** Network errors retry briskly (1→30 s), a wrong-credential rejection backs off ~6 min (longer than the master's 5-minute per-IP lockout window, so a misconfigured slave never trips the ban

against its own host), and a relays-declined master is re-checked every ~60 s. A silently-dead link is detected within ~2 minutes via SSH keepalive.

10. SSH Gateway (Outbound)

The SSH Gateway proxies your inbound telnet or SSH session through to a remote SSH server. Useful for accessing SSH boxes from a Commodore 64 or any other client that can only speak telnet.

10.1 Using It

From the main menu, press `S`. The gateway shows the currently-configured auth mode, then prompts for the remote host, port, and username:

```
Auth: password
Host: example.com
Port (22): 22
Username: alice
```

Auth mode is set by `ssh_gateway_auth` in the config (default `password`). You can change it either from the GUI's *Server → More* popup or from the telnet session's *Configuration → Gateway Configuration* menu. The two modes are mutually exclusive — the gateway never silently falls back from one to the other, so auth failures are unambiguous:

- **Password** — the gateway asks for the remote account's password after the username prompt. Works with any SSH server that allows password auth.
- **Key** — the gateway offers its own Ed25519 keypair (`ethernet_gateway_ssh_key`, generated on first use and persisted at mode `0600`). The remote must have the matching public half in `~/.ssh/authorized_keys` first. The public key is shown in the GUI's Server → More popup when Auth is set to Key; you can also extract it on the command line with `ssh-keygen -y -f ethernet_gateway_ssh_key`.

Press `ESC` twice, quickly, to disconnect. A single ESC is passed to the remote, so `vi` and other ESC-driven software work through a gateway session; the two presses that disconnect must be in quick succession (within half a second) and with no other key between them.

10.2 Host-Key Trust (TOFU)

The first time you dial a given host, the gateway shows the server's SHA-256 fingerprint and asks whether to trust it. Accepted fingerprints are persisted to `gateway_hosts` (mode `0600`) and checked on every subsequent dial. A changed key triggers a prominent warning:

```
WARNING: HOST KEY HAS CHANGED!
This could indicate a security threat.
New type: ssh-ed25519
New fingerprint: SHA256:...
Update key? (Y/N):
```

All trust decisions — first accept, update, and reject — are written to the server log (and the GUI console) so you have a forensic trail if a changed key turns out to be a man-in-the-middle attempt.

10.3 Output Filtering

For PETSCII and ASCII terminals, ANSI escape sequences from the remote are automatically stripped and text is converted to the appropriate encoding. ANSI terminals keep their sequences — colour and cursor addressing are why a terminal asks for ANSI — except for the remote's **window title**. PTY size is set to 40×25 for PETSCII and 80×24 for ANSI/ASCII.

A shell sets the title of the window it believes it is in: bash's default prompt sends `ESC ] 0 ; user@host: ~ BEL` before every prompt. No client of this gateway has a title bar, and a terminal without OSC support swallows the two-byte `ESC ]` and prints the rest — so the title appears on screen ahead of the prompt, which looks like the prompt printing twice. That sequence is therefore dropped for every terminal type — including a modern terminal that *does* have a title bar, which loses title updates through a gateway session as a result. Nothing can ask a terminal whether it implements OSC, and the terminals that cannot are the ones this gateway exists for. Only the title form is taken, and only when it completes: the same stream carries file transfers (run `sz` on the far host and its ZMODEM bytes come through here), so a candidate is *held* and released byte for byte unless it proves to be `ESC ]`, then `0;` / `1;` / `2;`, then printable ASCII, then BEL or `ESC \`, all inside 256 bytes. A download that merely happens to contain `1B 5D` is untouched.

10.4 Key File Summary

File	Role	Analogous to
`gateway_hosts`	Remote host fingerprints accepted by the SSH Gateway.	`~/.ssh/known_hosts`
`ethernet_ssh_host_key`	Host key the SSH *server* presents to incoming clients.	`/etc/ssh/ssh_host_ed25519_key`
`ethernet_gateway_ssh_key`	Outbound SSH client keypair used for pubkey auth to remote hosts.	`~/.ssh/id_ed25519`

11. Telnet Gateway (Outbound)

Press `T` from the main menu to dial a remote telnet server or any raw-TCP service on port 23 (BBSes, MUDs, legacy hand-rolled software).

11.1 Protocol Mode

The gateway displays the currently-configured mode on entry and proceeds straight to the host prompt. The mode is a persistent setting, not a per-session choice:

```
Mode: Telnet protocol
Host:
```

Change the mode from the GUI (*Server → More* popup, Telnet Gateway Mode dropdown) or from the telnet session's *Configuration → Gateway Configuration* menu. Changes take effect on the next dial — no restart needed.

11.2 The Three Modes Explained

11.2.1 Reactive mode (default)

`telnet_gateway_negotiate = false` and `telnet_gateway_raw = false`. The gateway parses IAC in both directions but does not send proactive option offers. It silently accepts cooperative `WILL ECHO` from the remote (so BBSes that expect the server to echo work correctly), refuses every other peer-initiated option with `DONT` / `WONT`, escapes outbound `0xFF` data bytes as `IAC IAC`, and parses inbound IAC so protocol bytes don't leak to the user's terminal. Works with any real telnet server *and* with raw-TCP services on port 23 that never initiate telnet options.

11.2.2 Cooperative mode

`telnet_gateway_negotiate = true`. In addition to reactive mode's behaviour, the gateway sends proactive `IAC WILL TTYPE`, `IAC WILL NAWS`, and `IAC DO ECHO` at connect time. On `SB TTYPE SEND` it replies with the user's terminal type (`PETSCII`, `ANSI`, or `DUMB`). On `DO NAWS` it replies with the user's actual window dimensions, and it forwards in-session NAWS updates if the user resizes their terminal. Option state is tracked through a full RFC 1143 six-state Q-method, so a mind-change while a prior offer is in flight resolves cleanly.

Enable this when dialling modern BBSes — you get echo cooperation, correct terminal-type adaptation, and full-screen UI at your actual window size.

11.2.3 Raw TCP

`telnet_gateway_raw = true`. The gateway disables its entire telnet-IAC layer: no IAC escaping on outbound, no IAC parsing on inbound. Supersedes `telnet_gateway_negotiate`. Intended for destinations that aren't really telnet — some legacy MUDs, hand-rolled BBS software, or services that happen to run on port 23 but speak some other byte-level protocol.

Bytes written *to* the local user are still IAC-escaped so their telnet client does not misinterpret a stray `0xFF` as a protocol byte.

11.3 Dialing

After the mode prompt, enter the host and port:

```
Host: bbs.example.com
Port (23): 23
```

Press `ESC` twice, quickly, to disconnect — a single ESC goes to the remote (§10.3). For PETSCII and ASCII terminals, ANSI escape sequences from the remote are filtered out; an ANSI terminal keeps them, apart from the remote's window title, which is dropped for every terminal type (§10.3 — the same filter serves both gateways). The gateway also has an 8 KiB cap on subnegotiation body length, so a malicious remote cannot exhaust memory by sending huge `SB` bodies without a terminating `IAC SE`.

12. AI Chat

The AI Chat feature sends prompts to the Groq API and displays the responses in the terminal. It is free to use — you need a Groq account and an API key — and the model is `llama-3.3-70b-versatile`.

12.1 Setup

1. Go to `https://console.groq.com/keys` and create a free account.
2. Generate an API key. It starts with `gsk_`.
3. In the GUI, paste the key into the *API Key* field (it displays as dots), or edit `egateway.conf` and set `groq_api_key = gsk_your_key_here`.
4. Save. No restart needed.

12.2 Using AI Chat

From the main menu, press [A]. Type a question and press Enter. The server shows *Thinking...* while waiting for the response (timeout 30 seconds). The answer is paginated:

- [N] / [P] — next / previous page.
- [Q] — return to the main menu.
- From the answer screen, type a new question to continue the conversation.

Responses are word-wrapped to fit the terminal — 38 columns for PETSCII, 78 for ANSI/ASCII. Long code blocks and unusual punctuation are preserved but may wrap awkwardly on narrow screens.

> **No API key?**
> If `groq_api_key` is empty and the user presses [A], the gateway displays instructions for obtaining a key instead of attempting the API call.

13. Web Browser

The text-mode browser renders HTML as plain text with numbered link markers. It works on all terminal types, including 40-column PETSCII.

13.1 Opening a Page

From the main menu, press B. The homepage loads automatically if `browser_homepage` is set (default `http://telnetbible.com`). You can also enter a URL or search query:

- A URL without a scheme — `example.com` — gets `https://` prepended automatically.
- Text without dots is treated as a search query and sent to DuckDuckGo.
- A `gopher://` URL uses the Gopher protocol.

13.2 Following Links

Links are marked with numbered tags: `[1]`, `[2]`, `[3]`. Press L and enter a number to follow.

13.3 Navigation Commands

Key	Action
N / P	Next / previous page.
T / E	Jump to top / end.
L	Follow link by number.
G	Go to a new URL or search.
S	Search for text in the page.
F	Fill out and submit forms.
K	Save as bookmark; on the home screen, opens the bookmarks list.
B	Back to the previous page.
R	Reload.
H	Help.
Q	Close page (back to browser home).
ESC	Exit to main menu.

13.4 Bookmarks and Forms

Up to 100 bookmarks are saved in `bookmarks.txt` next to the binary. When a page has forms, press F to interact — pick a form, edit fields by number, S to submit or Q to cancel.

13.5 Limits

- Maximum page size: 1 MB.
- Maximum rendered lines: 5000.
- HTTP request timeout: 15 s.
- Page history depth: 50.
- On a TLS error, an HTTPS *page load* (GET) retries over HTTP with a warning; a *form submission* (POST) is refused, never re-sent in the clear.

14. Weather

The Weather feature displays the current conditions and a 3-day forecast for any city or postal code **worldwide**. Data is pulled from Open-Meteo (`https://open-meteo.com`), which is free and requires no API key, with MET Norway (`https://api.met.no`) as an automatic fallback forecast provider.

1. From the main menu, press `W`.
2. Enter a city or postal code, or press Enter to accept the last-used one. Examples: `62051`, `London`, `London, GB` (or `London, Ontario`), `Zürich`, `Tokyo, JP`. A `City, Country` (or `City, Region`) qualifier disambiguates common names; the matched country is shown so you can confirm.
3. Current temperature, humidity, wind speed and compass direction, plus a 3-day forecast are displayed. Press `U` on the weather screen to cycle the display units.

Units follow the `weather_units` setting: `auto` (the default — Fahrenheit/mph for the US, Celsius/km/h everywhere else), `us`, or `metric`.

The last-used location is persisted to `weather_location` in `egateway.conf` so subsequent calls default to the same place. An older config's `weather_zip` value is migrated automatically on first load.

15. Signal Handling & Operations

The gateway installs POSIX signal handlers at startup using the `signal-hook` crate. On receipt of any of `SIGINT`, `SIGTERM`, or `SIGHUP`, the server begins an orderly shutdown.

Signal	Typical source	Effect
`SIGINT`	`Ctrl+C` from the terminal	Shut down. Connected sessions are notified with a broadcast message.
`SIGTERM`	`kill`, systemd	Shut down.
`SIGHUP`	systemd `reload`, terminal disconnect	Shut down. (The unit file maps *reload* to SIGHUP for familiar semantics.)

15.1 Shutdown Sequence

When a signal arrives:

1. An `AtomicBool` flag is set.
2. A watcher thread fires a tokio `Notify` to wake the async runtime.
3. Listeners stop accepting new connections.
4. A broadcast notification is sent to every active telnet, SSH, and master/slave relay session: "Server shutting down. Goodbye."
5. The serial modem thread emits the same notice and exits cleanly, releasing the port.
6. The GUI window closes automatically.
7. The process exits.

15.2 Under systemd

```
sudo systemctl status ethernetgateway   # current status
journalctl -u ethernetgateway -f        # live log
sudo systemctl reload ethernetgateway   # graceful stop (SIGHUP)
sudo systemctl restart ethernetgateway  # restart
sudo systemctl stop ethernetgateway     # stop (SIGTERM)
```

The unit's `Restart=on-failure` policy respawns the process if it exits with a non-zero code, rate-limited to 5 starts per 60 seconds via `StartLimitIntervalSec` / `StartLimitBurst`.

16. Troubleshooting

16.1 Telnet Session Refused (Public IP)

If an unauthenticated client sees "Connections from this address are not accepted", the source IP is outside the private ranges listed in chapter 5. Either connect from a private-network address or enable `security_enabled = true` and configure credentials.

16.2 Session Denied — Too Many Attempts

Per-IP lockout after three failures. Wait 5 minutes. The lockout covers both telnet and SSH — retrying on the other protocol will not reset the counter.

16.3 No Serial Port Detected

- Plug in the adapter before clicking the refresh (↻) button in the GUI.
- On Linux, make sure your user is in the `dialout` group (`sudo usermod -aG dialout "$USER"`), then log out and back in.
- On Linux under systemd, the unit file sets `PrivateDevices=yes` which hides most device nodes. If you need USB-serial access, remove that hardening line or switch to a raw `DeviceAllow=` rule for your exact adapter.
- On macOS, adapters typically show up as `/dev/tty.usbserial-XXXX` or `/dev/tty.usbmodemXXXX`.
- On Windows, check Device Manager. Ports appear as `COM3`, `COM4`, etc.

16.4 File Transfer Times Out

Turn on verbose mode (`verbose = true` in `egateway.conf`, or the *Verbose Transfer Logging* checkbox in the GUI General frame) and retry. The log will show every header byte, every CRC result, every timeout, tagged with the protocol (`XMODEM recv:`, `ZMODEM send:`, etc.). Common causes:

- Client and server disagree on CRC vs. checksum — the client is very old and never implemented CRC. The gateway falls back to checksum automatically after two-thirds of the negotiation timeout.
- IAC escaping mismatch when transferring binary files with `0xFF` bytes over telnet. Toggle it with `I` in the File Transfer menu.
- Slow or unreliable serial link — raise `xmodem_block_timeout` and `xmodem_max_retries` for XMODEM/YMODEM, `zmodem_frame_timeout` and `zmodem_max_retries` for ZMODEM, or `punter_block_timeout` and `punter_max_retries` for Punter (and lower `punter_block_size` toward ~40 on a noisy line).
- Client is slow to start after you pick the protocol — raise `xmodem_negotiation_timeout` (XMODEM/YMODEM), `zmodem_negotiation_timeout` (ZMODEM), or `punter_negotiation_timeout` (Punter).

- Log shows duplicate block 0 retransmissions during YMODEM setup — raise `xmodem_negotiation_retry_interval` toward the spec-recommended 10 s so fewer `C` pokes stack up in the sender's buffer before it starts responding.

16.5 SSH Won't Connect

- Check `ssh_enabled = true` and that the server has been restarted since the change.
- Confirm the port with `ss -tlnp | grep 2222` on Linux.
- On first connect, accept the gateway's host-key fingerprint. Your client records it in `~/.ssh/known_hosts`.
- If your client complains about host-key mismatch, you have probably regenerated `ethernet_ssh_host_key`. Delete the old entry from `known_hosts` or compare fingerprints carefully.
- Public-key auth against the *server* is not supported — only password auth. Public-key auth is only available on the *outbound* SSH Gateway.

16.6 Modem Shows "NO CARRIER" on Every Dial

- Check the host and port — `ATDT host:port` must include the port unless the host resolves to something listening on 23.
- S7 defaults to 15 seconds. Networks that need longer dial times should `ATS7=50` (or up to 60).
- Verify the host is reachable from the gateway machine with `nc -zv host port` or `telnet host port`.
- If you dialled a phone number (all-digit string), check `dialup.conf` — unmapped numbers return `NO CARRIER`.

16.7 GUI Won't Open

- Check that `enable_console = true`.
- On headless servers, leave it at `false` — the GUI requires a display server (X11 or Wayland).
- On Linux, make sure `libudev`, `libxkbcommon`, and your display-server libraries are installed.
- Missing GPU support? The GUI uses `wgpu` via eframe. Software-only fallback via a different feature set would require a rebuild; most modern Linux distros work out of the box.

16.8 Stuck Serial Session After Parameter Change

If you change baud from a serial session with mismatched terminal settings, don't panic. The gateway waits up to 60 seconds for confirmation and reverts automatically on timeout. Within 60 seconds your old settings come back.

16.9 Garbled Input from a Commodore 64 (Bit-Banged Serial)

Symptom: a command typed on a C64 over a userport RS-232 adapter runs *truncated or corrupted* — usually a single typed character arrives as a stray PETSCII graphics byte — and it tends to happen on *longer* commands that wrap past the 40-column screen edge. It is intermittent: the same command may go through cleanly on the next try.

Cause: passive userport adapters like the **EZ232** have no UART; the C64 times every serial bit in software ("bit-banging"), which has very little timing margin. When a long line wraps, the screen scrolls and the KERNAL briefly masks interrupts to move screen memory — long enough to disrupt the bit timing of a keystroke being transmitted at that moment, garbling the byte on the wire. This is a C64 transmit-side timing fault that happens *before* any byte reaches the gateway, so it affects the SSH Gateway, the Telnet Gateway, and every other feature that rides the same serial link.

Fix:

- **Use 1200 baud, not 2400**, on bit-banged userport adapters. Set `serial_x_baud = 1200` on the gateway port *and* set your C64 terminal program (e.g. CCGMS) to 1200. This roughly doubles the per-bit timing budget and is the reliable cure.
- If you must run faster, a hardware-UART interface (SwiftLink / Turbo232) or a WiFi modem with its own UART removes the bit-banging entirely and is not affected.
- To confirm the diagnosis on your own hardware, enable the gateway byte-trace (§16.10) and watch the per-byte timing and the assembled `[gw-in] line`.
- **If the bytes in that trace are correct** and only the *drawing* is wrong — wrap in the wrong column, backspace deleting the wrong character on screen, leftovers after a history recall — then this is not a timing fault at all. See §16.15 first (a PETSCII translation bug fixed in this release, which corrupted long lines on every C64 gateway session regardless of adapter), then §16.14 if the remote has been told the wrong terminal width.

> **Fixed in this release, and it looked exactly like this**
>
> Before this version the gateway translated a remote host's **backspace** into PETSCII `0x14` (DEL), which *deletes* a character instead of just moving over it. Long lines were mangled on every C64 gateway session, bit-banged adapter or not. If you are chasing this on an older build, that is the first thing to rule out — see §16.15.

16.10 Gateway Byte-Trace (Debug Mode)

The SSH Gateway and Telnet Gateway can log every byte that crosses the gateway, which is useful for diagnosing input corruption, terminal translation, and flow problems like §16.9. It costs nothing when off and needs no special build.

Enable it with the `gateway_debug` setting, which you can toggle from any of three places without restarting the program:

- **GUI:** the *General* frame — tick *Gateway Debug Trace* (next to *Show GUI on Startup*) and click *Save*.

- **Web console:** the *General* frame — same checkbox, then *Save*.
- **In-session telnet menu:** *Configuration → O Other Settings → D*, or *Configuration → M Serial Configuration → D*. Both toggle the same flag.

The flag is read fresh when each gateway session starts, so a change takes effect on the *next* SSH/Telnet Gateway connection — no restart needed. It is persisted to `egateway.conf` as `gateway_debug = true`.

> **Forcing it on from the environment**
>
> The `EGATEWAY_GATEWAY_DEBUG` environment variable still forces the trace on regardless of the config flag — handy for headless or scripted runs where you want it active from the moment the process starts:
>
> ```
> EGATEWAY_GATEWAY_DEBUG=1 ./ethernetgateway
> ```
>
> Under systemd, add `Environment=EGATEWAY_GATEWAY_DEBUG=1` to the unit. When set, it overrides the `gateway_debug` setting.

Where the trace appears: the same places as all gateway log output — the process's stderr, the GUI's live console, and (when `web_enabled = true`) the web-config console's `/logs` endpoint, e.g. `curl http://host:8080/logs`.

Reading the trace. When a gateway session opens you'll see a banner, then one line per input byte and a summary line per typed command:

Line	Meaning
`[gw] SSH gateway trace ON — …`	Session opened (or `Telnet gateway`); shows terminal type and PTY/ window size.
`[gw-in] +Δms t=…ms byte=0x*NN* '*c*' (petscii 0x*NN*)`	One byte forwarded to the remote. `+Δms` is the gap since the previous byte; `t=` is elapsed since the trace started; the `petscii` note appears only when the PETSCII→ASCII case-swap changed the byte.
`[gw-in] line (*N* bytes) -> *hex* | "*ascii*"`	The full line as the remote shell received it, flushed on RETURN (CR/LF). This is exactly what bash ran.
`[gw-out] raw *N* bytes -> … / filtered …`	Bytes received from the remote, shown before and after the PETSCII/ ASCII output filter.

The `+Δms` timing distinguishes terminal behaviours: large per-byte gaps mean the client sends each keystroke live (character mode, e.g. CCGMS), while a near-zero burst of bytes means a whole line was sent at once. This is verbose, per-byte output intended for troubleshooting — leave it disabled in normal operation.

16.11 Transferred File Won't Run (Wrong Size by a Few Bytes)

Symptom: a program or data file transfers with no reported errors — every block check passed, the client said the transfer completed — but the file won't run, or a game won't load its data. Comparing it against the original shows the size is off by a handful of bytes out of tens of thousands.

Cause: the *client* was in text (ASCII) file mode. Vintage Kermit clients default to it, and in text mode a CP/M client treats `^Z` (0x1A, the CP/M text end-of-file pad) as padding rather than data — so a `^Z` landing on a 128-byte record boundary gets dropped and everything after it shifts down one byte. Nothing in the protocol catches this: each packet is individually valid, so the corruption is completely silent.

Fix: set binary file mode on the client and transfer again — `SET FILE MODE BINARY` on kercpm3 / CP/M Kermit (some builds spell it `SET FILE TYPE BINARY`), `set file type binary` or the `-i` flag on C-Kermit. The gateway always transfers binary in both directions, so this is purely a client-side setting — which is why a download can be perfect while an upload of the same file is damaged. See the note in §8.8 for a worked example.

Verifying: compare a checksum (`md5sum`, `cmp`) of both copies. Do *not* compare *packet counts* between an upload and a download of the same file — those legitimately differ, because control bytes and high-bit bytes are quoted into two or three wire characters each and how much that expands depends on the negotiated packet length and on which side is sending. A differing packet count is not evidence of data loss, and a matching one is not evidence of success.

16.12 The Setup Wizard Appears (or Doesn’t)

The first-run wizard (§3.1) is gated on the single key `setup_wizard_completed`, so both symptoms have the same one-line answer.

- **It appears every launch.** The key isn’t being persisted — the wizard sets it to `true` on both *Save and Restart* and *Exit*, so this means the write failed. Check that `egateway.conf` and its directory are writable by the account the gateway runs as; the console log prints `Configuration NOT saved` with the reason. Closing the window with the title-bar X instead of using the wizard’s own buttons also leaves the key untouched, by design.
- **It never appears on a new install.** Either the GUI is off (`enable_console = false` — the wizard is GUI-only, and will show up the first time the GUI does run), or the `egateway.conf` in the working directory predates this version: a config file with no `setup_wizard_completed` key at all is treated as already configured. Add `setup_wizard_completed = false` to the file, or click *Run setup wizard…* in the GUI’s *Server — More…* window.
- **It won’t leave a screen.** The message in red under the fields says why — mismatched passwords, an empty username, a port that isn’t 1–65535, or a port another listener already claims (including the Kermit server on `kermit_server_port`, which the wizard doesn’t show).

16.13 “Address already in use” / No Listeners Came Up

On startup the gateway now says so plainly when nothing came up:

```
WARNING: NONE of the 4 configured listener(s) could bind (Kermit 2424, SSH 2222,
telnet 2323, web 8080).
         This process is serving no network connections at all.
         The ports are already in use — another copy of the gateway is
         almost certainly still running and holding them, which means
         anything you connect to is being served by that older copy.
         Check:  pgrep -a -x ethernetgateway     (and: ss -ltnp | grep <port>)
         Stop it with:  pkill -x ethernetgateway   then start this one again.
```

Almost always this is exactly what it says: a second copy of the gateway was started without stopping the first. The consequence is worth spelling out, because it is genuinely confusing — the older copy still holds the ports, so everything you telnet or SSH into is being served by *it*, running whatever binary and configuration it started with. A rebuild, a new setting, or a *Save and Restart* in the newer copy changes nothing you can see. On Linux the underlying error is `os error 98`.

- **Find it:** `pgrep -a -x ethernetgateway` lists every copy; `ss -ltnp | grep 2323` names the PID holding a port. A copy whose `/proc/<pid>/exe` link says `(deleted)` is running a binary that has since been rebuilt — a sure sign it predates your current build. On Windows, `netstat -ano | findstr :2323` gives the PID, and Task Manager stops it.
- **Stop it:** `pkill -x ethernetgateway`, then start the copy you want. Note this stops *every* copy, which is usually what you want.
- **Not a second copy?** If the warning appears but no other copy exists, something else on the machine has the port — check the PID that `ss -ltnp` reports. A total failure that is *not* an in-use error (ports below 1024 without root, for instance) prints the same first two lines without blaming another copy.
- **One listener only:** a partial failure prints a shorter note (`1 of 4 listeners could not bind …; the rest are running`) — the gateway is up, just missing that one service.

> **A RESTART DOES NOT LEAK A LISTENER**
> Restarting from the GUI (*Save and Restart*, including the setup wizard's), from the telnet menu, or with `systemctl reload` / `SIGHUP` tears the old listeners down before binding the new ones. If this warning appears right after such a restart, it is reporting a conflict that was already there — the restart re-ran the binds and put the failure back in front of you.

16.14 The Remote Wraps Lines in the Wrong Place

Symptom: on a gateway session, everything is fine until a line gets long. Past a certain column the remote’s echo lands in the wrong place: line wrap happens early or late, backspace deletes the wrong character on screen, and recalling a command from history or pressing Tab leaves stale characters behind. Short commands are always fine.

Cause: the remote host has been told the wrong terminal width. The gateway reports a size to the remote — the SSH Gateway in its PTY request, the Telnet Gateway via NAWS — and `bash`/readline does all of its cursor arithmetic against that number. If it disagrees with your screen, every redraw past the real margin is computed for a screen you do not have. Nothing is lost on the wire; the remote receives exactly what you typed. But if you then correct what you *see*, you can end up running a command that differs from the one you meant.

By default the gateway uses the size your client negotiated over telnet NAWS, falling back to the terminal-type default — 40×25 for PETSCII, 80×24 for ANSI/ASCII. That guess is wrong more often than it looks, because **terminal type does not imply terminal width**:

- A C64 running **CCGMS in ASCII mode** sends `0x08` for backspace, so the gateway detects it as ANSI and would report **80 columns for a 40-column screen**.
- CCGMS's soft **80-column mode** is the same mistake in reverse: detected as PETSCII, reported as 40 columns, actually 80.
- WiFi modems and `tcpser` do not send NAWS on the C64's behalf, so there is nothing for the automatic path to use.

Fix: pin the geometry with `gateway_term_width` and `gateway_term_height` — telnet *Configuration → Gateway* (`W` and `R`), or *Server → More* in the GUI and web UI. Set the width to what your screen actually is (40 for a stock C64) and leave the rows at `0` if you only need the width corrected; the two resolve independently. `0` means automatic on both. An override beats client NAWS on purpose — the point is to correct a client that cannot report itself accurately. Changes apply to the next gateway connection with no restart.

To see what is actually being sent, enable the byte-trace (§16.10) and read the diagnostic line, which names both the resolved geometry and which input won:

```
[gw-diag] window (NAWS):   <not negotiated>
[gw-diag] onward geometry:  40x25  (from config override)
```

> **Not the same as a garbled character**
>
> If instead of misplaced *drawing* you are getting genuinely wrong *bytes* — a stray PETSCII graphics character in place of one you typed, intermittently — that is a different fault with a different fix: see §16.9, which is a C64 transmit-timing problem on bit-banged userport adapters and is cured by dropping to 1200 baud. The two look alike because both show up on long lines that wrap. The byte-trace tells them apart: a timing fault corrupts the bytes in the `[gw-in]` log, a width mismatch leaves them perfectly correct.

16.15 Long Commands Corrupted on a C64 Gateway Session (fixed)

Symptom: on the SSH or Telnet Gateway from a C64 in PETSCII mode, short commands work perfectly but a command long enough to pass the 40-column margin comes out mangled — characters missing, the visible line disagreeing with what you typed — and editing with DEL makes it worse. On older builds this happened on every C64 gateway session, whatever serial interface was in use.

Cause (a real bug, fixed in this release): the gateway translated a host's ASCII backspace into PETSCII `0x14`. Those are not equivalents:

- ASCII `0x08` (BS) moves the cursor left one column and **erases nothing**.
- PETSCII `0x14` (DEL) **deletes** the character to the left and pulls the rest of the line back.
- PETSCII `0x9D` (CRSR LEFT) is the actual equivalent, and is what the gateway now sends.

A host uses backspace two ways, and the old mapping broke both. Measured against a real `bash` at `TERM=dumb` in a 40-column window: as soon as a typed line passes the margin, readline redraws it and then emits a run of **bare backspaces purely to reposition the cursor** — eleven of them in one observed case. Each one used to *delete* a character the host still believed was on screen. Separately, readline erases a character with the universal `BS SPACE BS`, which became `DEL SPACE DEL`: delete a character, insert a space, delete that.

Short commands never trigger a redraw, which is exactly why they always looked fine. With `0x9D`, `BS SPACE BS` renders as left, space, left — the character is overwritten with a blank and the cursor ends up before it, which is what the host means.

Two paths were affected and both are fixed: the SSH and Telnet Gateways, and a C64 dialling out through the modem emulator with `AT+PETSCII=1`. If you run an older build, both show the same corruption. The modem emulator's own `AT`-command echo is unaffected and always was correct — there it deliberately sends `0x14` to erase a character it echoed itself, which is the right primitive when you are the one originating the erase rather than translating someone else's output.

> **Why the ANSI filter was not the problem**
>
> It is natural to suspect the PETSCII path's escape-sequence stripping, since it discards every CSI sequence. Measurement ruled it out: at `TERM=dumb` readline emits **no CSI sequences at all** — it has no clr_eol or cursor-addressing capability to use, so it redraws with carriage return, spaces and backspaces only. There was nothing for the filter to discard.

16.16 The Prompt Appears Twice on a Gateway Session (fixed)

Symptom: on the SSH or Telnet Gateway from an ANSI terminal — measured on a real SC126 — every bash prompt is preceded by what looks like a second copy of itself with a stray number in front:

```
0;ricky@TelnetBible: ~ricky@TelnetBible:~$
```

Cause (a real bug, fixed in this release): bash sets the title of the window it believes it is in, by sending `ESC ] 0 ; user@host: ~ BEL` ahead of the prompt. A terminal that implements OSC consumes the whole sequence and puts the text in its title bar. A terminal that does not swallows only the two-byte `ESC ]` it does not recognise and *prints the rest* — and since the title is user@host, the result reads as the prompt printed twice. No client of this gateway has a title bar, so the sequence had no destination in the first place. It reached ANSI terminals because the ANSI path bypassed the output filter entirely; PETSCII and ASCII terminals,

which have every sequence stripped, never saw it. Seen under both EGT80 and QTERM, which is what ruled out a fault in either terminal and put the fix in the gateway.

Fix: the window-title sequence is now dropped for every terminal type. ANSI terminals keep everything else, colour and cursor addressing included — that is what they asked for. If you are on an older build and cannot upgrade, unset the title from your own shell: `PROMPT_COMMAND=` in bash, or a `PS1` without the `\e]0;…\a` part.

> **Why the title is weighed rather than swallowed**
>
> A gateway session is a plain terminal proxy, so a file transfer run on the far host comes down the same stream — run `sz` there and its ZMODEM bytes arrive here with nothing to distinguish them from a prompt. The bytes `1B 5D` occur about once per 64 KB of binary, so a filter that swallowed `ESC ]` through to the next BEL would eat part of the download, and eat it *identically* on every retry — so the protocol's own CRC could never recover it. The filter therefore holds a candidate and drops it only once it has proved to be a title: `ESC ]`, then `0;` / `1;` / `2;`, then printable ASCII, then BEL or `ESC \`, all inside 256 bytes. Anything that fails any of those is released byte for byte in the order it arrived, so passing data through is the default and dropping is the exception. And a candidate is released the moment the remote goes quiet (100 ms): carrying one across a read is necessary, since a burst larger than the 4 KB read buffer splits at an arbitrary byte, but carrying one indefinitely would deadlock a stop-and-wait transfer — a block whose last byte is being weighed never completes, the receiver times out, the sender resends the identical block, and the identical hold repeats.

16.17 A Single ESC Never Reaches the Remote (fixed)

Symptom: through the SSH gateway, the telnet gateway or the serial console bridge, pressing `ESC` once does nothing at the remote — `vi` will not leave insert mode, a menu driven by ESC does not respond — and pressing it a second time disconnects you back to the gateway menu. Dialling the same host straight from the modem with `ATDT host:port` works perfectly, which is what makes it look like the gateway rather than the remote.

Cause (a real bug, fixed in this release): the gateway *held* an ESC rather than sending it, and released it only when a following byte arrived. That is how an arrow key — `ESC [ A` — reached the remote intact, and it meant a *lone* ESC waited for a second byte that never came. So one press was swallowed and the next was read as the second half of the pair that disconnects. A modem dial-out is a raw TCP passthrough that intercepts nothing, which is why that path was unaffected. Reported from an SC126; the same defect was present in all three bridges, which were three copies of one rule.

Fix: nothing is held. Every ESC is passed to the remote along with every other byte, and what disconnects you is **two ESC presses in a row, within half a second**, with no other key between them. The second one is the gateway's and is not forwarded.

Why the pair is timed *and* consecutive

An arrow key is `ESC [ A`, so two cursor presses put two ESC bytes a few milliseconds apart — far closer together than any human double-tap. A rule that measured only elapsed time would disconnect you for pressing Up twice. The `[` between them is the entire signal, so any other key resets the pair. Note also what changed for the way you leave: the two presses must now be *quick*. They used to count as a pair however far apart, so an ESC to leave `vi`'s insert mode and another a minute later ended the session.

Appendix A — Configuration Key Reference

Every key recognised by `egateway.conf`, sorted alphabetically, with two intentional deferrals: the `kermit_*` protocol-tuning family (everything *except* `kermit_server_enabled` and `kermit_server_port`, which are listed here because they affect the gateway's listening surface) is documented in the standalone `kermit.html` reference, and §3.3 of this manual carries one-line summaries of the entire family for at-a-glance scanning. The parser is case-sensitive on keys but booleans (`true`/`false`) are compared case-insensitively. Unknown keys are silently ignored. Out-of-range numeric values fall back to the default.

Key	Type	Default	Range / validation	Description
allow_atdt_kermit	bool	false	true/false	Permit ATDT KERMIT (and the aliases ATDT kermit / ATDT kermit-server / ATDT kermit server) on the serial modem to drop the caller straight into Kermit server mode — no telnet menu, no auth gate. Off by default because it bypasses every security_enabled check.
allow_peer_dial	bool	false	true/false	Peer-dial: let a modem-mode port call another serial port directly (ATD <Port>@<IP>, or select a modem port in the Serial Gateway picker) and bridge to the device on it; a dialed modem port rings and answers per its own S0/ATA, a console port connects directly. **Off by default** — it lets any modem caller ring or connect to any addressable port, so it is opt-in even under the trusted-LAN threat model. It gates *dialing*, not registration: a slave announces its ports and its CP/M endpoint to its own master regardless (see §9.14.3), and a master needs it on to accept a third party's crossbar dial into a slave. See §9.2.3.
allow_relay_kermit	bool	false	true/false	**Master only.** Run this master's Kermit server on the relay channel of a slave port that is in Kermit-server mode, so that device's remote dir, uploads and downloads all resolve against the master's transfer_dir; the slave serves nothing itself and keeps no copy. **Off by default** — Kermit server mode is unauthenticated (the same reason allow_atdt_kermit and kermit_server_enabled are opt-in), though this path at least requires a slave that authenticated to the master over SSH with master_accept_relays already on. Editable from the telnet *Server › M Master/Slave* sub-screen (K), the web *Master/Slave* card, and the GUI Server *More…* popup. See §9.14.3.
kermit_wait_for_receiver	bool	true	true/false	Wait for the receiver's initiating NAK before sending our Send-Init on an interactive Kermit download, so the

Key	Type	Default	Range / validation	Description
				Send-Init doesn't appear as garbage on a terminal whose client isn't in receive mode yet. Sends anyway if no NAK arrives within the negotiation timeout, so peers that never NAK still work. Only the interactive telnet download consults it — server and test paths never wait.
`browser_homepage`	string	`http://telnetbible.com`	any URL or empty	Page loaded when the browser opens. Empty starts with a blank prompt.
`cpm_emu_enabled`	bool	`true`	true/false	The CP/M emulator `K` main-menu item, which runs arbitrary Z80 `.COM` software sandboxed to `CPM/A`–`CPM/P` under `transfer_dir`. **On by default**: the guest is jailed to those folders, bounded by `cpm_emu_max_minstr`, always escapable with a double-`ESC`, and has no path to a host command. Set `false` to hide the item and reject the key. See §8.11.
`cpm_screen_input`	bool	`true`	true/false	May the web UI's *VDM / Dazzler* page send keystrokes to a booted disk? The screen is readable either way. See §6.4.3.
`cpm_emu_max_minstr`	u32	`2000`	1–1000000	CP/M emulator runaway ceiling, in millions of instructions per program run (2000 = 2 billion). A compute-bound `.COM` that never reads the console is aborted at this count so the `A>` prompt always returns. Values above **1000000** are capped at it rather than refused, so a setting meant as “no limit” is kept as far as it goes instead of dropping to the default. See §8.11.
`cpm_boot_image`	string	*(empty)*	empty, or a filename in `CPM/images`	What the CP/M menu item runs. Empty = the CP/M emulator. A bare filename cold-boots that disk on the emulated controller its boot loader names and gives the disk’s own operating system the whole machine — no jail, no `A>` from us, no EGT8080, no `EXIT`; press ESC twice to leave. Opened **writable** unless `cpm_boot_writable` is turned off, and held by one session. A name that is not in the folder falls back to the emulator and

Key	Type	Default	Range / validation	Description
				is shown as `(missing)`; one carrying no boot program shows `(not bootable)`. See §8.11.
`cpm_boot_machine`	string	`auto`	`auto`, `altair_2sio`, `altair_sio`, `console_04`, `console_04_cuter`, `z80pack`, `cromemco`	Which machine a *booted* disk runs on — its console **and** the set of disk controllers it carries, which is why a machine is one setting rather than two: z80pack's disk device answers on the same ports as the Altair console, so the two cannot be mixed. `auto` reads the ports the disk's own boot loader drives and falls back to `altair_2sio`; the boot screen reports which happened. See §8.11.
`cpm_boot_backspace`	string	`backspace`	`backspace`, `rubout`	Which byte a *booted* disk gets for the Backspace key. Measured across 38 bootable images: 29 erase on BS (`08h`) and read DEL as a rubout that reprints the deleted character; 7 accept either; CP/M 1.3, 1.4 and the 1975 build are the opposite and need `rubout`. Ignored by the emulator. See §8.11.
`cpm_boot_writable`	bool	`true`	`true`, `false`	May a *booted* disk write to its images? On by default, because an operating system whose writes are discarded loses the work silently. Covers the boot disk and every image mounted beside it. Ignored by the emulator. See §8.11.
`cpm_cpu`	string	`z80`	`z80`, `8080`	Which processor **both** CP/M machines run — the only CP/M setting that reaches the emulator as well as a booted disk. `z80` runs everything here; `8080` is the more literal Altair and is what period 8080 diagnostics expect. Two terminals are placed on drive A: and in the transfer directory: `EGT8080.COM`, built to the 8080's instruction set and therefore running on either processor, and `EGT80.COM`, the Z80 build, which adds the Z180 ASCI ports and must not be run under `8080`. See §8.11.
`cpm_printer`	string	`text`	`off`, `odt`, `text`	Where CP/M printer output goes. Reaches **both** CP/M machines, like `cpm_cpu`, but by two entirely different routes: in the emulator the printer is an

Key	Type	Default	Range / validation	Description
				operating-system *service* (BDOS function 5 and the BIOS `LIST` vector), so WordStar, MBASIC's `LPRINT` and `PIP LST:=FILE.TXT` all arrive there; a *booted* disk drives a printer *board* instead, chosen with `cpm_printer_port`. Either way one document is written into a `printer` folder inside the transfer directory — its own folder so a printer left on does not scatter files through yours, and not onto a CP/M drive, because it is for you rather than for the guest. `odt` = OpenDocument text, monospaced, one page per form feed; `text` = plain text with the form feeds kept; `off` = printer output appears on your terminal, which is where it has always gone. **Text is the default since 0.9.2** — nothing is written until a guest actually prints, so an operator who never uses `LST:` never sees a file, whereas `off` meant the first printout anyone made was scattered across their terminal with no way to get it back. Named `PRINT-YYYYMMDD-HHMMSS` from this machine's clock. A job is finished 5 seconds after the last character printed — CP/M has no end-of-print signal, so silence is the only one there is — and in the emulator also the moment the program returns to `A>`, which is exact. Bold and underline survive into an `.odt`: period software does not ask for them, it **overstrikes**, and that is turned into real styling. See §8.11.
`cpm_printer_port`	string	`altair_c`	`off`, `altair_c`	Which printer board a *booted* disk finds. Ignored by the emulator, whose printer is a BDOS service with no port at all, and ignored entirely when `cpm_printer` is `off`. `altair_c` = the Altair line printer, data register `03h` — measured, not reasoned: Altair Hard Disk BASIC answering `LINEPRINTER? C` initialises with `OUT 03h←11h` / `OUT 02h←00h` and then sends one 7-bit ASCII character per byte to `03h`. `off` = a booted disk has no

Key	Type	Default	Range / validation	Description
				printer. The status register is deliberately not emulated: an unclaimed port reads `0xFF` here and every period convention reads a high bit as ready. The board also carries an **auto-line-feed** setting, the DIP switch real Centronics interfaces had, and for the same reason: a bare CR that does not advance the paper is how *overstrike* makes bold and underline, while a bare CR that does advance is how much other software ends a line, and only the hardware can settle which. Measured for `altair_c`: two `LPRINT`s send `ALPHA<CR>BETA<CR>` and no line feed at all, so the switch is on — otherwise a whole report prints on one line. A CR LF pair sent to it has the line feed absorbed rather than double-spacing. See §8.11.
`cpm_emu_uart`	string	`rc2014_1b`	a profile name	How the emulated CP/M's comms software reaches the virtual modem. Default `rc2014_1b` — the port the bundled EGT8080 expects. `off` = no modem; a machine/port profile: `rc2014_1a`…`rc2014_2b` (RC2014/ RomWBW Z80 SIO/2, 0x80–0x87), `altair_2sio1`/`altair_2sio2` (Altair 88-2SIO, 0x10–0x13), `altair_sio` (Altair 88-SIO, 0x00/0x01); `aux` (BDOS AUX: device, funcs 3/4, for SC126/RomWBW); or `hbios_1`/`hbios_2` (RomWBW HBIOS serial unit 1/2, reached by `RST 8` — for software built for RomWBW rather than a bare UART, e.g. the QTERM `h` builds). With a port selected, the guest dials out with `AT` commands (`ATD A`/`B` to the gateway's serial ports, `ATDT host:port` to a network host). See §8.11.
`disable_gateway_connections`	bool	`false`	true/false	Refuse connections from this network’s router (the default route’s next hop, queried from the OS — see §7.6; falls back to `*.*.*.1` if the query fails) while the private-IP allowlist applies. Off by default — traffic from that address is as often an administrator’s machine or hairpinned LAN traffic as it is something forwarded from outside, and refusing it

Key	Type	Default	Range / validation	Description
				left `disable_ip_safety` as the only way in. Loopback (`127.0.0.1`) is exempt either way. Takes effect on the next connection, no restart.
`disable_ip_safety`	bool	`false`	true/false	Remove the private-IP allowlist. For telnet this only matters when `security_enabled` is `false` (auth otherwise gates any IP). The configuration web server keeps the allowlist even with login on (its page shows the password/API key), so this flag is the only way to reach the web UI from a non-private IP. Lets any source address connect — use only on isolated networks; for internet-exposed telnet, enable `security_enabled` instead.
`enable_console`	bool	`true`	true/false	Show GUI window at startup. Set false for headless operation.
`gateway_debug`	bool	`false`	true/false	Byte-level trace of SSH/Telnet gateway sessions, and of every key typed at a session prompt, for diagnosing input corruption / terminal-translation issues. **Passwords are excluded** — the trace is suppressed for the length of any password prompt, so this cannot write a login, an SSH gateway password or an API key into the log (it did until 0.9.3; outbound tracing was never affected, since a password prompt echoes *). Read fresh each session, so toggling takes effect on the next gateway session. The `EGATEWAY_GATEWAY_DEBUG` environment variable forces it on regardless. See §16.10.
`gateway_term_height`	u16	`0`	0 = auto	Rows reported to the remote host by the SSH and Telnet gateways. `0` = automatic, resolved independently of the width — see `gateway_term_width`.
`gateway_term_width`	u16	`0`	0 = auto	Columns reported to the remote host by the SSH gateway (in its PTY request) and the Telnet Gateway (via NAWS). `0` = automatic: the size the local client negotiated by NAWS, else the terminal-type default (40×25 PETSCII, 80×24

Key	Type	Default	Range / validation	Description
				ANSI/ASCII). Set it when the automatic answer is wrong, which it is for most retro clients: terminal *type* does not imply terminal *width*. A C64 running CCGMS in ASCII mode sends `0x08` for backspace, so it is detected as ANSI and would be told it has 80 columns for a physically 40-column screen; CCGMS's soft 80-column mode is the same mistake in reverse under PETSCII. WiFi modems and tcpser do not send NAWS on the C64's behalf, so only the operator can say. When the remote has the wrong width, everything past the real margin — line wrap, backspace, history recall, tab completion — is drawn in the wrong place. Set from the telnet Gateway Configuration menu (`W/R`), or Server → More in the GUI and web UI. Takes effect on the next gateway connection, no restart. See §16.14.
`groq_api_key`	string	empty	any	Groq API key. Empty disables AI chat. Stored plaintext — protect the file.
`gui_window_geometry`	string	empty	`x,y,w,h`	Last desktop-GUI window position and size, written automatically so the window reopens where you left it. **Auto-managed — not a setting**: it appears in no configuration UI and is normally never hand-edited. Empty = default size and window-manager placement.
`idle_timeout_secs`	u64	`900`	0 = disabled	Per-session idle timeout. Applies to every prompt, including login.
`kermit_server_enabled`	bool	`false`	true/false	Bind a dedicated TCP listener that drops every accepted connection straight into Kermit server mode. Bypasses the telnet menu and auth gate (same risk profile as `allow_atdt_kermit`). See chapter 8 and the standalone `kermit.html` reference. The full `kermit_*` family is documented in `kermit.html`.
`kermit_server_port`	u16	`2424`	≥ 1	TCP port for the standalone Kermit-server listener.
`log_file`	string			Active log file, relative to the working directory unless absolute. Created owner-

Key	Type	Default	Range / validation	Description
		`ethernetgateway.log`	path, or empty to disable	only (mode 0600 on Unix) because log lines name hosts, ports and usernames. Reopened in *append* mode, so a restart extends the log rather than discarding the previous run. An empty value switches file logging off just as `log_to_file = false` does.
`log_max_files`	u32	`5`	0 = keep none	How many rotated generations to keep (`.1`…`.5` with the default). Anything older is **deleted** — that deletion is what bounds disk use. `0` keeps no history at all: the active file is truncated instead of rotated.
`log_max_size_kb`	u64	`1024`	0 = never rotate	Rotate once a write would take the active file past this size. Worst-case disk use is `log_max_size_kb × (log_max_files + 1)` — 6 MB with the defaults — and every UI shows that computed total. `0` disables size-based rotation, which lets the file grow without bound; the startup line and each config UI say so.
`log_to_file`	bool	`true`	true/false	**On by default.** Mirror the log to `log_file` in addition to stderr and the GUI/web console buffer. Nothing is rate-limited: `verbose` legitimately logs per protocol block, so a lines-per-second ceiling would discard exactly the lines an operator turned verbose on to see — growth is bounded by rotation, which loses *old* data instead of current data. Under systemd this duplicates journald's own rotation; set `false` if you would rather rely on the journal alone. Changes take effect on the next server restart.
`max_sessions`	usize	`50`	≥ 1	Concurrent telnet-session cap.
`password`	string	`changeme`	non-empty	Unified login password — shared by telnet, SSH, and the configuration web UI. Stored plaintext.
`punter_block_size`	u16	`255`	8–255	Punter C1 total block size in bytes (7-byte header included; 248-byte payload at the default). Lower toward ~40 on a noisy line to shrink the retransmit unit.
`punter_block_timeout`	u64	`20`	≥ 1 (seconds)	Per-block read timeout once a Punter transfer is live.

Key	Type	Default	Range / validation	Description
`punter_max_retries`	u32	`10`	≥ 1	Retry cap per block (BAD / SYN re-sends) before the session aborts.
`punter_max_bad_rounds`	u32	`30`	≥ 1	Consecutive corrupt-block resend rounds tolerated before giving up (kept higher than `punter_max_retries`; a real C64 peer never caps its resends, so a low value strands it).
`punter_hangup_on_failure`	bool	`false`	true / false	Drop carrier on give-up so a stranded C64 (C1 has no in-band abort) sees loss-of-carrier instead of hanging. Ends the whole session; off by default.
`punter_negotiation_retry_interval`	u64	`5`	≥ 1 (seconds)	Gap between handshake-code re-sends during Punter negotiation.
`punter_negotiation_timeout`	u64	`45`	≥ 1 (seconds)	Seconds to wait for the Punter file-type handshake before giving up.
`security_enabled`	bool	`false`	true/false	Require login on telnet and SSH. Also required to accept public-IP telnet.
`setup_wizard_completed`	bool	`false`	true/false	Whether the desktop GUI's first-run setup wizard has run. The wizard appears only while this is `false`, and it is set to `true` when the wizard is completed *or* skipped. Deliberately asymmetric on read: an existing config file that *lacks* the key resolves to `true`, so upgrading an already-configured gateway is never dropped into the wizard — only a config file the gateway creates itself carries `false`. Set it back to `false` (or use *Run setup wizard…* in the GUI) to walk through setup again. No server behaviour reads it. See §3.1.
`open_screen_after_restart`	bool	`false`	true/false	A one-shot marker rather than a setting, and shown on no configuration screen. Enabling the web server from the desktop's **VDM / Dazzler** button restarts the gateway; this survives that restart so the new window can open the screen that was asked for, and is cleared as soon as it is read — a marker that outlived a failed attempt would open a browser at every launch afterwards. See §8.11.
`serial_x_baud`	u32	`9600`	≥ 300	Serial baud rate. *x* = `a` or `b` (per port).

Key	Type	Default	Range / validation	Description
`serial_x_databits`	u8	`8`	5, 6, 7, or 8	Data bits.
`serial_x_dcd_mode`	u8	`1`	0 or 1	Saved `AT&C` DCD mode.
`serial_x_dtr_mode`	u8	`0`	0–3	Saved `AT&D` DTR-handling mode.
`serial_x_echo`	bool	`true`	true/false	Saved `ATE` echo state.
`serial_x_enabled`	bool	`false`	true/false	Activate this port. Pairs with `serial_x_mode`. Each port enables independently.
`serial_x_flow_mode`	u8	`0`	0–4	Saved `AT&K` modem-layer flow mode.
`serial_x_mode`	string	`modem`	`modem` or `console`	`modem` = Hayes AT emulator on this port. `console` = expose this port via **G Serial Gateway** as a telnet-serial bridge (§9.13).
`serial_x_flowcontrol`	string	`none`	none / hardware / software	Wire-level flow control for this port.
`serial_x_parity`	string	`none`	none / odd / even	Parity bit.
`serial_x_petscii_translate`	bool	`false`	true/false	Saved `AT+PETSCII` state — PETSCII translation on direct-TCP dials (Appendix B). Editable from the AT command and the telnet / web / GUI config surfaces.
`serial_x_port`	string	empty	OS-specific path	Serial device. `/dev/ttyUSB0`, `COM3`, etc.
`serial_x_quiet`	bool	`false`	true/false	Saved `ATQ` quiet mode.
`cpm_emu_echo`	bool	`true`	`true/false`	CP/M virtual modem saved profile: command echo (`ATE`). Written by `AT&W` inside the emulator, reloaded on power-up and `ATZ`.
`cpm_emu_verbose`	bool	`true`	`true/false`	Saved profile: word result codes rather than digits (`ATV`).
`cpm_emu_quiet`	bool	`false`	`true/false`	Saved profile: suppress result codes (`ATQ`).
`cpm_emu_x_code`	u8	`4`	0–4	Saved profile: result-code level (`ATX`). Out-of-range values are clamped on load.
`cpm_emu_dcd_mode`	u8	`1`	0–1	Saved profile: DCD handling (`AT&C`). 0 = forced on, 1 = follows carrier.
`cpm_emu_s_regs`	string	empty		Saved profile: S0–S27, comma-separated. Empty means the power-on registers; a

Key	Type	Default	Range / validation	Description
			CSV of up to 28 u8 values	missing or unparsable field keeps that register's default.
`serial_x_s_regs`	string	see note	CSV of 27 u8 values	S0–S26, comma-separated. Default: `5,0,43,13,10,8,2,15,2,6,14,95,50,0,0,0,0,0,0,0,0,0,0,0,5,1`.
`serial_x_stopbits`	u8	`1`	1 or 2	Stop bits.
`serial_x_stored_0` … `serial_x_stored_3`	string	empty	any	Stored phone-number slots, per-port.
`serial_x_verbose`	bool	`true`	true/false	Saved `ATV` verbose/numeric mode.
`serial_x_x_code`	u8	`4`	0–4	Saved `ATX` result-code level.
`ssh_enabled`	bool	`false`	true/false	Run the SSH server.
`ssh_gateway_auth`	string	`password`	`key` or `password`	Auth mode for the outbound SSH Gateway. See §10.1.
`ssh_port`	u16	`2222`	≥ 1	SSH listen port.
`telnet_enabled`	bool	`true`	true/false	Run the telnet server.
`telnet_gateway_negotiate`	bool	`false`	true/false	Outbound telnet gateway: send proactive TTYPE / NAWS / DO ECHO offers. Inert while `telnet_gateway_raw` is set (raw TCP carries no IAC layer); the control is greyed in that mode in every config surface, and saving then preserves rather than clears it.
`telnet_gateway_raw`	bool	`false`	true/false	Outbound telnet gateway: raw-TCP escape hatch, no IAC layer.
`telnet_port`	u16	`2323`	≥ 1	Telnet listen port.
`transfer_dir`	string	`transfer`	non-empty	Base directory for file transfers.
`place_bundled_terminals`	bool	`true`	`true`, `false`	Write `EGT8080.COM` and `EGT80.COM` into `transfer_dir` and onto CP/M drive A: when missing. Never overwrites a file that is already there.
`username`	string	`admin`	non-empty	Unified login username — shared by telnet, SSH, and the configuration web UI.
`verbose`	bool	`false`	true/false	Detailed per-block / per-subpacket protocol logging across XMODEM, YMODEM, ZMODEM, Kermit, and Punter.

Key	Type	Default	Range / validation	Description
weather_location	string	empty	city / postal code	Last-used location, worldwide (migrates legacy weather_zip). Updated automatically.
weather_units	string	auto	auto / us / metric	Weather display units.
web_enabled	bool	false	true/false	Run the configuration web server on web_port. Off by default; mirrors the GUI settings page in a browser. See §5.6 / §6.4.
web_port	u16	8080	≥ 1	Web-server listen port.
xmodem_block_timeout	u64	20	≥ 1 (seconds)	Per-block timeout during transfer.
xmodem_max_retries	usize	10	≥ 1	NAK retry cap per block.
xmodem_negotiation_retry_interval	u64	7	≥ 1 (seconds)	Gap between C/NAK pokes during the XMODEM/YMODEM handshake.
xmodem_negotiation_timeout	u64	45	≥ 1 (seconds)	Initial negotiation timeout for XMODEM/YMODEM send/receive.
zmodem_frame_timeout	u64	30	≥ 1 (seconds)	Per-frame read timeout during a ZMODEM transfer.
zmodem_max_retries	u32	10	≥ 1	Retry cap for ZRQINIT / ZRPOS / ZDATA frames.
zmodem_negotiation_retry_interval	u64	5	≥ 1 (seconds)	Gap between ZRINIT / ZRQINIT re-sends during the ZMODEM handshake.
zmodem_negotiation_timeout	u64	45	≥ 1 (seconds)	Initial ZRQINIT / ZRINIT handshake timeout.

Appendix B — AT Command Reference

Quick reference for every AT command the emulator understands. See chapter 9 for detailed explanations. All commands are case-insensitive. Multiple commands may be combined on a single `AT` line — see section 9.10 for the chaining rules (`ATE0Q1V0`, `ATE0DT host`, …).

B.1 Core Commands

Command	Effect
`AT`	Attention. Returns `OK`.
`AT?`	Display AT command help.
`ATZ`	Reset to stored profile (saved by `AT&W`).
`AT&F`	Factory reset to gateway-friendly defaults.
`AT&W` / `AT&W0`	Save current settings to `egateway.conf`.
`AT&V`	Display current configuration.
`ATH` / `ATH0`	Hang up any active connection.
`ATA`	Answer incoming ring.
`ATO` / `ATO0`	Return to online mode (after `+++` escape).
`A/`	Repeat last command (no AT prefix, no CR).
`+++`	Escape to command mode (guard-time gated).

B.2 Identification

Command	Response
`ATI` / `ATI0`	Product ID and version.
`ATI1`	ROM checksum (`000`).
`ATI2`	ROM self-test (`OK`).
`ATI3`	Firmware identifier.
`ATI4`	Product description.
`ATI5`	OEM / country code (`B00`).
`ATI6`	Link diagnostics.
`ATI7`	Product info.

B.3 Echo, Verbosity, Quiet, Result Codes

Command	Effect
`ATE0`	Echo off.
`ATE1` (default)	Echo on.
`ATV0`	Numeric result codes.
`ATV1` (default)	Verbose (text) result codes.
`ATQ0` (default)	Result codes enabled.
`ATQ1`	Quiet mode — all results suppressed.
`ATX0`	Basic result codes only.
`ATX1`	Basic plus baud rate in CONNECT.
`ATX2`	Adds NO DIALTONE detection.
`ATX3`	Adds BUSY detection.
`ATX4` (default)	Full extended result codes.

B.4 Hardware Pin State

Command	Effect
`AT&C0`	DCD always asserted.
`AT&C1` (Hayes default)	DCD reflects carrier state.
`AT&D0` (*gateway default*)	Ignore DTR.
`AT&D1`	DTR drop → command mode.
`AT&D2` (Hayes default)	DTR drop → hang up.
`AT&D3`	DTR drop → hang up + reset.
`AT&K0` (*gateway default*)	No modem-layer flow control.
`AT&K1`	Auto-detect (stored only; no wire-level effect).
`AT&K3`	Bidirectional RTS/CTS.
`AT&K4`	Bidirectional XON/XOFF.

B.5 Dialing

Command	Effect
`ATD target`	Dial *target*. Bare `D` (no T/P).
`ATDT host:port`	Tone-dial. Default for all TCP dials.
`ATDP host:port`	Pulse-dial (identical to `ATDT` over TCP).
`ATDT ethernetgateway`	Built-in shortcut: connect to this gateway's main menu.
`ATDT KERMIT`	Drop into Kermit server mode (requires `allow_atdt_kermit = true`). Aliases: `kermit`, `kermit-server`, `kermit server`.
`ATDL`	Redial last number.
`ATDS` / `ATDS0`	Dial stored number in slot 0.
`ATDS1` … `ATDS3`	Dial stored number in slots 1–3.
`AT&Zn=s`	Store phone number or host in slot *n* (0–3). Preserves hostname case.

Dial-string modifiers (phone-number context only, except `;` which applies to hostnames too):

Modifier	Effect
`,`	Pause for S8 seconds per comma.
`W`	Wait for dial tone (adds S6 seconds).
`;`	Return to command mode after connect.
`*`, `#`	DTMF digits, preserved for dial-up lookup.
`P`, `T`, `@`, `!`	Ignored.

B.6 S-Registers

Command	Effect
`ATS?`	Show S-register help.
`ATSn?`	Query S-register *n* (returns 3-digit decimal).
`ATSn=v`	Set S-register *n* to *v* (0–255).

Full register list is in chapter 9 (section 9.7).

B.7 Unknown Commands

Unrecognised AT commands return `OK` (silent-accept fallback). This covers commands the emulator has no hardware to implement: `ATB`, `ATC`, `ATL` (speaker loudness), `ATM` (speaker mute), `AT&B`, `AT&G`, `AT&J`, `AT&S`, `AT&T`, `AT&Y`, and others. The intent is to let legacy init strings run to completion rather than halt mid-setup with `ERROR`.

Disclaimer

This software is provided on an "as is" basis, without warranties of any kind, express or implied. Use at your own risk. The author is not responsible for any data loss, security breaches, or damages resulting from the use of this software. The user is solely responsible for securing their own network, credentials, and data. Telnet is an inherently insecure protocol — do not use this software on untrusted networks.

Portions of this project were developed with the assistance of AI tools, principally Claude Code (Anthropic) with ChatGPT (OpenAI) alongside it.
